



Universidad de Oviedo

DevorApp: recomendando restaurantes según tu ubicación, gustos y momento

ESCUELA POLITÉCNICA DE INGENIERÍA

Grado en Ingeniería Informática en Tecnologías de la
Información

Autor:

Alejandro Torre Llorente

Tutor:

D. Cristian Augusto

Dña. Ana Rodríguez Fernández

D. Pablo García Ullán

julio de 2026

DECLARACIÓN DE AUTORÍA

D. Alejandro Torre Llorente con DNI 58432430X, estudiante del Grado en Ingeniería Informática en Tecnologías de la Información en la Escuela Politécnica de Ingeniería de Gijón de la Universidad de Oviedo, realizado durante el curso 2025-2026.

DECLARO QUE:

El Trabajo Fin de Grado denominado “DevorApp: recomendando restaurantes según tu ubicación, gustos y momento” ha sido desarrollado respetando la propiedad intelectual (citando las fuentes bibliográficas utilizadas en la redacción de dicho trabajo), así como cualquier otro derecho, por ejemplo, el de imagen, que pudiese estar sujeto a protección del copyright.

Durante el desarrollo de este trabajo, se han utilizado correctores generativos basados en inteligencia artificial (GPT-4o, Gemini...) para la generación de texto y código, siempre bajo la supervisión y la posterior revisión del autor. El autor asume la responsabilidad de cualquier error que pueda haberse introducido durante el proceso de revisión.

En virtud de esta declaración afirmo que este trabajo es inédito y de mi autoría, por lo que me responsabilizo del contenido, veracidad y alcance del mismo, y asumo las consecuencias administrativas y jurídicas que se deriven en caso de incumplimiento de esta declaración.

Y para que así conste, firmo la presente declaración:

Gijón, lunes, 13 de julio de 2026

AGRADECIMIENTOS

Quiero expresar mi agradecimiento a todas las personas que han contribuido, directa o indirectamente, a la realización de este Trabajo de Fin de Grado. En primer lugar, deseo agradecer especialmente a mi tutor, Cristian Augusto Alonso, por su orientación, disponibilidad y paciencia durante el desarrollo del proyecto. Sus consejos y observaciones han sido claves para avanzar en las distintas etapas del trabajo y mejorar su resultado final.

También quiero agradecer a mi familia por su apoyo incondicional, comprensión y la confianza depositada en mí a lo largo de todo el proceso. Su motivación ha sido esencial para afrontar las dificultades y continuar avanzando hasta el final. A mis amigos y compañeros de la carrera, les agradezco las palabras de ánimo, las ideas compartidas y el apoyo recibido durante estos años. Su compañía ha contribuido a hacer más llevadero tanto el desarrollo de este trabajo como el conjunto de toda la etapa universitaria.

Asimismo, agradezco a la Universidad de Oviedo y a la empresa HP SCDS la oportunidad de desarrollar este proyecto y el acceso a los recursos necesarios para su realización. Del mismo modo, quiero reconocer la ayuda de todas las personas que han aportado sus conocimientos, experiencia o sugerencias. Finalmente, agradezco a todas aquellas personas que, de una forma u otra, han contribuido a que este trabajo haya podido llevarse a cabo.

CONTENIDOS

Capítulo 1	Descripción general del trabajo	6
1.1.	Resumen	6
1.2.	Palabras clave	6
1.3.	Abstract	7
1.4.	Keywords	7
Capítulo 2	Introducción.....	8
2.1.	Introducción	8
2.2.	Contexto	9
2.2.1.	Evaluación de las herramientas actuales.....	9
2.3.	Objetivos	10
2.4.	Alcance.....	11
Capítulo 3	Requisitos de usuario	12
Capítulo 4	Planificación y Gestión.....	15
4.1.	Planificación del Proyecto.....	15
4.1.1.	Identificación de los interesados	15
4.1.2.	Planificación inicial.....	15
4.1.3.	Riesgos	22
4.1.4.	Presupuesto inicial	25
4.2.	Ejecución del Proyecto	28
4.2.1.	Incidencias.....	28
4.3.	Cierre del proyecto	29
4.3.1.	Planificación final	29
4.3.2.	Informe final de riesgos	30
4.3.3.	Presupuesto final de costes	30
Capítulo 5	Análisis de alternativas	34
5.1.	Lenguajes de programación frontend	34
5.2.	Lenguajes de programación backend	35
5.3.	Framework frontend	37
5.4.	Framework backend	38
5.5.	APIs para la búsqueda de restaurantes.....	38
5.6.	Tipo de base de datos	40
5.7.	Servidor de base de datos	41
5.8.	Gestión de autenticación.....	42
5.9.	Motor de recomendación.....	43

5.10.	Despliegue.....	45
Capítulo 6	Requisitos del Sistema.....	47
6.1.	Requisitos Funcionales	47
6.2.	Requisitos no funcionales.....	55
6.2.1.	Modelo de datos del dominio	57
6.2.2.	Interfaz de usuario	59
Capítulo 7	Diseño	71
7.1.	Diseño de la arquitectura.....	71
7.2.	Diseño de detalle	75
7.3.	Diseño de pruebas	77
7.4.	Despliegue.....	89
Capítulo 8	Implementación	91
8.1.	Estructura de la aplicación	91
8.1.1.	Implementación del Backend.....	91
8.1.2.	Implementación del Frontend	92
8.1.3.	Implementación del Microservicio de IA	92
8.1.4.	Diagrama de clases	93
8.2.	Implementación de las pruebas.....	97
8.2.1.	Pruebas unitarias.....	97
8.2.2.	Pruebas de sistema	98
8.2.3.	Pruebas estáticas	102
8.3.	Despliegue.....	102
8.3.1.	Arquitectura de contenedores	102
8.3.2.	Orquestación de contenedores	104
8.3.3.	Integración y despliegue continuo	104
Capítulo 9	Manual de usuario	105
Capítulo 10	Conclusiones y Trabajo Futuro	113
10.1.	Conclusiones.....	113
10.2.	Trabajo Futuro	114
Capítulo 11	Bibliografía y Apéndices	115
11.1.	Referencias.....	115

ÍNDICE DE ILUSTRACIONES

Ilustración 4-1 Planificación inicial de los Sprints	16
Ilustración 4-2 Planificación inicial Sprint 1	17
Ilustración 4-3 Planificación inicial Sprint 2	17
Ilustración 4-4 Planificación inicial Sprint 3	18
Ilustración 4-5 Planificación inicial Sprint 4	18
Ilustración 4-6 Planificación inicial Sprint 5	19
Ilustración 4-7 Planificación inicial Sprint 6	19
Ilustración 4-8 Planificación inicial Sprint 7	20
Ilustración 4-9 Planificación inicial Sprint 8	21
Ilustración 4-10 Planificación inicial Sprint 9	21
Ilustración 4-11 Planificación final	29
Ilustración 6-1 Modelo de datos	57
Ilustración 8-1 Pruebas estáticas iniciales	102
Ilustración 8-2 Pruebas estáticas finales	102

ÍNDICE DE PROTOTIPOS

Prototipo 1 Inicio de sesión	59
Prototipo 2 Crear cuenta	60
Prototipo 3 Pantalla principal	61
Prototipo 4 Recomendación de restaurantes	62
Prototipo 5 Historial	63
Prototipo 6 Favoritos	64
Prototipo 7 Guardar para más tarde	65
Prototipo 8 Poner valoración	66
Prototipo 9 Reseñas de la comunidad	66
Prototipo 10 Mis valoraciones	67
Prototipo 11 Detalles de restaurante	68
Prototipo 12 Perfil de usuario	69

ÍNDICE DE TABLAS

Tabla 4-1 Planificación inicial: Sprints	16
Tabla 4-2 Planificación inicial Sprint 1	17
Tabla 4-3 Planificación inicial Sprint 2	17
Tabla 4-4 Planificación inicial Sprint 3	18
Tabla 4-5 Planificación inicial Sprint 4	18
Tabla 4-6 Planificación inicial Sprint 5	19
Tabla 4-7 Planificación inicial Sprint 6	19
Tabla 4-8 Planificación inicial Sprint 7	20
Tabla 4-9 Planificación inicial Sprint 8	21
Tabla 4-10 Planificación inicial Sprint 9	21
Tabla 4-11 Análisis de riesgos	23
Tabla 4-12 Capítulo 1: Costes de personal	26
Tabla 4-13 Capítulo 2: Infraestructura y servicios cloud	27

Tabla 4-14 Capítulo 3: Dispositivos y hardware de desarrollo	28
Tabla 4-15 Capítulo 4: licencias y herramientas software.....	28
Tabla 4-16 Presupuesto total	28
Tabla 4-17 Capítulo 1: Costes de personal	31
Tabla 4-18 Capítulo 2: Infraestructura y servicios cloud	33
Tabla 4-19 Capítulo 3: Dispositivos y hardware de desarrollo	33
Tabla 4-20 Capítulo 4: Licencias y herramientas software.....	33
Tabla 4-21 Presupuesto total	33
Tabla 5-1 Comparativa de lenguajes de programación de frontend	35
Tabla 5-2 Comparativa de lenguajes de programación de backend	36
Tabla 5-3 Comparativa de frameworks de frontend	37
Tabla 5-4 Comparativa de frameworks de backend.....	38
Tabla 5-5 Comparativa de APIs para la búsqueda de restaurantes.....	39
Tabla 5-6 Comparativa de tipos de bases de datos	41
Tabla 5-7 Comparativa de servidores de bases de datos	42
Tabla 5-8 Comparativa de gestión de autenticación.....	43
Tabla 5-9 Comparativa Motor de recomendación	44
Tabla 5-10 Comparativa Alternativas de despliegue	45
Tabla 7-1 Pruebas Inicio de sesión	79
Tabla 7-2 Pruebas Registro	81
Tabla 7-3 Pruebas Buscar restaurante.....	83
Tabla 7-4 Pruebas Valoraciones	84
Tabla 7-5 Pruebas Historial.....	85
Tabla 7-6 Pruebas Favoritos.....	86
Tabla 7-7 Pruebas Menú lateral.....	86
Tabla 7-8 Pruebas Perfil de usuario	88
Tabla 8-1 Casos de prueba inicio de sesión	99
Tabla 8-2 Casos de prueba registro	99
Tabla 8-3 Casos de prueba búsqueda de restaurantes.....	99
Tabla 8-4 Casos de prueba valoraciones	100
Tabla 8-5 Casos de prueba favoritos	100
Tabla 8-6 Casos de prueba menú lateral	100
Tabla 8-7 Casos de prueba perfil de usuario	100
Tabla 8-8 Pruebas E2E.....	100
Tabla 8-9 Pruebas API	101

ÍNDICE DE FRAGMENTOS DE CÓDIGO

Código 1 Ejemplo código JavaScript	34
Código 2 Ejemplo código TypeScript	34
Código 3 Ejemplo código Dart	34
Código 4 Ejemplo código Elm	34
Código 5 Ejemplo código Python	36
Código 6 Ejemplo código Java	36
Código 7 Ejemplo código TypeScript.....	36
Código 8 Ejemplo código Go	36
Código 9 Ejemplo SQL.....	40

Código 10 Ejemplo NoSQL	40
-------------------------------	----

ÍNDICE DE FIGURAS

Figura 6-1 Story mapping 1/2	47
Figura 6-2 Story mapping 2/2	48
Figura 6-3 Diagrama de transiciones	70
Figura 7-1 Arquitectura general	71
Figura 7-2 Arquitectura Backend	72
Figura 7-3 Arquitectura Frontend	73
Figura 7-4 Arquitectura Microservicio	73
Figura 7-5 Flujo de información	77
Figura 7-6 Diagrama despliegue	90
Figura 8-1 Diagrama de clases backend	94
Figura 8-2 Diagrama de clases frontend	95
Figura 8-3 Diagrama de clases microservicio de IA	96

ÍNDICE DE PANTALLAS

Pantalla 1 Inicio de sesión	106
Pantalla 2 Registro 1	106
Pantalla 3 Registro 2	106
Pantalla 4 Página principal	107
Pantalla 5 Menú lateral	107
Pantalla 6 Buscador de restaurantes	108
Pantalla 7 Resultados de búsqueda	108
Pantalla 8 Detalles de restaurante	108
Pantalla 9 Listas de Favoritos	109
Pantalla 10 Restaurantes favoritos	109
Pantalla 11 Historial	110
Pantalla 12 Guardar para más tarde	110
Pantalla 13 Lista de valoraciones	111
Pantalla 14 Editar reseña	111
Pantalla 15 Perfil de usuario	112
Pantalla 16 Zona de peligro	112

Capítulo 1 DESCRIPCIÓN GENERAL DEL TRABAJO

1.1. RESUMEN

En los últimos años, el fenómeno de la globalización ha incrementado significativamente la diversidad de la oferta gastronómica en la mayoría de las ciudades. Junto a los establecimientos tradicionales de cada región, han surgido numerosos restaurantes de diversas culturas y estilos. Aunque disponer de una amplia diversidad de opciones puede resultar positivo, en muchas ocasiones el exceso de alternativas dificulta la toma de decisiones.

En este contexto, los usuarios recurren con frecuencia a plataformas digitales para facilitar su elección. Aplicaciones como TripAdvisor, TheFork o Google Maps muestran a los usuarios las opciones mejor valoradas y clasificaciones de establecimientos. No obstante, las recomendaciones genéricas suelen no ajustarse a los gustos o necesidades del momento, lo que puede provocar experiencias poco satisfactorias.

La principal motivación de este proyecto radica en la necesidad de reducir la complejidad de la toma de decisiones en un entorno con una amplia oferta de opciones. Mediante el desarrollo de esta aplicación, se pretende aportar una herramienta que facilite y agilice la elección de restaurantes en el día a día, incrementando así la satisfacción del usuario.

Este Trabajo de Fin de Grado, desarrollado en el marco de una colaboración con una empresa cuyo nombre se omite (en adelante, “la empresa”) tiene como objetivo el diseño e implementación de un sistema de recomendación de restaurantes personalizado. La propuesta plantea el desarrollo de una aplicación web multiplataforma que permita al usuario no solo buscar restaurantes según una serie de criterios, sino que además se adapte a sus preferencias y gustos. La aplicación será capaz de generar sugerencias a partir de características como la ubicación, el rango de precios o las preferencias alimentarias del usuario. Para ello, se implementará un modelo de recomendación que permita analizar dichas variables y ofrecer resultados ajustados al contexto. Además, la aplicación dispondrá de un mecanismo de aprendizaje basado en el historial, para así garantizar mejores experiencias de manera progresiva.

1.2. PALABRAS CLAVE

Informática, Sistemas de Información, Ingeniería del Software, Desarrollo, Aplicación Web Multiplataforma, Inteligencia Artificial, Sistemas de Recomendación.

1.3. ABSTRACT

In recent years, globalization has significantly increased the diversity of gastronomic offerings in most cities. Alongside the traditional establishments of each region, numerous restaurants from diverse cultures and styles have emerged. Although a wide range of options can be positive, on many occasions, the excess of alternatives makes decision-making difficult.

In this context, users frequently turn to digital platforms to facilitate their choice. Applications such as TripAdvisor, TheFork, or Google Maps show users the highest-rated options and classifications of establishments. However, generic recommendations often fail to adapt to the tastes or needs of the moment, leading to unsatisfactory experiences.

The main motivation for this project is the need to reduce the complexity of decision-making in an environment with a wide range of options. The aim of this application is to provide a tool that facilitates and streamlines the daily selection of restaurants, thereby increasing user satisfaction.

This Bachelor's Thesis, developed within the framework of a collaboration with a company whose name is omitted (hereinafter "the company"), aims to design and implement a personalized restaurant recommendation system. The proposal outlines the development of a multiplatform web application that allows users not only to search for restaurants based on a set of criteria but also to adapt the search to their preferences and tastes. The application will generate suggestions based on factors such as location, price range, and the user's dietary preferences. To achieve this, a recommendation model will be implemented to analyze these variables and provide context-specific results. Furthermore, the application will feature a history-based learning mechanism to progressively improve the experience.

1.4. KEYWORDS

Computer Science, Information Systems, Software Engineering, Development, Mobile Application, Artificial Intelligence, Recommendation System.

Capítulo 2 INTRODUCCIÓN

2.1. INTRODUCCIÓN

Este Trabajo de Fin de Grado expone el ciclo de vida completo de diseño, desarrollo, validación y despliegue de *DevorApp*, una plataforma de software orientada a la recomendación personalizada de restaurantes. El proyecto nace de la necesidad de mitigar la sobrecarga de información y optimizar la toma de decisiones de los usuarios, combinando técnicas clásicas de persistencia con modelos de aprendizaje. Para ello, se empleará un sistema de recomendación que considerará distintos factores relevantes, como la ubicación del usuario, sus preferencias alimentarias o el rango de precios deseado.

La presencia en aplicaciones y plataformas de restaurantes influye directamente en la captación de clientes. Diversos estudios señalan que los comentarios online influyen en el desempeño financiero de los restaurantes, al afectar tanto a la confianza del consumidor como a su intención de visita. En consecuencia, una buena reputación digital puede traducirse en más clientes y un mayor volumen de negocio, mientras que una valoración negativa puede frenar la elección del establecimiento [1]. El impacto de esta huella digital repercute drásticamente en el volumen de negocio de los establecimientos. Investigaciones en el sector demuestran que un incremento de una sola estrella en la valoración de plataformas como Yelp o GoogleMaps puede asociarse con un aumento de entre el 5% y el 9% en la facturación [2]. Asimismo, la gestión activa de un perfil optimizado genera una validación social que aporta mayor credibilidad que la publicidad de pago. Mientras que la publicidad tradicional solo ofrece visibilidad inicial, un perfil con buenas valoraciones y reseñas constructivas genera seguridad en el usuario, facilitando su elección y aumentando las visitas al local sin necesidad de inversiones económicas adicionales [3], [4]. Estos datos muestran que las aplicaciones no solo facilitan la elección del usuario, sino que también pueden favorecer la visibilidad y la afluencia de clientes a los establecimientos, convirtiéndose en una herramienta de apoyo comercial para los restaurantes.

Además del desarrollo funcional de la aplicación, este trabajo abarca el análisis, diseño, implementación y validación de la solución propuesta, y evalúa su capacidad para mejorar el proceso de toma de decisiones de los usuarios al seleccionar un establecimiento.

El proyecto se apoya en un stack tecnológico maduro (React, FastAPI, PostgreSQL y Keras) consolidado mediante un *pipeline* de integración continua. Este flujo automatiza la ejecución sistemática de los conjuntos de pruebas y el despliegue de la aplicación ante cada cambio del código.

La memoria se estructura en varios capítulos. En primer lugar, se presenta el contexto y situación actual relacionada con las aplicaciones de restauración existentes. Posteriormente, se detallan los objetivos y requisitos identificados. A continuación, se describe el análisis de alternativas de las tecnologías usadas en el desarrollo del proyecto, seguido del diseño e implementación de la solución. Finalmente, se exponen las pruebas realizadas, los resultados obtenidos y las conclusiones derivadas del trabajo.

2.2. CONTEXTO

En la actualidad, los consumidores se enfrentan a una sobrecarga de información al intentar tomar decisiones cotidianas, como elegir un restaurante. A pesar de la gran popularidad de las diferentes plataformas de reseñas y mapas, muchos usuarios experimentan lo que se conoce como la “paradoja de la elección” [5]. Disponer de un amplio catálogo de opciones dificulta, en lugar de facilitar, la toma de decisiones rápida y satisfactoria. La falta de herramientas que filtren y seleccionen el establecimiento adecuado, adaptándose al contexto, impide que se satisfagan eficazmente las necesidades del consumidor.

El uso de plataformas generales como Google Maps o TripAdvisor suele resultar ineficiente y genérico. Al basarse en los promedios estadísticos de los usuarios, las preferencias generales pueden ser radicalmente diferentes a las del individuo actual. Esto conlleva problemas como la inversión excesiva de tiempo comparando reseñas y establecimientos o una experiencia poco satisfactoria al elegir un local que no encaja con las necesidades del momento. Además, la interfaz de estas aplicaciones suele contener información irrelevante o publicidad, lo que supone una barrera para el usuario que busca una solución directa.

En este contexto, surge la necesidad de una plataforma que ofrezca una solución accesible, intuitiva y personalizada para la selección de restaurantes. La implementación de un sistema de búsqueda basado en diferentes entradas proporcionadas por el usuario permitirá obtener resultados personalizados que se ajusten a las necesidades del momento. Al abordar estas problemáticas, se espera reducir la fatiga en la toma de decisiones y garantizar sugerencias relevantes, aumentando así el nivel de satisfacción del usuario.

El análisis de la situación actual revela que la mayoría de los usuarios requieren una solución tecnológica que simplifique la toma de decisiones mediante la personalización [6]. Con este proyecto, se pretende cubrir estas necesidades, brindando una herramienta que evolucione con el usuario y le permita descubrir restaurantes de manera efectiva y sin provocar una sobrecarga de alternativas.

2.2.1. Evaluación de las herramientas actuales

Actualmente, existen diversas plataformas ampliamente consolidadas para la búsqueda y recomendación de establecimientos, las cuales ofrecen funcionalidades similares a las planteadas en este Trabajo de Fin de Grado. Sin embargo, pese a su grado de madurez tecnológica, todas ellas presentan determinadas limitaciones en términos de personalización y adaptación a necesidades específicas o contextos concretos. Esto ofrece la oportunidad de implementar nuevas soluciones que se puedan ajustar a contextos más específicos. A continuación, se realiza una evaluación detallada de las herramientas actuales:

- **Google Maps:** Es una de las herramientas más utilizadas a nivel mundial para la localización y consulta de establecimientos. Su éxito se basa principalmente en el volumen de datos que ofrece al usuario, incluyendo establecimientos en prácticamente cualquier parte del mundo. No obstante, solo ofrece soluciones de carácter general, sin una especialización profunda según la situación del usuario.

Su sistema de recomendación se basa únicamente en valoraciones externas, pudiendo provocar sensaciones negativas debido a la dependencia total de las contribuciones del público.

- **TripAdvisor:** es una plataforma turística centrada en opiniones y valoraciones. Su principal fortaleza es su amplio sistema de reseñas y opiniones generadas por otros usuarios. Se caracteriza por un modelo de negocio fuertemente orientado al sector turístico, lo que a menudo sobrecarga la interfaz con publicidad y opciones que no se alinean con el uso cotidiano local. A diferencia de esta plataforma, este proyecto busca eliminar la sobrecarga de información y centrarse en recomendaciones que prioricen la necesidad inmediata y el contexto del usuario antes que los datos turísticos.
- **TheFork:** es una plataforma especializada en la reserva de restaurantes, ofreciendo gestión de disponibilidad, promociones y un sistema propio de pago y fidelización. A pesar de ser una potente herramienta, dispone de un ecosistema cerrado y está condicionado por un modelo de comisiones. Además, su modelo se centra más en el apartado económico que en el descubrimiento basado en el usuario. Este proyecto se diferencia de esta alternativa al plantear un sistema que prioriza un algoritmo de recomendación personalizado, lo que otorga un mayor control sobre los datos y una mayor adaptación a las preferencias de los consumidores.

2.3. OBJETIVOS

El proyecto tiene los siguientes objetivos:

- 1) Analizar los requisitos funcionales y no funcionales de la aplicación, identificando las necesidades clave de los usuarios en la búsqueda y selección personalizada de restaurantes.
 - 1.1) Investigar y seleccionar las fuentes de datos y APIs más adecuadas.
 - 1.2) Definir los casos de uso principales e identificar las restricciones del sistema.
- 2) Diseñar una arquitectura de software escalable y modular que permita la integración de APIs externas y la persistencia de datos.
 - 2.1) Diseñar un modelo de datos que asegure la integridad de la información y permita establecer relaciones complejas.
 - 2.2) Definir una interfaz de comunicación eficiente entre los distintos componentes del sistema.
- 3) Implementar un sistema de gestión de usuarios y personalización de la experiencia gastronómica.
 - 3.1) Desarrollar un sistema de gestión de perfiles, con funcionalidades como la creación de listas de favoritos y almacenamiento del historial
 - 3.2) Implementar un motor de recomendación capaz de procesar filtros avanzados.
- 4) Aplicar principios de usabilidad y experiencia de usuario (UX/UI) para garantizar una interfaz intuitiva que facilite la interacción y la toma de decisiones.
 - 4.1) Diseñar una interfaz que minimice la carga cognitiva del usuario durante el proceso de selección de restaurantes.
- 5) Validar el desempeño de la aplicación y la calidad de los datos obtenidos mediante pruebas con usuarios y evaluación de métricas de eficiencia y satisfacción.

- O 6) Garantizar la seguridad y fiabilidad de la aplicación a través de buenas prácticas en el desarrollo, como la gestión de autenticación y la validación de datos en el servidor.
- O 7) Planificar y gestionar el ciclo de vida del desarrollo de software mediante una metodología organizada que integre prácticas de integración y despliegue continuo y aseguramiento de la calidad. Esto incluye la automatización de pruebas unitarias y de integración, así como la implementación de flujos de despliegue automatizado en el entorno de producción.

La documentación debe describir el proceso de diseño e implementación del proyecto, siguiendo las buenas prácticas para la documentación de proyectos de software para garantizar trazabilidad, mantenibilidad y escalabilidad.

2.4. ALCANCE

El alcance del proyecto delimita el conjunto de funcionalidades y objetivos que se abordarán durante el desarrollo del sistema, estableciendo de forma clara qué se incluye y qué queda fuera del Trabajo de Fin de Grado. Su definición permite acotar el esfuerzo del proyecto y evitar desviaciones respecto a los objetivos inicialmente planteados.

En muchas ocasiones, la elección de un restaurante es una tarea complicada, sobre todo cuando hay muchas opciones y no hay una idea clara de qué se prefiere. Las recomendaciones genéricas suelen no ajustarse a los gustos o necesidades del momento, lo que puede derivar en experiencias poco satisfactorias. Este trabajo de fin de grado busca satisfacer la necesidad de encontrar el restaurante que mejor se adapte a los momentos precisos. Concretamente, este proyecto consiste en el desarrollo de una aplicación web multiplataforma, conectada a un servidor para la gestión de peticiones, que ofrezca un servicio de recomendación de restaurantes en función de las preferencias del usuario empleando un sistema de recomendación basado en un modelo de inteligencia artificial.

El usuario establecerá una serie de preferencias que, en combinación con el historial de restaurantes visitados y las experiencias de otros usuarios con gustos similares, permitirá al sistema generar una lista priorizada de los establecimientos que mejor se adapten a sus necesidades. Además, se encargará de gestionar a los diferentes usuarios que usen la aplicación y aprender según los gustos y preferencias de cada uno para mejorar próximas recomendaciones. Para ello, el sistema permitirá valorar y reseñar los establecimientos visitados. Estas valoraciones actuarán como fuente de datos para el aprendizaje del modelo. Asimismo, el usuario podrá consultar recomendaciones pasadas o establecimientos almacenados en diferentes listas.

El sistema se limita a ser una herramienta de búsqueda y recomendación. Se consideran fuera del alcance funcionalidades como introducir un método de reserva y pago por adelantado del restaurante seleccionado, o indicar mediante un mapa la ruta en tiempo real hasta el establecimiento de destino.

Capítulo 3 REQUISITOS DE USUARIO

En este apartado se describen los requisitos de usuario definidos para la aplicación desarrollada. Estos requisitos representan las necesidades y funcionalidades que el sistema debe ofrecer desde la perspectiva del usuario final, sirviendo como base para el diseño y desarrollo de la aplicación.

RQ 1) El usuario debe poder recibir recomendaciones de restaurantes basadas en la ubicación, el presupuesto y las preferencias.

RQ 1.1) El usuario podrá indicar su ubicación actual, una dirección o utilizar su ubicación almacenada en el sistema para realizar la búsqueda en diferentes formatos.

RQ 1.1.1) [Calle], [Ciudad], [País]

RQ 1.1.2) [Barrio], [Ciudad], [País]

RQ 1.1.3) [Código Postal], [Ciudad], [País]

RQ 1.1.4) [Ciudad], [País]

RQ 1.1.5) [País]

RQ 1.2) El usuario podrá especificar un rango de precios mediante un sistema de niveles cualitativos con símbolos de moneda.

RQ 1.2.1) La moneda en la que se indique el precio dependerá de la ubicación seleccionada para realizar la recomendación.

RQ 1.3) El usuario podrá indicar sus preferencias mediante una serie de palabras clave, provenientes de una serie de categorías disponibles [7].

RQ 2) El usuario podrá ordenar los resultados en función de diferentes aspectos:

RQ 2.1) Mejor valorados.

RQ 2.2) Más cercanos.

RQ 2.3) Más populares según el número de reseñas.

RQ 3) El usuario debe poder crear una cuenta, iniciar sesión y cerrar sesión.

RQ 3.1) El usuario debe poder registrarse de dos formas:

RQ 3.1.1) Usando Google. Además, deberá proporcionar:

RQ 3.1.1.1) Nombre de usuario único en la plataforma.

RQ 3.1.1.2) Ubicación preferida.

RQ 3.1.2) Proporcionando:

RQ 3.1.2.1) Nombre y apellidos.

RQ 3.1.2.2) Nombre de usuario único en la plataforma.

RQ 3.1.2.3) Correo electrónico.

RQ 3.1.2.4) Contraseña.

RQ 3.1.2.5) Ubicación preferida.

RQ 3.2) El usuario debe poder iniciar sesión en la plataforma:

RQ 3.2.1) Usando Google.

RQ 3.2.2) Especificando su nombre de usuario y su contraseña.

RQ 3.2.3) Especificando su correo electrónico y su contraseña.

RQ 4) El usuario debe poder acceder a su perfil para consultar y modificar sus datos.

- RQ 4.1) Podrá actualizar el correo electrónico.
- RQ 4.2) Podrá actualizar su contraseña.
- RQ 4.3) Podrá actualizar su ubicación preferida.
- RQ 4.4) Podrá actualizar su nombre y apellidos.
- RQ 5) El usuario debe poder guardar restaurantes en una lista de favoritos.
 - RQ 5.1) El usuario podrá gestionar sus listas de favoritos.
 - RQ 5.1.1) Por defecto, habrá una lista principal llamada “Favoritos”.
 - RQ 5.1.2) El usuario podrá crear, al mismo nivel que la lista por defecto, nuevas listas, eligiendo el nombre e icono de cada una.
 - RQ 5.1.3) Al añadir un restaurante, el usuario podrá seleccionar guardarlo en la lista que desee.
 - RQ 5.1.4) El usuario podrá eliminar restaurantes de las listas de favoritos.
 - RQ 5.1.5) El usuario podrá eliminar listas de favoritos y con ellas todos los restaurantes asociados a la misma.
- RQ 6) El usuario debe poder visualizar su historial restaurantes visitados.
 - RQ 6.1) Se mostrarán en una lista ordenada por fecha descendente los restaurantes recomendados.
 - RQ 6.2) Aparecerán solamente los restaurantes que se han visitado en el último año.
 - RQ 6.3) El usuario debe poder eliminar del historial los restaurantes visitados.
- RQ 7) El usuario podrá añadir restaurantes a una lista de “Guardar para más tarde”.
 - RQ 7.1) Existirá una única lista en la que se almacenarán los restaurantes añadidos.
 - RQ 7.2) Para eliminar los restaurantes de la lista:
 - RQ 7.2.1) Se pueden eliminar manualmente.
 - RQ 7.2.2) Se eliminarán al seleccionarlos desde la propia lista.
- RQ 8) El usuario podrá acceder a los datos del restaurante. La aplicación mostrará:
 - RQ 8.1) Nombre del restaurante.
 - RQ 8.2) Ubicación del restaurante.
 - RQ 8.3) Valoración del restaurante. Se mostrará:
 - RQ 8.3.1) Una puntuación de un máximo de 5 estrellas.
 - RQ 8.3.2) Al lado de la puntuación, el número de reseñas.
 - RQ 8.4) Número de teléfono (si dispone de él).
 - RQ 8.4.1) Se redirigirá al usuario a la aplicación de llamadas con el teléfono ya marcado.
 - RQ 8.5) Página web (si dispone de ella).
- RQ 9) El usuario debe poder indicar su opinión de la recomendación del sistema utilizando un sistema de puntuación.
 - RQ 9.1) El usuario podrá indicar mediante un sistema de 1 a 5 estrellas diferentes aspectos del local: calidad, precio, trato e higiene.
 - RQ 9.1.1) El usuario deberá rellenar todos los campos obligatoriamente.
 - RQ 9.2) De forma opcional, el usuario podrá añadir un comentario sobre el establecimiento para que el sistema lo tenga en cuenta en futuras recomendaciones.

- RQ 10)** El usuario debe poder visualizar una serie de sugerencias basadas en su ubicación.
- RQ 10.1)** Se mostrarán, del mismo modo que en la recomendación de restaurantes, los establecimientos más buscados por otros usuarios en la ubicación del usuario actual.
- RQ 11)** El usuario podrá visualizar las valoraciones de otros usuarios de la aplicación.
- RQ 11.1)** Al consultar información de un restaurante, habrá un apartado de “Reseñas de otros usuarios”.
- RQ 11.1.1)** Estas reseñas no afectarán a la valoración mostrada, sino que serán información adicional.
- RQ 11.1.2)** Aparecerán a modo de lista las diferentes valoraciones de los usuarios.
- RQ 11.1.3)** Se mostrará la nota puesta a cada uno de los cuatro criterios seguido del comentario textual, en caso de que exista.
- RQ 11.1.4)** Los usuarios que vean la reseña podrán indicar si les ha resultado útil mediante un botón de “Me gusta”.
- RQ 11.1.4.1)** Se mostrará el número de usuarios a los que le ha resultado útil la reseña.

Capítulo 4 PLANIFICACIÓN Y GESTIÓN

4.1. PLANIFICACIÓN DEL PROYECTO

La planificación constituye una fase fundamental para garantizar el correcto desarrollo del proyecto. En este apartado se definen los elementos necesarios para su correcta organización y seguimiento. En él se analizan los principales interesados, los riesgos asociados al desarrollo, la planificación temporal de las tareas y el presupuesto inicial estimado para la realización del trabajo.

4.1.1. Identificación de los interesados

Uno de los factores influyentes en el éxito de un proyecto es la correcta identificación de las necesidades de los interesados, definidos como todas aquellas personas u organizaciones que se ven afectadas directa o indirectamente por el desarrollo y despliegue del sistema. Para este proyecto se han identificado y clasificado los siguientes interesados:

Interesados internos

- **La empresa:** es la entidad externa que impulsa el proyecto. Actúa como el Product Owner principal. Su interés radica en que la aplicación cumpla con los requisitos solicitados. Además, al ser los promotores de la idea, se encarga de dictar las especificaciones técnicas del proyecto.
- **Desarrollador:** responsable del diseño y desarrollo de la aplicación. Su interés es asegurar la viabilidad técnica y el cumplimiento de los plazos.
- **Tutor del TFG:** representa la validación metodológica y académica del proyecto. Su interés es garantizar que se apliquen buenas prácticas durante el desarrollo del proyecto.

Interesados externos

- **Usuarios finales:** público objetivo de la aplicación. Son los usuarios que buscan agilizar la toma de decisiones al elegir un restaurante.
- **Restaurantes:** de forma indirecta, los locales proporcionados por la API de Google Places son interesados del proyecto, pues las recomendaciones de la aplicación llevarán clientes a sus negocios.

4.1.2. Planificación inicial

Para el desarrollo del proyecto se ha adoptado el framework Scrum, debido a su naturaleza ágil y su capacidad para adaptarse a cambios.

La planificación se organiza en sprints de dos semanas (10 días laborables), de miércoles a miércoles, coincidiendo con las tutorías quincenales. De este modo, la retroalimentación recibida actúa como punto de partida para la planificación del próximo ciclo. Cabe

destacar que el primer sprint cuenta con una extensión superior a la estándar con el fin de realizar el análisis de la situación inicial junto con el análisis de alternativas.

El desarrollo de este proyecto se ha dividido en 9 sprints diferentes, con inicio en la última semana de enero y cuya fecha estimada de finalización es el 2 de junio. La distribución de los sprints se puede observar en la Ilustración 4-1 Planificación inicial de los Sprints.

Con el objetivo de garantizar la coherencia entre la implementación y la parte teórica, la redacción de la memoria se lleva a cabo de forma paralela al desarrollo de la aplicación. Este enfoque permite el registro y documentación de todas las decisiones, usando dicho documento como un diario de bitácora y así evitar la pérdida de la información.

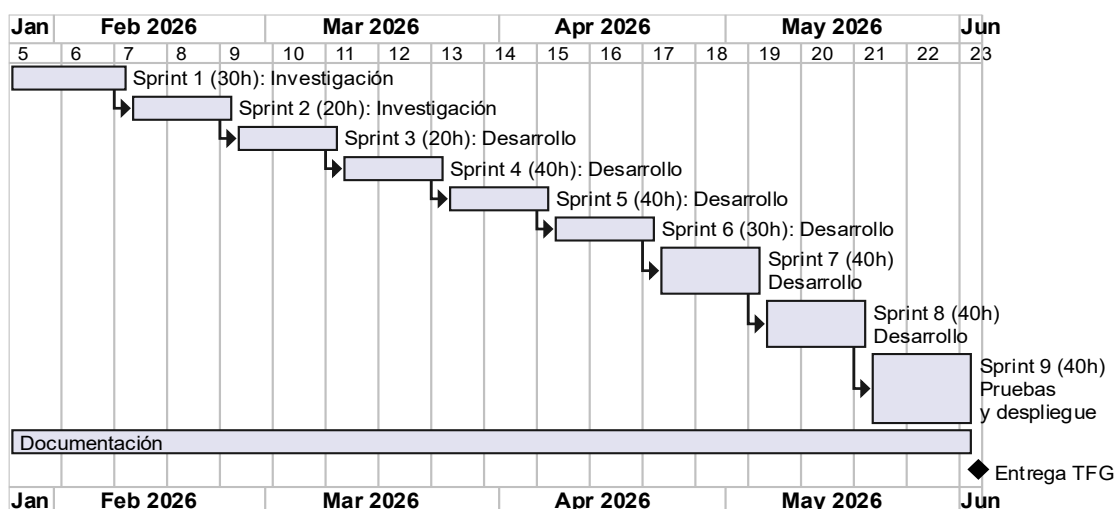


Ilustración 4-1 Planificación inicial de los Sprints

La duración y fechas asignadas a cada uno de los sprints se puede encontrar en la Tabla 4-1

Tabla 4-1 Planificación inicial: Sprints

Sprint	Inicio	Fin	Días
Sprint 1	26-01	10-02	12
Sprint 2	11-02	24-02	10
Sprint 3	25-02	10-03	10
Sprint 4	11-03	24-03	10
Sprint 5	25-03	07-04	10
Sprint 6	08-04	21-04	10
Sprint 7	22-04	05-05	10
Sprint 8	06-05	19-05	10
Sprint 9	20-05	02-06	10

Este sprint inicial se centrará en la investigación de alternativas tecnológicas y en la configuración del entorno de desarrollo. En él se definirán formalmente el alcance y los requisitos del sistema, sentando las bases técnicas de la aplicación.

En la Ilustración 4-2 se puede observar la planificación detallada del primer sprint del proyecto. Además, las diferentes tareas del Sprint 1 junto con su duración y fechas de inicio y fin se detallan en la Tabla 4-2.

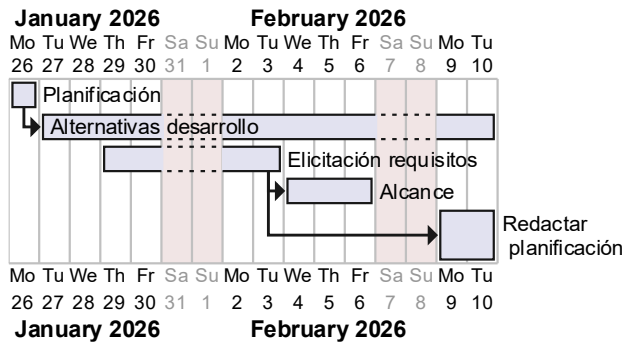


Ilustración 4-2 Planificación inicial Sprint 1

Tarea	Inicio	Fin	Días
Planificar el sprint	26-01	26-01	1
Analizar alternativas de desarrollo	27-01	10-02	11
Elicitación de los requisitos de usuario	29-01	03-02	4
Determinar el alcance	04-02	06-02	3
Redactar planificación	09-02	10-02	2

Tabla 4-2 Planificación inicial Sprint 1

El segundo sprint se enfocará en la selección definitiva de alternativas y en el inicio del desarrollo, priorizando la implementación del módulo de inicio de sesión. Durante este periodo se llevará a cabo una revisión de los requisitos, además de redactar la introducción del proyecto y elaborar el diseño inicial de la arquitectura, estableciendo la estructura sobre la que se construirá el resto de la aplicación.

En la Ilustración 4-3 se muestra la planificación de las actividades a realizar durante el Sprint 2. Además, las diferentes tareas planeadas para el Sprint 2 junto con su duración y fechas de inicio y fin se detallan en la Tabla 4-3

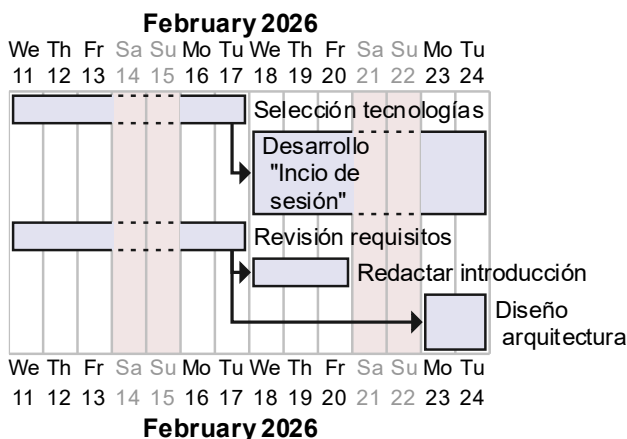


Ilustración 4-3 Planificación inicial Sprint 2

Tarea	Inicio	Fin	Días
Selección de las tecnologías a utilizar	11-02	17-02	4
Desarrollo del "inicio de sesión" inicial	18-02	23-02	5
Revisión de los requisitos de usuario	11-02	17-02	4
Redactar la introducción	18-02	20-02	3
Diseño inicial de la arquitectura	23-02	24-02	2

Tabla 4-3 Planificación inicial Sprint 2

El tercer sprint se centrará en la evolución de las funcionalidades de acceso, optimizando el inicio de sesión e implementando el sistema de creación de cuentas. Además, se comenzará el desarrollo de la pantalla principal de la aplicación a la que redirigir al usuario tras la autenticación. En la documentación se redactarán apartados referentes a la situación actual y se continuará con el desarrollo de la arquitectura.

En la Ilustración 4-4 se muestra la planificación de las actividades a realizar del Sprint 3. Además, en la Tabla 4-4 se pueden observar las tareas del Sprint 3 junto con sus fechas de inicio y fin y su duración en días.

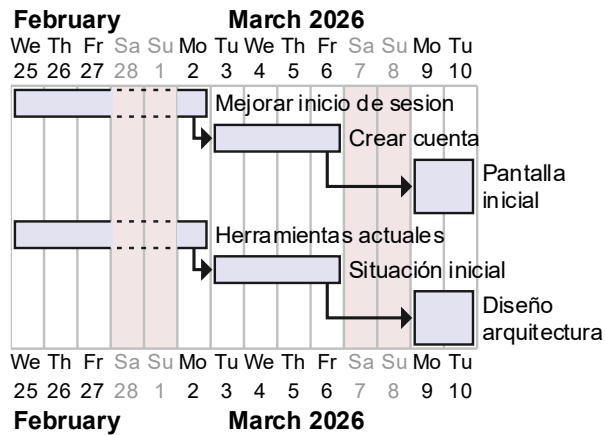


Ilustración 4-4 Planificación inicial Sprint 3

Tarea	Inicio	Fin	Días
Mejorar el inicio de sesión	25-02	02-03	4
Realizar la pantalla "Crear cuenta"	03-03	06-03	4
Realizar la pantalla inicial de la aplicación	06-03	10-03	3
Análisis de las herramientas actuales	25-02	02-02	4
Redactar situación inicial	03-03	06-03	4
Continuar el diseño de la arquitectura	06-03	10-03	3

Tabla 4-4 Planificación inicial Sprint 3

En el cuarto sprint se abordará el núcleo funcional de la aplicación, comenzando con la recomendación de restaurantes. Además, se realizará la elección definitiva de la base de datos para desarrollar nuevas funcionalidades del sistema. En lo que respecta a la documentación, se llevará a cabo una corrección de la memoria con base en los comentarios proporcionados por el tutor, se redactará el análisis de alternativas y se formalizará el diseño del modelo de datos.

En la Ilustración 4-5 se muestra la planificación de las actividades a realizar del Sprint 4. Además, en la Tabla 4-5 se pueden observar las tareas del Sprint 4 junto con sus fechas de inicio y fin y su duración en días.

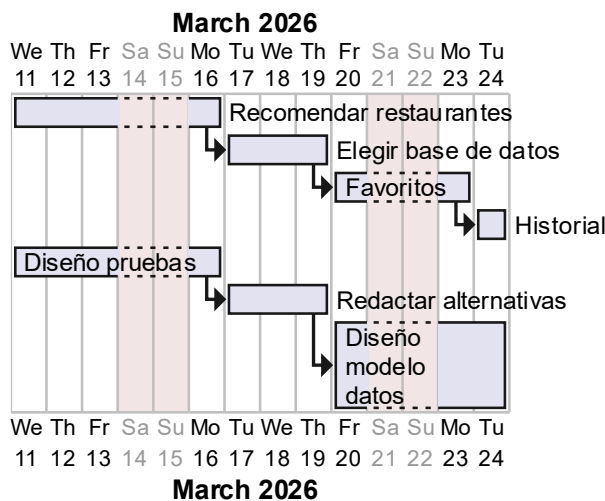


Ilustración 4-5 Planificación inicial Sprint 4

Tarea	Inicio	Fin	Días
Implementar la recomendación de restaurantes	11-03	16-03	4
Elegir servidor de base de datos	17-03	19-03	3
Implementar las listas de favoritos	20-03	23-03	2
Implementar el historial de búsqueda	24-03	24-03	1
Diseño de las pruebas de sistema	11-03	16-03	4
Redactar el análisis de alternativas	17-03	19-03	3
Diseñar el modelo de datos	20-03	24-03	3

Tabla 4-5 Planificación inicial Sprint 4

En el quinto sprint se continuará con la implementación de diferentes funcionalidades de la aplicación con el fin de mejorar la experiencia del usuario. Además, se comenzará con el diseño de las pruebas del sistema, se fijarán los objetivos del sistema y se formalizará tanto el diseño de interfaces como el diseño en detalle de la arquitectura del sistema.

En la Ilustración 4-6 se muestra la planificación de las actividades a realizar del Sprint 5. Además, en la Tabla 4-6 se pueden observar las tareas del Sprint 5 junto con sus fechas de inicio y fin y su duración en días.

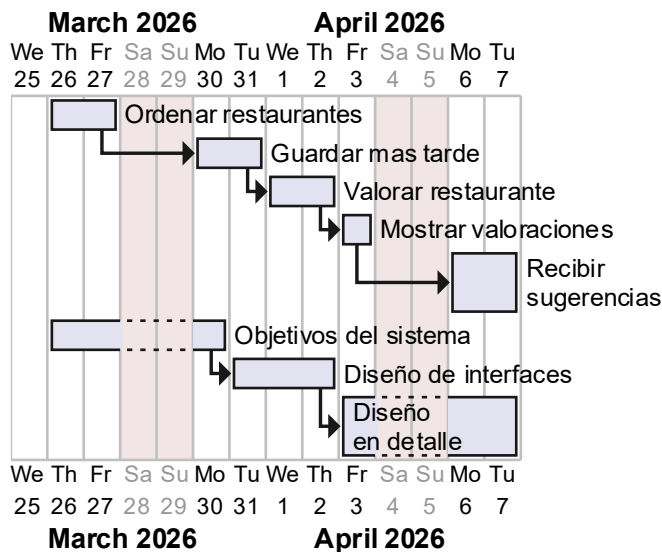


Ilustración 4-6 Planificación inicial Sprint 5

Tarea	Inicio	Fin	Días
Implementar ordenar restaurantes	26-03	27-03	2
Implementar guardar restaurantes para más tarde	30-03	31-03	2
Implementar valorar un restaurante	01-04	02-04	2
Mostar valoraciones de otros usuarios	03-04	03-04	1
Recibir sugerencias por la ubicación	06-04	07-04	2
Redactar los objetivos del sistema	26-03	30-03	3
Diseñar las interfaces del sistema	31-03	02-04	3
Diseño en detalle de la arquitectura	03-04	07-04	3

Tabla 4-6 Planificación inicial Sprint 5

El sexto sprint se dedicará al cierre de las tareas comenzadas en el ciclo anterior. Se completa tanto la lógica de sugerencias como las tareas de diseño de la memoria. Además, se comenzará a realizar las primeras pruebas de concepto referentes al sistema de recomendación y se realizará una mejora visual de la interfaz de usuario para garantizar una navegación fluida.

En la Ilustración 4-7 se muestra la planificación de las actividades a realizar del Sprint 6. Además, en la Tabla 4-7 se pueden observar las tareas para el Sprint 6 junto con sus fechas de inicio y fin y su duración en días.

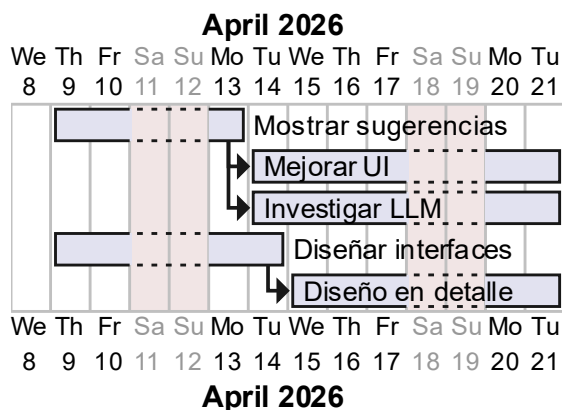


Ilustración 4-7 Planificación inicial Sprint 6

Tarea	Inicio	Fin	Días
Mostrar sugerencias según la ubicación	09-04	13-04	3
Mejorar la interfaz de usuario	14-04	21-04	6
Investigar el uso de un LLM	15-04	21-04	6
Diseñar boceto de las interfaces	09-04	14-04	4
Realizar el diseño en detalle de la arquitectura	15-04	21-04	6

Tabla 4-7 Planificación inicial Sprint 6

El séptimo sprint se centrará en el sistema de recomendación, implementando en la aplicación el prototipo desarrollado como prueba de concepto en el ciclo anterior y adaptando la memoria a la decisión tomada. Además, se completará el abanico de funcionalidades con la implementación de la edición de perfil y se iniciará la investigación sobre el despliegue. Respecto a la documentación, se finalizará la redacción de la planificación y se definirá la estructura de la aplicación.

En la Ilustración 4-8 se muestra la planificación de las actividades a realizar del Sprint 7. Además, en la Tabla 4-8 se pueden observar las tareas del Sprint 7 junto con sus fechas de inicio y fin y su duración en días.

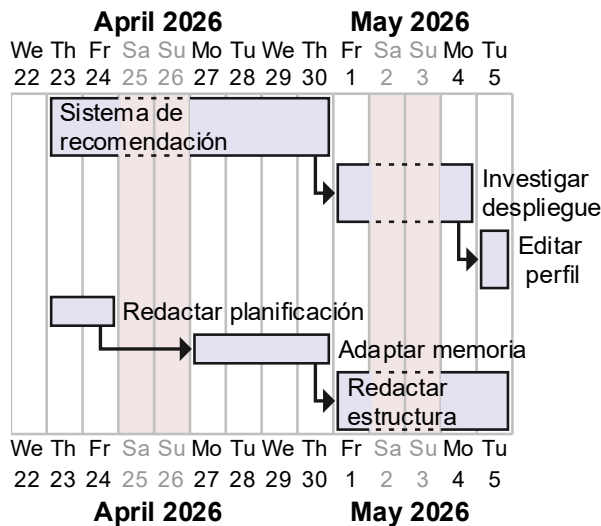


Ilustración 4-8 Planificación inicial Sprint 7

Tarea	Inicio	Fin	Días
Implementar el sistema de recomendación	23-04	30-04	6
Investigar el despliegue de la aplicación	01-05	04-05	2
Editar el perfil de usuario	05-05	05-05	1
Redactar la planificación del proyecto	23-04	24-04	2
Adaptar la memoria al sistema de recomendación	27-04	30-04	4
Redactar la estructura de la aplicación	01-05	05-05	3

Tabla 4-8 Planificación inicial Sprint 7

En el octavo sprint se abarcan las mejoras y características opcionales de la parte técnica del proyecto. Durante este periodo, se abordará la implementación del inicio de sesión y registro usando Google, simplificando el proceso de autenticación para los usuarios. Una vez finalizada, se llevará a cabo el despliegue de la aplicación en la nube, configurando los contenedores necesarios para que sea completamente operativa. De forma paralela se realizará el story mapping del proyecto y posteriormente se realizará la implementación del plan de pruebas.

En la Ilustración 4-9 se muestra la planificación de las actividades a realizar del Sprint 8. Además, en la Tabla 4-9 se pueden observar las del Sprint 8 junto con sus fechas de inicio y fin y su duración en días.

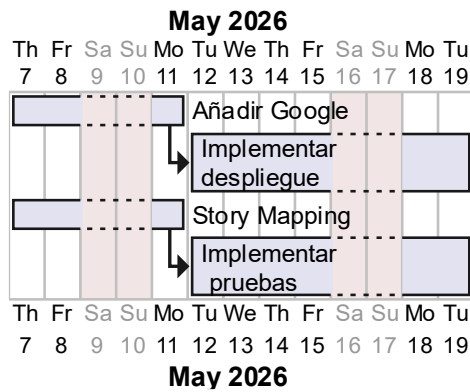


Ilustración 4-9 Planificación inicial Sprint 8

Tarea	Inicio	Fin	Días
Implementar autenticación con Google	07-05	11-05	3
Implementar despliegue de la aplicación	12-05	19-05	6
Realizar el Story Mapping	07-05	11-05	3
Implementar plan de pruebas	12-05	19-05	6

Tabla 4-9 Planificación inicial Sprint 8

En el noveno sprint se completan los apartados restantes de la memoria. Se llevará a cabo la redacción definitiva de los requisitos funcionales al tener todas las historias de usuario definidas, se redactará la organización seleccionada para el despliegue de la aplicación y, al tener todas las funcionalidades completadas, se llevará a cabo la implementación del plan de pruebas diseñado en sprints anteriores. Por otro lado, se desplegará la aplicación para ser usable por los usuarios.

En la Ilustración 4-10 se muestra la planificación de las actividades a realizar del Sprint 9. Además, en la Tabla 4-10 se pueden observar las tareas del Sprint 9 junto con sus fechas de inicio y fin y su duración en días.

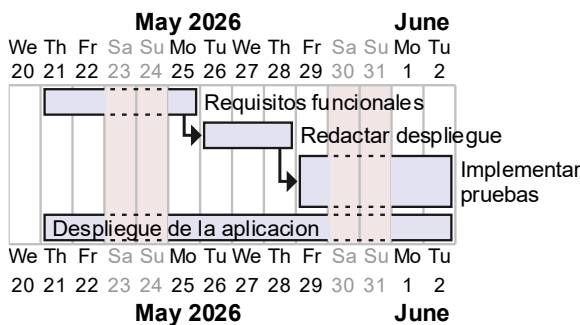


Ilustración 4-10 Planificación inicial Sprint 9

Tarea	Inicio	Fin	Días
Redactar requisitos funcionales	21-05	25-05	3
Redactar despliegue de la aplicación	26-05	28-05	3
Implementar plan de pruebas	29-05	02-06	3
Desplegar la aplicación	21-05	02-06	9

Tabla 4-10 Planificación inicial Sprint 9

4.1.3. Riesgos

La identificación y evaluación de riesgos permite anticipar posibles incidencias que puedan afectar al desarrollo del proyecto. En este apartado se analizan los principales riesgos asociados al proyecto, valorando su impacto y probabilidad de ocurrencia, así como las medidas previstas para minimizar sus posibles consecuencias.

4.1.3.1. Identificación de riesgos

La planificación y ejecución de un proyecto pueden verse afectadas por eventos imprevistos. Con el propósito de mitigar estos impactos durante el desarrollo de la aplicación, es necesario llevar a cabo un proceso de identificación de riesgos. Este análisis permite clasificar las amenazas potenciales en función de su naturaleza y origen, estableciendo una base sólida para diseñar, en fases posteriores, planes de contingencia y estrategias de mitigación que aseguren la robustez y la viabilidad del proyecto.

- R-1) Degradación del rendimiento del modelo:** riesgo de que el volumen de datos de entrenamiento eleve los tiempos de respuesta, penalizando la experiencia del usuario.
- R-2) Sobreajuste del modelo de recomendación:** riesgo de que la red neuronal se optimice en exceso para el conjunto de datos de prueba inicial, perdiendo la capacidad de generalización y ofreciendo sugerencias sin valor real.
- R-3) Bloqueo o revocación de credenciales en APIs externas:** modificaciones en las políticas que invaliden de manera repentina los tokens de acceso a servicios externos, inhabilitando funciones de la aplicación.
- R-4) Inconsistencia por concurrencia:** riesgo de que peticiones simultáneas generen condiciones de carrera o bloqueos en la base de datos.
- R-5) Riesgos de privacidad:** riesgo de filtración de datos sensibles de los usuarios almacenados en la base de datos del sistema.
- R-6) Caída de servicios externos:** inactividad o fallo en los servidores de los proveedores externos, provocando la inoperatividad de funcionalidades de la aplicación.
- R-7) Información incorrecta de datos de terceros:** posibilidad de que la información proporcionada acerca de un restaurante no corresponda con la realidad debido a inconsistencias o errores en los servicios de la API externa.
- R-8) Exposición accidental de credenciales:** riesgo de que claves privadas, tokens o secretos de firma queden expuestos en el repositorio debido a un error en el archivo de exclusión o a una gestión incorrecta de las variables de entorno.
- R-9) Corrupción o pérdida de consistencia en contenedores:** riesgo de que una mala configuración en las directivas de Docker o un reinicio inesperado en el servidor destruya o ignore el volumen asignado a la base de datos.

A continuación, se evalúa cada riesgo en función de su impacto en el sistema y de la probabilidad de que se produzca.

Tabla 4-11 Análisis de riesgos

Riesgo	Impacto	Probabilidad	Severidad
R-1	Medio	Media	Moderada
R-2	Medio	Media	Moderada
R-3	Alto	Baja	Moderada
R-4	Alto	Baja	Moderada
R-5	Alto	Media	Alta
R-6	Alto	Baja	Moderada
R-7	Bajo	Baja	Baja
R-8	Alto	Baja	Moderada
R-9	Alto	Baja	Moderada

Tras esta evaluación, los riesgos se pueden ordenar de la siguiente manera:

Severidad Alta

- R-5) Riesgos de privacidad: riesgo de filtración de datos sensibles de los usuarios almacenados en la base de datos del sistema.

Severidad Moderada

- R-1) Degradación del rendimiento del modelo: riesgo de que el volumen de datos de entrenamiento eleve los tiempos de respuesta, penalizando la experiencia del usuario.
- R-2) Sobreajuste del modelo de recomendación: riesgo de que la red neuronal se optimice en exceso para el conjunto de datos de prueba inicial, perdiendo la capacidad de generalización y ofreciendo sugerencias sin valor real.
- R-3) Bloqueo o revocación de credenciales en APIs externas: modificaciones en las políticas que invaliden de manera repentina los tokens de acceso a servicios externos, inhabilitando funciones de la aplicación.
- R-4) Inconsistencia por concurrencia: riesgo de que peticiones simultáneas generen condiciones de carrera o bloqueos en la base de datos.
- R-6) Caída de servicios externos: inactividad o fallo en los servidores de los proveedores externos, provocando la inoperatividad de funcionalidades de la aplicación.
- R-8) Exposición accidental de credenciales: riesgo de que claves privadas, tokens o secretos de firma queden expuestos en el repositorio debido a un error en el archivo de exclusión o a una gestión incorrecta de las variables de entorno.
- R-9) Corrupción o pérdida de consistencia en contenedores: riesgo de que una mala configuración en las directivas de Docker o un reinicio inesperado en el servidor destruya o ignore el volumen asignado a la base de datos.

Severidad Baja

- R-7) Información incorrecta de datos de terceros: posibilidad de que la información proporcionada acerca de un restaurante no corresponda con la realidad debido a inconsistencias o errores en los servicios de la API externa.

4.1.3.2. Plan de prevención de riesgos

Una vez identificados y catalogados los riesgos, se define el plan de prevención. Este plan detalla las acciones proactivas destinadas a anular o minimizar la probabilidad de

ocurrencia de cada riesgo, así como los mecanismos reactivos diseñados para mitigar el impacto en caso de que la amenaza se materialice.

A continuación, se detalla el plan de acción para cada uno de los riesgos específicos:

R-1) Degradación del rendimiento del modelo

R-1.1) Acción preventiva: programar el script de reentrenamiento de la red neuronal fuera de las horas de mayor concurrencia de la aplicación, mediante tareas en segundo plano.

R-1.2) Plan de contingencia: implementar un almacén de caché en el backend que guarde temporalmente las listas de recomendaciones más solicitadas. Si el módulo sufre una sobrecarga o latencia, el backend servirá las peticiones directamente de la caché, evitando llamadas repetitivas.

R-2) Sobreajuste del modelo de recomendación

R-2.1) Acción preventiva: dividir el conjunto de datos inicial en conjuntos de entrenamiento y de prueba. Sobre el conjunto de entrenamiento, aplicar técnicas de validación cruzada para evaluar el rendimiento del modelo y así evaluar el rendimiento en diferentes subconjuntos de datos.

R-2.2) Plan de contingencia: desarrollar un algoritmo de respaldo basado en reglas simples. Si se detectan sesgos o recomendaciones repetitivas, se activará este flujo secundario mientras se recalibran los pesos de la red neuronal.

R-3) Bloqueo o revocación de credenciales en APIs externas

R-3.1) Acción preventiva: configurar alertas automáticas por correo en los paneles correspondientes para monitorizar cuotas y cambios de políticas, además de usar flujos de autenticación estándar de OAuth 2.0.

R-3.2) Plan de contingencia: desarrollar un middleware que intercepte los códigos de error provenientes de servicios de terceros. Ante su detección, el backend suspenderá la petición y lanzará una solicitud de renovación o actualización del token de acceso sin interrumpir la sesión.

R-4) Inconsistencia por concurrencia en la base de datos

R-4.1) Acción preventiva: diseñar la base de datos utilizando restricciones de integridad referencial estrictas y emplear el sistema de control de concurrencia proporcionado por el gestor de base de datos.

R-4.2) Plan de contingencia: en las operaciones críticas de escritura, implementar reintentos automáticos en el backend. Si una transacción falla debido a una condición de carrera, el sistema la reintentará varias veces antes de reportar un error controlado.

R-5) Riesgos de privacidad y filtración de datos

R-5.1) Acción preventiva: aplicar algoritmos de hashing para almacenar las contraseñas e implementar cifrado en reposo para los campos sensibles y configurar los contenedores bajo el principio de mínimos privilegios.

R-5.2) Plan de contingencia: en caso de detectarse un acceso anómalo o una fuga, revocar de inmediato las claves criptográficas de firma de los tokens desde las variables de entorno, forzando el cierre de sesión de los usuarios activos y aislando el contenedor de la base de datos para auditoría.

R-6) Caída de servicios externos

R-6.1) Acción preventiva: envolver todas las peticiones salientes a las APIs en el patrón de diseño *Circuit Breaker* y establecer límites estrictos de tiempo de espera.

R-6.2) Plan de contingencia: si el servicio externo no responde y el disyuntor se abre tras superar el umbral de fallos, el backend interceptará la excepción y degradará la funcionalidad, ya sea mostrando la última información conocida o enviando un mensaje al frontend para mostrar un aviso controlado al usuario.

R-7) Información incorrecta de datos de terceros

R-7.1) Acción preventiva: programar una capa de validación semántica en el backend que verifique la integridad de los campos críticos de la API externa.

R-7.2) Plan de contingencia: si la respuesta de la API carece de datos o estos son inconsistentes, el backend sustituirá dichos campos por cadenas de texto genéricas para asegurar que la interfaz no se rompa visualmente.

R-8) Exposición accidental de credenciales

R-8.1) Acción preventiva: centralizar todas las claves privadas, secretos y tokens de APIs en variables de entorno protegidas y enmascaradas dentro de la plataforma de integración continua y asegurar que los archivos locales de configuración estén estrictamente declarados en el archivo de exclusión.

R-8.2) Plan de contingencia: si se detecta un filtrado accidental de credenciales en el repositorio, se deben invalidar y revocar las claves comprometidas en las consolas correspondientes y generar nuevas claves.

R-9) Corrupción o pérdida de consistencia en contenedores

R-9.1) Acción preventiva: configurar los volúmenes de almacenamiento mediante enlaces directos a rutas físicas persistentes del sistema de archivos del servidor anfitrión, garantizando que los datos sobrevivan a cualquier reinicio o recreación de los contenedores

R-9.2) Plan de contingencia: automatizar mediante scripts de copias de seguridad periódicas. Estos respaldos deben empaquetarse, cifrarse y transferirse a una ubicación de almacenamiento externa y aislada del servidor principal, lo que permite la restauración completa del sistema en pocos minutos.

4.1.4. Presupuesto inicial

En esta sección se detalla el análisis económico y la estimación de costes requeridos para el diseño, desarrollo, validación y despliegue del proyecto. El cálculo financiero se ha desglosado en diferentes capítulos para reflejar correctamente los recursos utilizados a lo largo de los sprints del proyecto:

- **Capítulo 1:** Costes de personal (Tabla 4-12)
- **Capítulo 2:** Infraestructura y servicios cloud (Tabla 4-13)

Capítulo 3: Dispositivos y hardware de desarrollo (

- Tabla 4-14)
- **Capítulo 4:** Licencias y herramientas software (Tabla 4-15)

Antes de realizar los presupuestos de cada uno de los capítulos especificados anteriormente, se tienen en cuenta las siguientes consideraciones:

- La aplicación es usada por unos 1000 usuarios al mes.
- Cada usuario realiza una media de 4 peticiones al LLM al mes.
- Cada petición consta de 11500 tokens (10000 de entrada y 1500 de salida).
- Cada usuario realiza 20 peticiones a la API de Google Places al mes.

Tabla 4-12 Capítulo 1: Costes de personal

ID	Descripción	Unidad	Cantidad	Precio unit.	Total
Capítulo 1: Costes de personal					
1.1	Personal de ingeniería y desarrollo				
1.1.1	Ingeniero de backend	Horas	58	35€	2.030,00 €
1.1.1.1	Desarrollo inicio de sesión		3		
1.1.1.2	Desarrollo registro		2		
1.1.1.3	Desarrollo página principal		1		
1.1.1.4	Mejora inicio de sesión		4		
1.1.1.5	Desarrollo recomendación de restaurantes		5		
1.1.1.6	Desarrollo historial		1		
1.1.1.7	Desarrollo favoritos		2		
1.1.1.8	Desarrollo ordenar restaurantes		6		
1.1.1.9	Desarrollo guardar para más tarde		5		
1.1.1.10	Desarrollo valorar restaurante		4		
1.1.1.11	Desarrollo ver reseñas de usuarios		3		
1.1.1.12	Desarrollo sugerencias por ubicación		2		
1.1.1.13	Desarrollo editar perfil		1		
1.1.1.14	Desarrollo autenticación con Google		7		
1.1.1.15	Implementar algoritmo de recomendación		12		
1.1.2.	Ingeniero de frontend	Horas	43	28€	1.204,00 €
1.1.2.1	Interfaz inicio de sesión		2		
1.1.2.2	Interfaz registro		2		
1.1.2.3	Interfaz página principal		1		
1.1.2.4	Interfaz recomendación de restaurantes		3		
1.1.2.5	Interfaz historial		1		
1.1.2.6	Interfaz favoritos		2		
1.1.2.7	Interfaz ordenar restaurantes		2		
1.1.2.8	Interfaz guardar para más tarde		3		
1.1.2.9	Interfaz valorar restaurante		4		
1.1.2.10	Interfaz ver reseñas de usuarios		5		
1.1.2.11	Interfaz sugerencias por ubicación		2		
1.1.2.12	Diseño de interfaces		8		
1.1.2.13	Mejorar UI		6		
1.1.2.14	Interfaz editar perfil		1		
1.1.2.15	Interfaz autenticación con Google		1		
1.1.3	Ingeniero de sistemas y despliegue	Horas	27	30€	810,00 €

1.1.3.1	Diseño inicial arquitectura		3		
1.1.3.2	Ampliar diseño de arquitectura		2		
1.1.3.3	Modelo de datos del sistema		6		
1.1.3.4	Diseño en detalle de la arquitectura		6		
1.1.3.5	Implementar despliegue		10		
1.2	Personal de gestión y control de calidad				
1.2.1	Scrum Master	Horas	6	40€	240,00 €
1.2.1.1	Planificación inicial		6		
1.2.2	Ingeniero de pruebas	Horas	28	23€	644,00 €
1.2.2.1	Diseño plan de pruebas		8		
1.2.2.2	Implementar plan de pruebas		20		
1.2.3	Analista	Horas	138	35€	4.830,00 €
1.2.3.1	Análisis de alternativas		12		
1.2.3.2	Requisitos de usuario		8		
1.2.3.3	Alcance del proyecto		4		
1.2.3.4	Seleccionar tecnologías		5		
1.2.3.5	Revisar requisitos		5		
1.2.3.6	Redactar introducción		2		
1.2.3.7	Redactar situación inicial		4		
1.2.3.8	Evaluar herramientas actuales		4		
1.2.3.9	Determinar servidor BD		6		
1.2.3.10	Redactar análisis de alternativas		6		
1.2.3.11	Redactar objetivos del sistema		8		
1.2.3.12	Investigar uso de LLM		6		
1.2.3.13	Investigar despliegue		6		
1.2.3.14	Adaptar la memoria		8		
1.2.3.15	Redactar estructura de la aplicación		6		
1.2.3.16	Redactar la planificación		6		
1.2.3.17	Realizar Story Mapping		8		
1.2.3.18	Realizar el presupuesto		8		
1.2.3.19	Redactar riesgos		8		
1.2.3.20	Requisitos no funcionales		8		
1.2.3.21	Redactar requisitos funcionales		10		

Tabla 4-13 Capítulo 2: Infraestructura y servicios cloud

ID	Descripción	Unidad	Cantidad	Precio unit.	Total
Capítulo 2: Infraestructura y servicios cloud					
2.1.	Servicios de cómputo y servidores				
2.1.1.	Alquiler máquina virtual en la nube	Horas	8760	0,011€	96,97 €
2.1.2	Almacenamiento principal	GB/mes	30*12	0,08€	28,80 €
2.1.3	Redundancia y copias de seguridad	Horas	8760	0,0025€	21,90 €
2.1.4	Almacenamiento en frio para backups	GB/mes	30*12	0,0225€	8,10 €
2.1.5	Tráfico de datos entrante	GB	500	0,00€	0,00€
2.1.6	Tráfico de datos saliente	GB	150	0,0876€	13,14 €
2.1.7	Registro y renovación de dominio corporativo	Años	1	11,09€	11,09 €
2.1.8	Protección perimetral contra ataques DDoS (Cloudflare)	Meses	12	0,00 €	0,00 €
2.2.	Servicios de integración y APIs				
2.2.1	Consumo de Google Places API	peticiones	240000	0,017€	4.080,00 €
2.2.2.	Consumo API de LLM	1M tokens	552	0,60€	331,20 €

Tabla 4-14 Capítulo 3: Dispositivos y hardware de desarrollo

ID	Descripción	Unidad	Cantidad	Precio unit.	Total
Capítulo 3: Dispositivos y hardware de desarrollo					
3.1.	Equipo informático				
3.1.1.	Amortización de ordenador para el desarrollo.	Meses	12	16,67€	200,00 €

Tabla 4-15 Capítulo 4: licencias y herramientas software

ID	Descripción	Unidad	Cantidad	Precio unit.	Total
Capítulo 4: Licencias y herramientas software					
4.1.	Licencias Software				
4.1.1.	Cuenta de desarrollador Google Play	Licencia	1	25€	25,00 €
4.1.2.	Suscripción Anthropic Pro	Meses	12	18€	216,00 €
4.2	Herramientas software				
4.2.1	Microsoft Office 365	Meses	12	10,00€	120,00 €

La cuota de desarrollador de Google Play se repercute al cliente debido a que el registro se efectúa directamente a nombre de la empresa.

Tabla 4-16 Presupuesto total

Tarea	Precio
Capítulo 1: Costes de personal	9.758,00 €
Capítulo 2: Infraestructura y servicios cloud	4.591,20 €
Capítulo 3: Dispositivos y hardware de desarrollo	200,00 €
Capítulo 4: Licencias y herramientas software	361,00 €
Total Presupuesto	14.910,20€

El presupuesto total asciende a catorce mil novecientos diez euros con veinte céntimos.

4.2. EJECUCIÓN DEL PROYECTO

La ejecución del proyecto recoge el desarrollo de las tareas realizadas durante la fase de implementación del sistema, conforme a la planificación inicialmente establecida. Durante este proceso se han abordado las distintas actividades necesarias para la construcción de la aplicación y la consecución de los objetivos del proyecto.

4.2.1. Incidencias

Durante el desarrollo del proyecto han surgido diversas incidencias y dificultades que han afectado, en mayor o menor medida, a la ejecución de algunas tareas previstas. Este tipo de situaciones es habitual en el desarrollo de sistemas software y ha requerido la toma de decisiones y la aplicación de soluciones para garantizar la continuidad del proyecto.

- **Configuración de OAuth2:** la implementación del inicio de sesión y registro a través de Google presentó dificultades técnicas. El principal problema era que la comunicación con el proveedor externo no fluía ni era adecuada. Conseguir que el proceso funcionase correctamente requirió un número de horas ligeramente superior a lo esperado.

- **Desviación temporal en la redacción de la memoria:** la redacción de la memoria del proyecto demandó un esfuerzo y un tiempo superiores a lo previsto. El sistema requería completar una gran cantidad de apartados detallados, lo que hizo que el proceso se alargara más de lo previsto inicialmente.
- **Descartar el uso del LLM:** tras evaluar la viabilidad de utilizar un LLM para el motor de recomendaciones, se descartó por completo esta alternativa. La investigación demostró un esfuerzo excesivo para el rendimiento esperado. Este descarte invalidó parte del tiempo invertido en el trabajo, pero se mitigó reorientando el esfuerzo al diseño de una red neuronal.

4.3. CIERRE DEL PROYECTO

El cierre del proyecto recoge la fase final del Trabajo de Fin de Grado, en la que se revisan y consolidan los principales aspectos de gestión y de desarrollo. En este apartado se presentan la planificación final del proyecto, el presupuesto estimado final y el informe de riesgos tras la ejecución, con el objetivo de ofrecer una visión global del resultado obtenido.

4.3.1. Planificación final

En este apartado se presenta la planificación final del proyecto, que muestra la distribución temporal de las actividades desarrolladas durante la realización del Trabajo de Fin de Grado. En ella se reflejan las diferentes fases llevadas a cabo junto con las desviaciones ocurridas a lo largo del desarrollo.

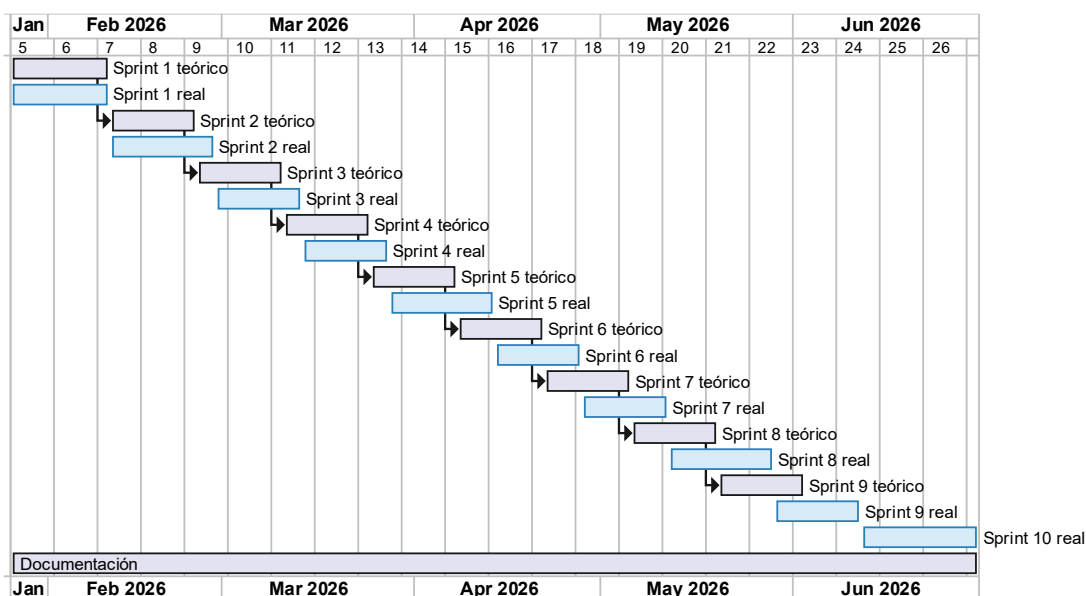


Ilustración 4-11 Planificación final

En la planificación, se pueden observar cuatro principales desviaciones:

- El Sprint 2 se vio afectado por la carga de trabajo simultáneo de las prácticas en empresa, lo que redujo significativamente el tiempo disponible para el proyecto durante ese período y provocó que una tarea quedara sin completar dentro del plazo previsto.

- En el Sprint 5, el retraso se debió a una estimación incorrecta del tiempo disponible. La planificación inicial no tuvo en cuenta los días festivos de Semana Santa, lo que resultó en menos horas de trabajo efectivas para ese sprint.
- El Sprint 8 coincidió con el periodo de exámenes y entrega de trabajos de otras asignaturas, lo que limitó nuevamente el tiempo dedicado al proyecto e impidió completar la totalidad de las tareas planificadas.
- Finalmente, fue necesario incorporar un Sprint 10 no previsto en la planificación original. Este sprint adicional se debe por un lado a la acumulación de las pequeñas desviaciones anteriores, y por otro lado al tiempo invertido en revisar y corregir los comentarios recibidos sobre la memoria.

4.3.2. Informe final de riesgos

Una vez finalizado el proyecto, resulta necesario evaluar la evolución de los riesgos identificados, si han llegado a manifestarse y su impacto en el sistema. En este apartado se presenta el informe final de riesgos, que refleja el estado de cada riesgo y las acciones realizadas durante la ejecución del proyecto.

- **Sobreajuste del modelo de recomendación:** durante las primeras pruebas del modelo se observó un sobreajuste, con recomendaciones poco variadas en determinados conjuntos de datos. Para mitigar este efecto, se aplicaron técnicas de validación cruzada y de separación entre datos de entrenamiento y de prueba, lo que mejoró la capacidad de generalización del modelo.
- **Caída de servicios externos:** durante la ejecución del proyecto se produjeron incidencias puntuales relacionadas con la indisponibilidad o latencia de servicios externos utilizados por la aplicación. Estas situaciones fueron gestionadas mediante el manejo de errores y tiempos de espera definidos en la implementación, sin necesidad de activar el plan de contingencia de forma completa, aunque el sistema se comportó según lo previsto ante estos fallos.
- **Información incorrecta de datos de terceros:** se detectaron casos en los que la información proporcionada por APIs externas era incompleta o inconsistente. Para resolverlo, se aplicaron mecanismos de validación y sustitución de valores por defecto, garantizando la correcta visualización de la información en la aplicación. No fue necesario activar el plan de contingencia completo.

El resto de los riesgos no llegaron a materializarse. Especialmente en el caso del R-5, el único catalogado con severidad alta, las medidas preventivas adoptadas resultaron eficaces, manteniéndose bajo control durante el ciclo de vida del ecosistema.

4.3.3. Presupuesto final de costes

El impacto económico del proyecto se ha visto condicionado por la evolución del ciclo de desarrollo y por decisiones tomadas durante la fase de implementación. Con el objetivo de reflejar el coste real, el cálculo del presupuesto final se ha reestructurado atendiendo a dos desviaciones principales respecto a la planificación inicial:

- **Ampliación temporal:** debido a los imprevistos durante el desarrollo, el cronograma del proyecto sufrió una extensión equivalente a un sprint adicional de

40 horas. Esta desviación temporal ha requerido la dedicación de horas extraordinarias del personal involucrado.

- **Reorientación tecnológica del motor de recomendación:** inicialmente se planteó el uso de un modelo de lenguaje de gran tamaño. En su lugar, el sistema migró hacia el desarrollo propio de una red neuronal personalizada. Esta decisión altera en gran medida los costes, puesto que se eliminan los gastos recurrentes de la API del modelo de lenguaje.

Tabla 4-17 Capítulo 1: Costes de personal

ID	Descripción	Unidad	Cantidad	Precio unit.	Total
Capítulo 1: Costes de personal					
1.1	Personal de ingeniería y desarrollo				
1.1.1	Ingeniero de backend	Horas	58	35€	2.030,00 €
1.1.1.1	Desarrollo inicio de sesión		3		
1.1.1.2	Desarrollo registro		2		
1.1.1.3	Desarrollo página principal		1		
1.1.1.4	Mejora inicio de sesión		4		
1.1.1.5	Desarrollo recomendación de restaurantes		5		
1.1.1.6	Desarrollo historial		1		
1.1.1.7	Desarrollo favoritos		2		
1.1.1.8	Desarrollo ordenar restaurantes		6		
1.1.1.9	Desarrollo guardar para más tarde		5		
1.1.1.10	Desarrollo valorar restaurante		4		
1.1.1.11	Desarrollo ver reseñas de usuarios		3		
1.1.1.12	Desarrollo sugerencias por ubicación		2		
1.1.1.13	Desarrollo editar perfil		1		
1.1.1.14	Desarrollo autenticación con Google		7		
1.1.1.15	Implementar algoritmo de recomendación		12		
1.1.2.	Ingeniero de frontend	Horas	43	28€	1.204,00 €
1.1.2.1	Interfaz inicio de sesión		2		
1.1.2.2	Interfaz registro		2		
1.1.2.3	Interfaz página principal		1		
1.1.2.4	Interfaz recomendación de restaurantes		3		
1.1.2.5	Interfaz historial		1		
1.1.2.6	Interfaz favoritos		2		
1.1.2.7	Interfaz ordenar restaurantes		2		
1.1.2.8	Interfaz guardar para más tarde		3		
1.1.2.9	Interfaz valorar restaurante		4		
1.1.2.10	Interfaz ver reseñas de usuarios		5		
1.1.2.11	Interfaz sugerencias por ubicación		2		
1.1.2.12	Diseño de interfaces		8		
1.1.2.13	Mejorar UI		6		
1.1.2.14	Interfaz editar perfil		1		
1.1.2.15	Interfaz autenticación con Google		1		
1.1.3	Ingeniero de sistemas y despliegue	Horas	35	30€	1.050,00 €
1.1.3.1	Diseño inicial arquitectura		3		
1.1.3.2	Ampliar diseño de arquitectura		2		
1.1.3.3	Modelo de datos del sistema		6		

1.1.3.4	Diseño en detalle de la arquitectura		6		
1.1.3.5	Implementar despliegue		10		
1.1.3.6	README del proyecto		8		
1.2	Personal de gestión y control de calidad				
1.2.1	Scrum Master	Horas	6	40€	240,00 €
1.2.1.1	Planificación inicial		6		
1.2.2	Ingeniero de pruebas	Horas	28	23€	644,00 €
1.2.2.1	Diseño plan de pruebas		8		
1.2.2.2	Implementar plan de pruebas		20		
1.2.3	Analista	Horas	170	35€	5950,00 €
1.2.3.1	Análisis de alternativas		12		
1.2.3.2	Requisitos de usuario		8		
1.2.3.3	Alcance del proyecto		4		
1.2.3.4	Seleccionar tecnologías		5		
1.2.3.5	Revisar requisitos		5		
1.2.3.6	Redactar introducción		2		
1.2.3.7	Redactar situación inicial		4		
1.2.3.8	Evaluar herramientas actuales		4		
1.2.3.9	Determinar servidor BD		6		
1.2.3.10	Redactar análisis de alternativas		6		
1.2.3.11	Redactar objetivos del sistema		8		
1.2.3.12	Investigar uso de LLM		6		
1.2.3.13	Investigar despliegue		6		
1.2.3.14	Adaptar la memoria		8		
1.2.3.15	Redactar estructura de la aplicación		6		
1.2.3.16	Redactar la planificación		6		
1.2.3.17	Realizar Story Mapping		8		
1.2.3.18	Realizar el presupuesto		8		
1.2.3.19	Redactar riesgos		8		
1.2.3.20	Requisitos no funcionales		8		
1.2.3.21	Redactar requisitos funcionales		10		
1.2.3.22	Planificación final		8		
1.2.3.23	Trabajo futuro		8		
1.2.3.24	Manual de usuario		8		
1.2.3.25	Redactar conclusiones		8		

Tabla 4-18 Capítulo 2: Infraestructura y servicios cloud

ID	Descripción	Unidad	Cantidad	Precio unit.	Total
Capítulo 2: Infraestructura y servicios cloud					
2.1	Servicios de cómputo y servidores				
2.1.1	Alquiler máquina virtual en la nube	Horas	8760	0,011€	96,97 €
2.1.2	Almacenamiento principal	GB/mes	30*12	0,08€	28,80 €
2.1.3	Redundancia y copias de seguridad	Horas	8760	0,0025€	21,90 €
2.1.4	Almacenamiento en frío para backups	GB/mes	30*12	0,0225€	8,10 €
2.1.5	Tráfico de datos entrante	GB	500	0,00€	0,00€
2.1.6	Tráfico de datos saliente	GB	150	0,0876€	13,14 €
2.1.7	Registro y renovación de dominio corporativo	Años	1	11,09€	11,09 €
2.1.8	Protección perimetral contra ataques DDoS (Cloudflare)	Meses	12	0,00 €	0,00 €
2.2	Servicios de integración y APIs				
2.2.1	Consumo de Google Places API	peticiones	240000	0,017€	4.080,00 €

Tabla 4-19 Capítulo 3: Dispositivos y hardware de desarrollo

ID	Descripción	Unidad	Cantidad	Precio unit.	Total
Capítulo 3: Dispositivos y hardware de desarrollo					
3.1	Equipo informático				
3.1.1	Amortización de ordenador para el desarrollo.	Meses	12	16,67€	200,00 €

Tabla 4-20 Capítulo 4: Licencias y herramientas software

ID	Descripción	Unidad	Cantidad	Precio unit.	Total
Capítulo 4: Licencias y herramientas software					
4.1	Licencias Software				
4.1.1	Cuenta de desarrollador Google Play	Licencia	1	25€	25,00 €
4.1.2	Suscripción Anthropic Pro	Meses	12	18€	216,00 €
4.2	Herramientas software				
4.2.1	Microsoft Office 365	Meses	12	10€	120,00 €

Tabla 4-21 Presupuesto total

Tarea	Precio
Capítulo 1: Costes de personal	11.118,00 €
Capítulo 2: Infraestructura y servicios cloud	4.260,00 €
Capítulo 3: Dispositivos y hardware de desarrollo	200,00 €
Capítulo 4: Licencias y herramientas software	361,00 €
Total Presupuesto	15.939,00€

El presupuesto final asciende a quince mil novecientos treinta y nueve euros.

A pesar de haber reducido costes con la eliminación del modelo de lenguaje, el aumento de horas de trabajo ha provocado un aumento en el presupuesto final del proyecto.

Capítulo 5 ANÁLISIS DE ALTERNATIVAS

La elección de las tecnologías que componen la arquitectura de una aplicación no debe basarse únicamente en modas o preferencias personales, sino en un proceso de evaluación fundamentado en los distintos requisitos del proyecto. Para ello, es recomendable identificar primero las necesidades y, a continuación, comparar las distintas opciones con los mismos criterios, de modo que la decisión final pueda justificarse.

En esta sección se evalúan las diferentes herramientas, lenguajes y servicios disponibles para el desarrollo del proyecto. A partir de este análisis, se justificará la selección final de cada una de las tecnologías empleadas en el proyecto, priorizando aquellas que mejor se adapten a sus condiciones y necesidades.

5.1. LENGUAJES DE PROGRAMACIÓN FRONTEND

En esta sección se evalúan los diferentes lenguajes para el desarrollo de la interfaz de usuario y la interacción con el usuario final. Esta elección es fundamental, pues influye en aspectos como la mantenibilidad, la escalabilidad y la productividad del desarrollo. Para realizar esta selección, se tendrán en cuenta varias opciones según un conjunto de criterios adaptados a las necesidades del proyecto.

Ejemplos

Los ejemplos de cada uno de los lenguajes analizados se encuentran en los fragmentos Código 1, Código 2, Código 3 y Código 4:

<pre>let i = 1; i = i + 1; console.log(i);</pre>	<pre>let i: number = 1; i = i + 1; console.log(i);</pre>	<pre>void main() { int i = 1; i = i + 1; print(i); }</pre>	<pre>import Html exposing (text) main = let i = 1 + 1 in text (String.fromInt i)</pre>
Código 1 Ejemplo código JavaScript	Código 2 Ejemplo código TypeScript	Código 3 Ejemplo código Dart	Código 4 Ejemplo código Elm

Criterios de evaluación

- **Experiencia previa:** grado de familiaridad con el lenguaje que reduce la curva de aprendizaje y previene errores comunes.
- **Curva de aprendizaje:** facilidad para aprender el lenguaje hasta poder desarrollar tareas básicas de manera independiente, considerando la disponibilidad de recursos formativos.
- **Frameworks:** cantidad de marcos de trabajo y librerías del ecosistema del lenguaje.

- **Productividad:** rapidez y comodidad con la que se puede desarrollar y mantener funcionalidad incluyendo cantidad de código necesario, herramientas disponibles, o reutilización de código.
- **Soporte / Comunidad:** tamaño y actividad de la comunidad, traducido en soporte, facilidad para encontrar soluciones, calidad de la documentación o existencia de páginas y foros especializados en el lenguaje específico.

Conclusiones

Con base en las alternativas seleccionadas, se realiza una comparación según los criterios de evaluación mencionados anteriormente en la Tabla 5-1.

Tabla 5-1 Comparativa de lenguajes de programación de frontend

Característica	JavaScript	TypeScript	Dart	Elm
Experiencia previa	Baja	Baja	Nula	Nula
Curva de aprendizaje	Leve	Media	Media	Pronunciada
Frameworks	React, Vue, Angular, Svelte	React, Angular, Vue, Next.js, etc.	Flutter	Elm con The Elm Architecture
Productividad	Alta	Muy alta	Alta	Media
Soporte / Comunidad	Muy grande	Muy grande	Grande, orientada a móvil	Pequeña

Tras analizar los distintos lenguajes con base en los criterios definidos, JavaScript se presenta como la opción más accesible y con un ecosistema más amplio. Sin embargo, su naturaleza dinámica incrementa el riesgo de errores [8]. Por otro lado, Dart ofrece una alta productividad gracias a su integración con Flutter, pero está excesivamente orientado al desarrollo móvil, lo que limita su flexibilidad [9]. En el extremo opuesto se encuentra Elm, que garantiza una gran robustez, presenta una curva de aprendizaje pronunciada y una comunidad reducida [10].

Finalmente, se ha seleccionado TypeScript como lenguaje de desarrollo para el frontend [11]. Ofrece un buen equilibrio entre la curva de aprendizaje y la productividad, sumado a una amplia comunidad para resolver cualquier tipo de duda. Además, es un lenguaje en el que se cuenta con experiencia previa, lo cual facilita el desarrollo y reduce el riesgo de aparición de errores básicos.

5.2. LENGUAJES DE PROGRAMACIÓN BACKEND

El lenguaje de programación del backend es el encargado de gestionar la lógica interna de la aplicación, el procesamiento de datos y la comunicación con la base de datos y servicios externos, como la API de búsqueda de restaurantes. En esta sección, se analizan diferentes lenguajes para elegir aquel que permita un desarrollo rápido y que cuente con las librerías necesarias para el tratamiento de datos e integración de modelos de inteligencia artificial.

Ejemplos

Los ejemplos de cada uno de los lenguajes analizados se encuentran en los fragmentos Código 5, Código 6, Código 7 y Código 8:

<pre>i = 1 i = i + 1 print(i)</pre>	<pre>public class Main { public static void main(String[] args) { int i = 1; i = i + 1; System.out.println(i); } }</pre>	<pre>let i: number = 1; i = i + 1; console.log(i);</pre>	<pre>package main import "fmt" func main() { i := 1 i = i + 1 fmt.Println(i) }</pre>
Código 5 Ejemplo código Python	Código 6 Ejemplo código Java	Código 7 Ejemplo código TypeScript	Código 8 Ejemplo código Go

Criterios de evaluación

Los criterios de evaluación aplicados en esta sección son los mismos que en Lenguajes de programación frontend.

Conclusiones

Con base en las alternativas seleccionadas, se realiza una comparación bajo los criterios de evaluación mencionados anteriormente en la Tabla 5-2.

Tabla 5-2 Comparativa de lenguajes de programación de backend

Característica	Python	Java	JavaScript/TypeScript	Go
Experiencia previa	Media	Alta	Baja	Nula
Curva de aprendizaje	Leve	Media	Media	Media-Pronunciada
Frameworks	FastAPI, Django, Flask	Spring Boot, Quarkus, Micronaut	Express, DenoJS, NestJS	Gin, Fiber, Echo
Productividad	Muy alta	Media	Alta	Media
Soporte / Comunidad	Muy grande	Muy grande	Muy grande	Grande

A partir de la comparativa, se observa que todos los lenguajes presentan un ecosistema sólido y una comunidad amplia, con múltiples frameworks disponibles [12].

Lenguajes como Go [13] o JavaScript ofrecen un rendimiento excelente, pero su menor especialización en librerías de IA, sumado a la falta de experiencia previa, suponen un riesgo para el desarrollo del proyecto.

Finalmente, al enfrentar Java [14] y Python [15] debido a su amplia experiencia previa se ha seleccionado Python como lenguaje de programación para el backend ya que resulta especialmente adecuado para su uso en este proyecto, debido a su facilidad de integración de APIs externas y su uso extendido en aplicaciones relacionadas con inteligencia artificial [16]. Además, Python destaca por una curva de aprendizaje más leve, que se vuelve menos pronunciada con los conocimientos previos, y ofrece una mayor productividad, permitiendo un desarrollo más rápido que otras alternativas.

5.3. FRAMEWORK FRONTEND

Una vez definido el lenguaje, es necesario seleccionar un framework o un marco de trabajo. Esta tecnología proporciona una estructura organizada y un conjunto de componentes y herramientas predefinidas que evitan tener que programar cada elemento visual desde cero. Esta decisión facilita la gestión de la información que se muestra en pantalla y asegura que la aplicación cumpla con ciertos estándares de calidad.

Criterios de evaluación

- **Curva de aprendizaje:** facilidad para aprender el lenguaje hasta poder desarrollar tareas básicas de manera independiente, considerando la disponibilidad de recursos formativos.
- **Rendimiento:** medido en términos de la velocidad de renderización y actualización de la interfaz en escenarios reales.
- **Boilerplate:** cantidad de código y estructura que el *framework* exige para tareas comunes.
- **Documentación:** calidad, cantidad y actualización de la documentación oficial y recursos formales.
- **Async nativo:** soporte del *framework* de las operaciones asíncronas típicas del frontend.
- **Ecosistema de plugins:** conjunto de librerías y herramientas que permiten extender el *framework* para cubrir más casos de uso.

Conclusiones

Con base en las alternativas seleccionadas, se realiza una comparación bajo los criterios de evaluación mencionados anteriormente en la Tabla 5-3.

Tabla 5-3 Comparativa de frameworks de frontend

Característica	React	Angular	Vue
Curva de aprendizaje	Media	Alta	Media
Rendimiento	Muy bueno	Muy bueno	Muy bueno
Boilerplate	Bajo	Alto	Bajo
Documentación	Buena + comunidad activa	Muy amplia	Muy buena y detallada
Async nativo	Sí	Sí	Sí
Ecosistema de plugins	Alto	Algo diverso	Medio

Tras analizar los diferentes marcos de trabajo, Angular se presenta como una plataforma con una pronunciada curva de aprendizaje y un elevado *boilerplate* que podrían ralentizar el desarrollo en las fases iniciales [17]. Por otro lado, Vue destaca por una documentación clara. Sin embargo, su ecosistema de librerías y *plugins* no alcanza la madurez para cubrir las necesidades específicas de la aplicación [18].

Finalmente se ha optado por React como framework para el desarrollo del frontend [19]. La decisión se basa principalmente en su enorme ecosistema de plugins, que ofrece soluciones preexistentes para casi cualquier necesidad del proyecto, y en su comunidad activa, que facilita la resolución de errores. Además, su bajo *boilerplate* permite un desarrollo más directo y centrado en las funcionalidades.

Como razón adicional, React es un framework que no se estudia en el grado universitario, por lo que resulta interesante aprender a utilizarlo debido a su amplio uso y popularidad en el mundo laboral.

5.4. FRAMEWORK BACKEND

Una vez seleccionado Python como lenguaje principal, es necesario elegir un framework que proporcione la estructura necesaria para construir el servidor. Este se encarga de la gestión de rutas, estructuración de código y seguridad básica, por lo que influye directamente en la productividad, rendimiento y mantenibilidad del sistema.

Criterios de evaluación

Los criterios de evaluación aplicados en esta sección son los mismos que en Framework frontend.

Conclusiones

Con base en las alternativas seleccionadas, se realiza una comparación bajo los criterios de evaluación mencionados anteriormente en la Tabla 5-4.

Tabla 5-4 Comparativa de frameworks de backend

Característica	FastAPI	Django	Flask
Curva de aprendizaje	Baja	Media-Pronunciada	Baja
Rendimiento	Alto	Medio	Medio
Documentación automática	Swagger/ReDoc (OpenAPI)	DRF browsable API + OpenAPI	Manual (con extensiones para OpenAPI)
Boilerplate	Mínimo	Medio	Mínimo
Async nativo	Sí	Parcial	No
Ecosistema de plugins	Creciente	Enorme	Enorme

Los tres marcos de trabajo cuentan con comunidades maduras y un ecosistema de plugins muy extenso, además de la posibilidad de añadir documentación de forma automatizada. No obstante, existen diferencias en su arquitectura. Mientras que Django presenta una curva de aprendizaje más pronunciada y un *boilerplate* elevado [20], FastAPI y Flask [21] destacan por un código inicial mínimo y una implementación mucho más ágil. Por otro lado, FastAPI sobresale frente al resto en términos de rendimiento, además de ofrecer soporte asíncrono nativo.

Finalmente, se ha optado por FastAPI como framework para el desarrollo del backend [22]. Además de lo comentado anteriormente, su especialización en el uso de APIs lo convierte en el entorno más adecuado para el proyecto.

5.5. APIs PARA LA BÚSQUEDA DE RESTAURANTES

En esta sección se analizan distintas APIs externas utilizadas para obtener información sobre restaurantes. Para que el motor de recomendación sea efectivo, es necesario contar con una API que proporcione datos precisos, actualizados y que cubra una amplia zona geográfica. La elección de este servicio externo condiciona la utilidad de la aplicación para

los usuarios, pues la calidad de los datos de entrada será directamente proporcional a la calidad de las salidas.

Criterios de evaluación

- **Cobertura:** alcance geográfico y nivel de detalle de la API.
- **Datos ofrecidos:** variedad de la información disponible.
- **Complejidad legal:** restricciones y obligaciones en las condiciones de uso.
- **Facilidad de uso:** sencillez para integrar la API en la aplicación.
- **Fortaleza principal:** característica distintiva de una API frente a las demás.
- **Historial/Favoritos:** capacidades específicas para gestionar las ubicaciones guardadas por el usuario.
- **Plan gratuito:** disponibilidad de una modalidad de uso sin coste o con créditos mensuales que permitan el desarrollo y las pruebas del proyecto.

Conclusiones

Con base en las alternativas seleccionadas, se realiza una comparación según los criterios de evaluación mencionados anteriormente en la Tabla 5-5.

Tabla 5-5 Comparativa de APIs para la búsqueda de restaurantes

Característica	Google Places API	Yelp Fusion API	Foursquare Places API	TripAdvisor API
Cobertura	Muy alta, incluye ciudades pequeñas, pueblos, etc.	Limitada fuera de grandes ciudades	Buena en zonas urbanas, más limitada en ámbito global	Buena en destinos muy turísticos
Datos ofrecidos	- Nombre - Dirección - Coordenadas - Tipos/categorías - Rating medio - Número total de reseñas - Reseñas de usuarios - Horario - Rango de precios - Teléfono - Página web - Fotos - Identificador estable (place_id) - Otros metadatos del negocio.	- Similar a Google para los datos básicos - Sin campos como place_id. - Añade el ecosistema propio de Yelp	- Datos básicos similares a los de Google - Sin identificadores como place_id - Sin ciertos campos específicos de Maps. - Popularidad agregada por tráfico - Datos de cadenas comerciales - Estadísticas de tips - Enlaces a redes sociales	- Datos básicos comparables a los de Google - Sin identificadores como place_id - Sin algunos campos de negocio. - Ranking dentro del destino - "Bubble ratings" de viajeros - Desglose de valoraciones - Posición relativa en las listas de TripAdvisor.
Complejidad legal	Media-Alta, caché solo place_id, atribución obligatoria	Media, Atribución + límites	Baja Open source puro	Alta Partner program, aprobación necesaria
Facilidad de uso	Alta (SDKs completos y con soporte)	Media-Alta (REST simple)	Alta (REST bien documentado)	Media, setup complejo
Fortaleza principal	Gran cobertura y gran cantidad de datos	Reseñas con gran cantidad de datos.	Buen equilibrio datos/condiciones	Reseñas de viajeros
Historial/Favoritos	Guardar IDs + tus propios datos	IDs + 24 h de datos, más restrictivo	Cache permitido según plan, algo más flexible	Solamente guarda los IDs

Plan gratuito	Gratis hasta 10000 peticiones al mes	Prueba gratuita de 1 mes	Gratis hasta 10000 peticiones al mes	No, requiere aprobación como partner
---------------	--------------------------------------	--------------------------	--------------------------------------	--------------------------------------

Tras el análisis de las diferentes alternativas, se descartan tanto Yelp Fusion [23] como TripAdvisor [24] por no disponer de un plan gratuito compatible con el desarrollo del proyecto .

Respecto a Foursquare Places, se presenta como una alternativa competitiva con una política de almacenamiento de datos más permisiva. Sin embargo, su cobertura en zonas rurales o municipios pequeños es menor, centrando su potencial en grandes núcleos urbanos [25].

Finalmente, se ha seleccionado la API de Google Places como fuente de datos principal [26]. El factor determinante ha sido su amplia cobertura, que garantiza que la aplicación funcione correctamente en prácticamente cualquier punto geográfico. Además, ofrece un gran volumen de datos y una facilidad de integración que aseguran el correcto funcionamiento del motor de búsqueda.

5.6. TIPO DE BASE DE DATOS

Una decisión fundamental en la arquitectura de cualquier sistema es determinar cómo se almacenará y organizará la información de forma permanente. En esta sección se analizan los dos tipos de base de datos con el fin de seleccionar aquel que mejor se ajuste a la naturaleza de los datos que maneja la aplicación.

Ejemplos

Los ejemplos de cada uno de los tipos analizados se encuentran en los fragmentos Código 9 y Código 10:

-- SQL SELECT nombre, puntuacion FROM restaurantes WHERE puntuacion > 4;	#NoSQL (MongoDB) db.restaurantes.find({ puntuacion: { \$gt: 4 } })
---	---

Código 9 Ejemplo SQL

Código 10 Ejemplo NoSQL

Criterios de evaluación

- **Modelo de datos:** estructura y formato en los que se representa la información.
- **Lenguaje de consulta:** herramientas y estándares disponibles para extraer información de la base de datos.
- **Relaciones entre entidades:** capacidad nativa del sistema para conectar diferentes conjuntos de datos garantizando la integridad de la información.
- **Complejidad de consultas:** facilidad para realizar búsquedas avanzadas que requieran filtrar, agrupar y cruzar múltiples tablas sin penalizar el rendimiento.
- **Experiencia previa:** familiaridad con el tipo de base de datos y conocimientos previos que agilizan el desarrollo y reducen los errores comunes.
- **Curva de aprendizaje:** facilidad para aprender el lenguaje hasta poder desarrollar tareas básicas de manera independiente, considerando la disponibilidad de recursos formativos.

- **Encaje con el proyecto:** grado en que se ajusta a las necesidades concretas del Trabajo de Fin de Grado. Reduce el riesgo y esfuerzo de implementación en este contexto en específico.

Conclusiones

Con base en las alternativas seleccionadas, se realiza una comparación bajo los criterios de evaluación mencionados anteriormente en la Tabla 5-6.

Tabla 5-6 Comparativa de tipos de bases de datos

Característica	Relacional	No relacional
Modelo de datos	Esquema bien definido	Esquema flexible o inexistente
Lenguaje de consulta	SQL	Lenguajes o APIs específicas
Relaciones entre entidades	Relaciones explícitas	Relaciones implícitas
Complejidad de consultas	Bueno para consultas complejas	Consultas complejas costosas
Experiencia previa	Media	Nula
Curva de aprendizaje	Leve	Media
Encaje con el proyecto	Muy alto	Medio

Finalmente, se ha optado por un modelo de base de datos relacional. Su encaje con el proyecto es muy alto debido a la naturaleza de sus datos [27].

Mientras que el modelo no relacional ofrece una flexibilidad que no se requiere para este proyecto, el modelo relacional garantiza la integridad de los datos mediante esquemas definidos y relaciones, fundamentales para conectar a los usuarios con sus datos. En cuanto a la experiencia previa y la curva de aprendizaje, el modelo SQL resulta mucho más accesible, lo que permite realizar consultas complejas de forma óptima y sin el sobre coste computacional que supondría implementarlas en un sistema NoSQL [28].

5.7. SERVIDOR DE BASE DE DATOS

Tras definir que se utilizará un modelo relacional, es necesario seleccionar el software encargado de gestionar la información. En esta sección se analizarán diferentes servidores de bases de datos para encontrar aquel que ofrezca el mejor rendimiento, la capacidad de realizar consultas complejas y la integración con otros servicios. Además, se valorará la combinación de un servidor tradicional con un servicio gestionado.

Criterios de evaluación

- **Tipo de servicio:** flexibilidad de la herramienta para operar tanto en entornos locales como en plataformas como servicio.
- **Curva de aprendizaje:** facilidad para configurar y administrar el servidor sin dedicar tiempo excesivo a la infraestructura.
- **Puntos fuertes:** características diferenciales, como la velocidad de lectura o la gestión de datos específicos.

- **Puntos débiles:** carencias o desventajas operativas de cada alternativa frente a las demás.
- **Limitaciones:** restricciones técnicas o costes asociados al uso del servicio.
- **Encaje con el proyecto:** grado en que la herramienta satisface las necesidades específicas de este proyecto.

Conclusiones

Con base en las alternativas seleccionadas, se realiza una comparación bajo los criterios de evaluación mencionados anteriormente en la Tabla 5-7.

Tabla 5-7 Comparativa de servidores de bases de datos

Característica	PostgreSQL	MySQL	SQL Server
Tipo	Híbrido (SGBD / DBaaS)	Híbrido (Clásico / Nube)	Ecosistema PaaS
Curva de aprendizaje	Media	Media	Alta
Puntos fuertes	Consultas complejas, geodatos (PostGIS)	Muy rápido en lecturas	Integración muy buena con ecosistema Microsoft
Puntos débiles	Mayor exigencia de configuración	Falta de algunas funcionalidades modernas y menor potencia en geodatos.	Dependencia fuerte del ecosistema de Microsoft
Limitaciones	Algo complejo de administrar	Menos orientado a analítica compleja	Licencias, foco empresarial
Encaje con el proyecto	Muy alto	Alto	Medio

Tras realizar la comparativa, se observa que, aunque MySQL ofrezca una menor curva de aprendizaje y un muy buen rendimiento en lecturas, presenta puntos débiles en cuanto a la potencia de sus funciones [29]. Por otro lado, SQL Server se descarta debido a su fuerte dependencia del ecosistema de Microsoft y a las posibles limitaciones económicas de sus licencias [30].

Finalmente, se elige PostgreSQL como servidor de base de datos por destacarse como la opción más equilibrada [31]. A pesar de su mayor exigencia de configuración, su mejor gestión de consultas complejas compensa el esfuerzo adicional. Por todo ello, su encaje en el proyecto es el más elevado entre las distintas alternativas.

5.8. GESTIÓN DE AUTENTICACIÓN

La seguridad de los datos personales y la correcta identificación de los usuarios son requisitos fundamentales para cualquier plataforma. En este apartado se evalúa el método para gestionar el registro, el inicio de sesión y la protección de las credenciales. Esta decisión influye directamente en la seguridad, la facilidad de desarrollo y el mantenimiento del sistema.

Criterios de evaluación

- **Protección de datos:** evalúa dónde se almacenan físicamente los datos, así como otros aspectos, como el cumplimiento de la normativa vigente.

- **Facilidad de implementación:** rapidez para integrar el sistema de usuarios en el frontend y el backend mediante herramientas y librerías existentes.
- **Seguridad por defecto:** nivel de protección intrínseco que ofrece la solución frente a ataques comunes sin necesidad de medidas complejas.
- **Flexibilidad / Control:** capacidad del desarrollador para personalizar el flujo de autenticación o los datos almacenados.
- **Coste:** análisis de la viabilidad económica, priorizando los niveles gratuitos.
- **Encaje con el proyecto:** grado en el que la solución se adapta a las necesidades específicas del proyecto.

Conclusiones

Con base en las alternativas seleccionadas, se realiza una comparación según los criterios de evaluación mencionados anteriormente en la Tabla 5-8.

Tabla 5-8 Comparativa de gestión de autenticación

Característica	Firestore Auth	Auth propia	Supabase Auth
Protección de datos	Datos fuera de la UE. RGPD posible	Control total de datos, pero toda la responsabilidad	Regiones de la UE. Control fino vía Postgres
Facilidad de implementación	Muy alta	Media	Alta
Seguridad por defecto	Alta	Depende de seguir bien OWASP	Alta
Flexibilidad / control	Menos flexible	Máximo control	Alto
Coste	Plan gratuito elevado	Solo infraestructura	Plan gratuito + pago por uso moderado
Encaje con el proyecto	Muy alto	Bueno	Alto

Tras analizar las diferentes alternativas, se observa que el desarrollo de una autenticación propia ofrece el máximo control y el cumplimiento más directo de la protección de datos al residir en servidores locales. Sin embargo, supone un riesgo elevado, ya que la seguridad depende del cumplimiento correcto de los estándares OWASP [32], lo que aumenta el tiempo de desarrollo. Por otro lado, Supabase Auth se presenta como una opción equilibrada, pero con una menor facilidad de implementación que la de Firestore [33].

Finalmente, se escoge Firestore Auth como método de gestión de la autenticación [34]. Destaca como la solución más eficiente, pues a pesar de ser menos flexible y almacenar datos en servidores externos, ofrece seguridad por defecto y una gran facilidad de implementación. Además, su plan gratuito cubre perfectamente las necesidades del sistema, por lo que su encaje con el proyecto es el más alto.

5.9. MOTOR DE RECOMENDACIÓN

El motor de recomendación constituye el núcleo inteligente de la aplicación, siendo el responsable de transformar los datos de los establecimientos y el comportamiento de los usuarios en sugerencias personalizadas. Su función principal es filtrar la información disponible para ofrecer una experiencia a medida que facilite la toma de decisiones del usuario.

Criterios de evaluación

- **Datos necesarios:** tipo de información que exige cada enfoque para funcionar correctamente.
- **Cantidad de datos:** volumen de datos necesario para entrenar el modelo y ofrecer recomendaciones fiables.
- **Complejidad:** esfuerzo técnico necesario para poner el método en funcionamiento.
- **Coste computacional:** recursos que consume el método tanto en entrenamiento como en ejecución.
- **Ventajas principales:** puntos fuertes específicos de cada alternativa.
- **Puntos débiles:** limitaciones o riesgos propios de cada enfoque.

Conclusiones

Con base en las alternativas seleccionadas, se realiza una comparación bajo los criterios de evaluación mencionados anteriormente en la Tabla 5-9

Tabla 5-9 Comparativa Motor de recomendación

Característica	LLM	Red neuronal con Keras	Algoritmo clustering	Sistema de similitudes
Datos necesarios	Texto + historial	Historial estructurado	Historial o atributos	Atributos + pequeño historial
Cantidad de datos	Alta	Alta	Media	Baja-Media
Complejidad	Alta	Media-Alta	Media	Baja-Media
Coste computacional	Alto	Medio	Bajo-Medio	Bajo
Ventajas	Criterios complejos y explicaciones en lenguaje natural	Modelo flexible y potente	Segmentación interpretable	Recomendaciones simples y transparentes
Puntos débiles	Complejo de implementar y mantener	Altamente dependiente de los datos de entrenamiento	Puede perder preferencias individuales	No usa directamente el comportamiento de otros usuarios

Tras analizar las alternativas, se observa que, aunque todas poseen ventajas distintas, presentan limitaciones importantes para el alcance del proyecto. El uso de un LLM se descarta debido a su elevada complejidad, pues tras la prueba de concepto del Sprint 6 se confirmó que su implementación resulta excesiva para el alcance de la aplicación. Por otro lado, los algoritmos de clustering tienden a simplificar los perfiles y a generalizar el comportamiento, lo que conlleva el riesgo de perder las preferencias individuales de cada usuario. Finalmente, los sistemas de similitud, a pesar de la baja cantidad de datos necesarios, se consideran insuficientes, ya que ofrecen recomendaciones lineales que limitan la capacidad de descubrimiento y de generar recomendaciones complejas [35].

Finalmente, se ha seleccionado una red neuronal con Keras como motor de recomendación [36]. Esta decisión se fundamenta en la flexibilidad y la potencia que ofrece para gestionar relaciones no lineales entre los usuarios y los establecimientos. A pesar de que este enfoque requiere un trabajo exhaustivo de preprocesamiento de datos, permite construir un sistema capaz de aprender a partir de los atributos del restaurante y del historial del usuario.

5.10. DESPLIEGUE

El entorno de despliegue constituye la base operativa del proyecto, siendo el responsable de trasladar el sistema desde su fase de desarrollo a un entorno de producción real. Su función principal es proporcionar la infraestructura necesaria para alojar los diferentes componentes de la aplicación, garantizando que todos estén accesibles para los usuarios finales de forma estable, segura y escalable.

Criterios de evaluación

- **Entorno:** indica dónde se alojan, se compilan y se ejecutan los servicios y componentes de la aplicación.
- **Facilidad de despliegue:** nivel de complejidad y tiempo requerido para configurar la infraestructura inicial y poner el sistema en funcionamiento.
- **Coste:** inversión económica necesaria para el aprovisionamiento, la operación y la disponibilidad continua de los recursos.
- **Accesibilidad:** capacidad de la infraestructura para exponer el sistema de forma segura al exterior, permitiendo el consumo de la aplicación por parte de usuarios finales desde redes externas y ubicaciones geográficas diversas.
- **Mantenimiento:** esfuerzo continuo del desarrollador para asegurar la estabilidad, la seguridad, la monitorización y la actualización del entorno.
- **Personalización:** nivel de control y configuración disponible sobre el entorno de despliegue.
- **Escalabilidad:** capacidad del entorno para adaptar de forma elástica sus recursos ante un aumento de la carga de trabajo o de la concurrencia de usuarios, sin afectar el rendimiento del sistema.

Conclusiones

Con base en las alternativas seleccionadas, se realiza una comparación bajo los criterios de evaluación mencionados anteriormente en la Tabla 5-10

Tabla 5-10 Comparativa Alternativas de despliegue

Característica	PaaS	IaaS en la nube	Servidor propio
Entorno	Plataforma externa	Plataforma externa	Máquina remota administrada
Facilidad de despliegue	Alta	Alta	Media
Coste	Variable	Variable	Medio
Accesibilidad	Alta	Alta	Alta
Mantenimiento	Bajo	Bajo-Medio	Medio-Alto
Personalización	Media-Baja	Alta	Muy alta
Escalabilidad	Alta	Muy alta	Media

Tras analizar las alternativas, se observa que PaaS ofrece una alta facilidad de despliegue. Sin embargo, aunque simplifica la puesta en marcha y el mantenimiento, puede implicar ciertas limitaciones de personalización al depender del proveedor [37]. Por otro lado, el uso de un servidor propio se presenta como una opción que otorga control total sobre el entorno, pero con una elevada carga de mantenimiento y administración, además de un desembolso inicial para adquirir la infraestructura necesaria.

Finalmente, se escoge IaaS en la nube como infraestructura para el alojamiento. Destaca como la opción más eficiente, pues ofrece accesibilidad total, alta disponibilidad y escalabilidad inmediata [38]. Además, al delegar la gestión, permite centrar los esfuerzos en el resto del proyecto.

Capítulo 6 REQUISITOS DEL SISTEMA

En esta sección se recogen los requisitos del sistema, los cuales describen las condiciones y funcionalidades que la aplicación debe cumplir para satisfacer las necesidades planteadas en el proyecto.

Por un lado, los requisitos funcionales hacen referencia a lo que el sistema debe hacer desde el punto de vista del usuario. Por otro lado, los requisitos no funcionales definen las restricciones de calidad del sistema, como el rendimiento, la seguridad o la usabilidad.

6.1. REQUISITOS FUNCIONALES

Para definir el alcance funcional de la aplicación se ha utilizado el framework User Story Mapping. Como se puede ver en Figura 6-2 Story mapping y, esta técnica organiza el desarrollo en una estructura bidimensional donde el eje horizontal agrupa las funcionalidades principales de negocio de manera secuencial, mientras que el eje vertical distribuye las historias de usuario a lo largo de los sprints planificados para este proyecto.

Figura 6-1 Story mapping 1/2



Figura 6-2 Story mapping 2/2



Una vez detallada la distribución temporal y los bloques de trabajo en el story mapping, se procede a detallar de manera individual cada una de las historias del proyecto.

Sprint 1:

- #1) Planificación inicial (6h):** como desarrollador, quiero realizar la planificación inicial del proyecto.
 - **Criterios de aceptación:** se elaborará un diagrama de Gantt que muestre las diferentes fases del desarrollo, indicando en cada una el tiempo estimado en horas y las dependencias entre ellas. De forma adicional, se podrá realizar una planificación más detallada de alguno de los sprints en la que se indiquen las diferentes tareas a realizar.
- #2) Análisis alternativas tecnológicas (12h):** como desarrollador, quiero analizar las alternativas tecnológicas de desarrollo para el proyecto.
 - **Criterios de aceptación:** se realizará una investigación inicial para determinar las posibles tecnologías frontend y backend, así como las APIs a utilizar. Una vez identificadas, se compararán entre sí y se seleccionarán aquellas que mejor se adapten al proyecto en cada caso.
- #3) Identificar requisitos de usuario (8h):** como desarrollador, quiero identificar los requisitos de usuario.

- **Criterios de aceptación:** se identificarán los requisitos de usuario del proyecto a realizar, que recogerán las diferentes necesidades a cubrir. Se especificará una breve descripción de cada uno y se organizarán en una lista jerárquica, con los subrequisitos correspondientes para aquellos que se consideren necesarios.
- #4) **Determinar alcance del proyecto (4h):** como desarrollador, quiero determinar el alcance del proyecto.
- **Criterios de aceptación:** se describirán la situación inicial del proyecto, las diferentes funcionalidades que el sistema podrá cumplir y las que no, delimitando así el alcance del proyecto.

Sprint 2:

- #5) **Inicio de sesión (5h):** como usuario, quiero poder iniciar sesión en la aplicación.
- **Criterios de aceptación:** un usuario podrá acceder a la aplicación proporcionando dos datos de identificación: por un lado, el correo electrónico o el nombre de usuario, y por otro, la contraseña. Tras una identificación correcta, se redirigirá al usuario a la página principal de la aplicación.
De forma adicional, se debe permitir al usuario recuperar su contraseña en caso de que se olvide de ella.
- #6) **Seleccionar tecnologías (5h):** como desarrollador, quiero seleccionar las tecnologías a utilizar.
- **Criterios de aceptación:** con base en el análisis de alternativas del primer sprint, y pudiendo complementar con información adicional, se determinarán de forma definitiva las tecnologías a usar durante el desarrollo del proyecto.
- #7) **Revisar requisitos (5h):** como alumno del TFG, quiero revisar los requisitos de usuario.
- **Criterios de aceptación:** con base en la retroalimentación proporcionada por los tutores, se realizará una mejora de los requisitos de usuario ya propuestos.
- #8) **Redactar introducción (2h):** como alumno del TFG, quiero realizar la introducción de la memoria.
- **Criterios de aceptación:** se redactará la introducción de la memoria indicando el tema del proyecto, así como sus características principales.
- #9) **Diseño inicial arquitectura (3h):** como alumno del TFG, quiero realizar un diseño inicial de la arquitectura del proyecto.
- **Criterios de aceptación:** se decidirá la arquitectura general del sistema, así como la arquitectura que seguirán tanto el backend como el frontend de la aplicación. No se entrará en un alto nivel de detalle. Se deberá añadir un diagrama general de la arquitectura.

Sprint 3:

- #10) **Registro en la aplicación (4h):** como usuario, quiero poder crear una cuenta en el sistema.
- **Criterios de aceptación:** a partir de una serie de datos, un usuario podrá crear una cuenta en el sistema que se almacene de forma permanente. Estos datos serán: nombre de usuario, correo electrónico, nombre, apellidos, contraseña y ubicación preferida.
Todos los campos serán obligatorios y, una vez introducidos, se redirigirá a la pantalla de inicio de sesión.

#11) Página principal (2h): como usuario, quiero acceder a la vista general de la aplicación.

- **Criterios de aceptación:** tras iniciar sesión, se redirigirá a los usuarios a una pantalla general de la aplicación desde la que se podrá acceder a las diferentes funcionalidades del servicio.

#12) Mejorar inicio de sesión (4h): como desarrollador, quiero mejorar el inicio de sesión de la aplicación.

- **Criterios de aceptación:** tras realizar la versión inicial, se realizarán cambios en el inicio de sesión para que sea completamente funcional y realice las comprobaciones correctas. Deberá tanto diferenciar correctamente entre el uso del nombre de usuario y del correo electrónico como proporcionar al usuario un token de sesión.

#13) Redactar situación inicial (4h): como alumno del TFG, quiero redactar la situación inicial.

- **Criterios de aceptación:** se realizará un análisis de la situación inicial para comprobar la viabilidad del proyecto.

#14) Evaluación de herramientas actuales (4h): como alumno del TFG, quiero realizar la evaluación de herramientas actuales.

- **Criterios de aceptación:** se analizarán otras aplicaciones existentes y se determinarán las diferencias con respecto a la aplicación a desarrollar.

#15) Ampliar diseño de arquitectura (2h): como alumno del TFG, quiero ampliar el diseño de la arquitectura del sistema.

- **Criterios de aceptación:** se continuará con el diseño iniciado en el sprint anterior y se especificarán las características principales de las arquitecturas seleccionadas. Se añadirá un diagrama de la arquitectura de cada componente del sistema.

Sprint 4:

#16) Recomendación de restaurantes (8h): como usuario, quiero recibir recomendaciones de restaurantes.

- **Criterios de aceptación:** desde la pantalla principal, se permitirá al usuario solicitar una recomendación de restaurantes en función de la ubicación, los precios y las preferencias. Se tomará la ubicación almacenada al crear la cuenta del usuario, o se permitirá seleccionar una nueva. Se permitirá al usuario seleccionar un rango de precios. Para indicar preferencias, se permitirá al usuario elegir entre un conjunto de etiquetas predefinidas. Se permitirá al usuario seleccionar si desea incluir únicamente establecimientos abiertos.

#17) Historial de restaurantes (2h): como usuario, quiero visualizar el historial de mis restaurantes visitados.

- **Criterios de aceptación:** se permitirá al usuario acceder a una vista específica para consultar su historial. En ella se mostrarán, en una lista ordenada, los restaurantes que ha visitado, apareciendo primero los establecimientos más recientes. Además, los restaurantes estarán agrupados por el mes que se han visitado.

#18) Restaurantes favoritos (4h): como usuario, quiero visualizar mis restaurantes favoritos.

- **Criterios de aceptación:** se permitirá al usuario acceder a una vista específica para consultar sus restaurantes favoritos. En ella, el usuario podrá elegir una de sus listas de favoritos, cada una de las cuales contendrá los diferentes establecimientos almacenados por el usuario, y los añadidos más recientes aparecerán primero.
- #19) Determinar servidor de BD (6h):** como desarrollador, quiero determinar el servidor de base de datos a utilizar.
- **Criterios de aceptación:** se realizará el análisis de alternativas de manera análoga a tecnologías anteriores, con diferentes servicios de bases de datos para almacenar a los usuarios y diferentes valores del sistema.
- #20) Redactar análisis de alternativas (6h):** como alumno del TFG, quiero redactar el análisis de las alternativas seleccionadas.
- **Criterios de aceptación:** se redactará el análisis de alternativas realizado en sprints anteriores, explicando los aspectos negativos de las alternativas no seleccionadas e indicando de forma razonada la opción elegida.
- #21) Modelo de datos del sistema (6h):** como alumno del TFG, quiero elaborar el modelo de datos del sistema.
- **Criterios de aceptación:** se redactará en la memoria el apartado "Modelo de datos del sistema", que explicará la estructura de clases, su funcionamiento e incluirá un diagrama para representarlo.
- #22) Diseño del plan de pruebas (8h):** como alumno del TFG, quiero diseñar el plan de pruebas.
- **Criterios de aceptación:** se diseñarán las pruebas de las diferentes interfaces de la aplicación, analizando las posibles situaciones con base en las entradas y salidas de cada una.

Sprint 5:

- #23) Ordenar restaurantes (8h):** como usuario, quiero poder ordenar los restaurantes recomendados.
- **Criterios de aceptación:** tras realizar una búsqueda de restaurantes, se permitirá al usuario ordenarlos según una de las siguientes características: cercanía, valoración o popularidad.
Por defecto, los restaurantes aparecerán ordenados por valoración.
- #24) Guardar para más tarde (8h):** como usuario, quiero poder guardar un restaurante para más tarde.
- **Criterios de aceptación:** junto con la información mostrada de un restaurante al realizar una búsqueda, se añadirá un botón para guardarlo para más tarde. Los establecimientos almacenados se mostrarán en una vista con un formato similar al historial, en una lista ordenada por fecha de añadido. Se mostrarán los mismos datos del restaurante que en la vista de recomendación. Una vez seleccionado un restaurante de esta lista, se eliminará de ella.
- #25) Valorar restaurante (8h):** como usuario, quiero valorar un restaurante que he visitado.
- **Criterios de aceptación:** tras visitar un restaurante, se habilitará un botón "Valorar restaurante" en el historial. Este llevará a una vista en la que se permitirá al usuario indicar la valoración de su experiencia en el establecimiento. Contará

con los siguientes campos, valorados de 1 a 5 estrellas: calidad, precio, trato e higiene.

Opcionalmente, el usuario podrá añadir un comentario para expresar su opinión con mayor detalle. El sistema notificará al usuario sobre los restaurantes que aún no han sido reseñados en el historial. En caso de visitar el mismo restaurante dos veces, no será necesario realizar la valoración de nuevo. El usuario podrá acceder al historial y modificar la reseña de un restaurante en cualquier momento.

#26) Ver reseñas de usuarios (8h): como usuario, quiero ver reseñas de otros usuarios.

- **Criterios de aceptación:** al consultar los detalles de un restaurante, se permitirá al usuario desplegar un apartado de reseñas de otros usuarios. En él, se mostrarán, junto con el nombre del usuario que ha realizado la reseña, su valoración de los diferentes aspectos de un local y el comentario adicional, en caso de haberse redactado.

Se podrá indicar si el comentario ha resultado útil al usuario mediante un botón, y se mostrará el número de usuarios a los que el comentario les ha gustado.

#27) Redactar objetivos del sistema (8h): como alumno del TFG, quiero redactar los objetivos del sistema.

- **Criterios de aceptación:** se elaborará una lista de los objetivos del sistema, indicando qué función o funciones representa cada uno.

Sprint 6:

#28) Sugerencias por ubicación (4h): como usuario, quiero recibir sugerencias basadas en mi ubicación.

- **Criterios de aceptación:** en la vista principal de la aplicación, se mostrará una lista de restaurantes sugeridos, con los establecimientos que más han sido seleccionados por los usuarios en su ubicación actual. Se mostrarán los mismos datos de los restaurantes que en la vista de recomendación.

#29) Investigar uso de LLM (6h): como desarrollador, quiero investigar el uso de un LLM para el proyecto.

- **Criterios de aceptación:** con base en el análisis de alternativas correspondiente, se investigarán métodos para implementar inteligencia artificial en el motor de recomendación del proyecto. Se planteará el uso de un sistema de recomendación basado en algoritmos en lugar de un LLM.

#30) Mejorar UI de la aplicación (6h): como desarrollador, quiero mejorar la UI de la aplicación.

- **Criterios de aceptación:** se modificarán diferentes aspectos de la interfaz de usuario para que sea más agradable visualmente y cumpla con las prácticas de usabilidad y accesibilidad.

#31) Diseño inicial de interfaces (8h): como alumno del TFG, quiero realizar el diseño inicial de las interfaces del sistema.

- **Criterios de aceptación:** se diseñarán las diferentes interfaces de la aplicación para su posterior implementación y se explicará cada uno de los elementos que contienen. Además, se añadirá un diagrama de transiciones entre las distintas vistas.

#32) Diseño en detalle de la arquitectura (6h): como alumno del TFG, quiero realizar el diseño en detalle de la arquitectura.

- **Criterios de aceptación:** se describirá en detalle la arquitectura de la aplicación, explicando con mayor detalle los diferentes componentes de la misma.

Sprint 7:

#33) Editar perfil (2h): como usuario, quiero editar mi perfil.

- **Criterios de aceptación:** el usuario debe poder acceder, desde el menú desplegable, a su perfil, donde puede consultar los datos ingresados en el registro. Además, podrá cambiar: nombre y apellidos, contraseña (para la cual se solicitará la contraseña antigua para evitar robos de cuenta), correo electrónico (para el cual se solicitará la contraseña antigua para evitar robos de cuenta) y ubicación preferida.
El usuario también podrá eliminar su cuenta de forma permanente.

#34) Implementar algoritmo de recomendación (12h): como desarrollador, quiero implementar el algoritmo de recomendación.

- **Criterios de aceptación:** se implementará el algoritmo seleccionado en el sistema de recomendación de la aplicación. Deberá ofrecer a cada usuario una lista personalizada de restaurantes en función de sus gustos y de los locales que ha visitado.

#35) Investigar el despliegue (6h): como desarrollador, quiero investigar el despliegue de la aplicación.

- **Criterios de aceptación:** se realizará una investigación para seleccionar una alternativa de despliegue y la estructura a seguir para albergar las diferentes partes del sistema.

#36) Adaptar la memoria (8h): como alumno del TFG, quiero adaptar la memoria.

- **Criterios de aceptación:** con base a la alternativa seleccionada en el sprint anterior, se realizará una adaptación de algunos apartados de la memoria para que sea coherente con la opción elegida.

#37) Redactar estructura de la aplicación (6h): como alumno del TFG, quiero redactar la estructura de la aplicación.

- **Criterios de aceptación:** se explicará la organización del proyecto en carpetas y su relación con las capas descritas en el diseño de la arquitectura. Se realizará un diagrama de clases y se explicarán algunos flujos principales, así como los problemas encontrados que motivaron la creación o modificación de clases.

#38) Redactar la planificación (6h): como alumno del TFG, quiero redactar la planificación.

- **Criterios de aceptación:** se redactará formalmente la planificación de sprints realizada al inicio del desarrollo del proyecto, explicando las tareas realizadas en cada sprint.

Sprint 8:

#39) Autenticación con Google (8h): como usuario, quiero iniciar sesión o registrarme con Google.

- **Criterios de aceptación:** desde la interfaz de inicio de sesión, el usuario podrá seleccionar Google como método de autenticación. Para iniciar sesión, se utilizará su correo electrónico y se buscará en la base de datos. Para el registro, se recopilarán todos los datos posibles de la cuenta de Google, dejando al usuario introducir únicamente el nombre de usuario y la ubicación preferida.

#40) Realizar Story Mapping (8h): como alumno del TFG, quiero realizar el Story Mapping.

- **Criterios de aceptación:** se diseñará el Story Mapping que reflejará todas las historias de usuario desarrolladas a lo largo del proyecto. Las historias se ordenarán por grupos de funcionalidades y por sprints.

#41) Realizar el presupuesto (8h): como alumno del TFG, quiero realizar el presupuesto final del proyecto.

- **Criterios de aceptación:** se deberán presupuestar todos los costes asociados al proyecto según la planificación inicial.

#42) Redactar los riesgos (8h): como alumno del TFG, quiero identificar los riesgos del proyecto.

- **Criterios de aceptación:** se identificarán y evaluarán los posibles riesgos de la aplicación, indicando además sus medidas preventivas y planes de contingencia en caso de que se materialicen.

#43) Requisitos no funcionales (8h): Como alumno del TFG, quiero redactar los requisitos no funcionales del proyecto.

- **Criterios de aceptación:** se redactarán de forma detallada los requisitos que debe cumplir el sistema para satisfacer los requisitos de usuario definidos anteriormente.

Sprint 9:

#44) Implementar despliegue (10h): como desarrollador, quiero implementar el despliegue de la aplicación.

- **Criterios de aceptación:** siguiendo el diseño realizado en la memoria, se configurará el despliegue de la aplicación para que pueda ser utilizada por los usuarios. Se redactará en la memoria el apartado “Implementación de despliegue”, indicando su funcionamiento.

#45) Implementar plan de pruebas (20h): como alumno del TFG, quiero implementar el plan de pruebas.

- **Criterios de aceptación:** se implementarán en la aplicación las pruebas diseñadas en sprints anteriores para comprobar su robustez y calidad.

#46) Redactar requisitos funcionales (10h): como alumno del TFG, quiero redactar los requisitos funcionales.

- **Criterios de aceptación:** se redactarán los requisitos funcionales de la aplicación con base en las historias de usuario previamente definidas. Además, se indicarán los criterios de aceptación de cada uno de ellos con un nivel de detalle mayor a los requisitos de usuario.

Sprint 10:

#47) Planificación final (8h): como alumno del TFG, quiero redactar la planificación final del proyecto.

- **Criterios de aceptación:** se elaborará un diagrama de Gantt que detalle las fechas reales de ejecución de cada fase. Además, se especificarán las razones por las que se produjo cada uno de los desvíos.

#48) Trabajo futuro (8h): como alumno del TFG, quiero determinar el trabajo futuro del proyecto.

- **Criterios de aceptación:** se redactarán las líneas de evolución para la aplicación posterior al Trabajo de Fin de Grado. Se deben nombrar los cambios a realizar junto con su descripción.
- #49) Manual de usuario (8h):** como alumno del TFG, quiero realizar el manual de usuario de la aplicación.
- **Criterios de aceptación:** el manual deberá describir de forma clara y estructurada el uso de la aplicación, indicando sus principales funcionalidades y los pasos necesarios para realizar las acciones más relevantes.
- #50) README del proyecto (8h):** como desarrollador, quiero detallar el README del proyecto.
- **Criterios de aceptación:** el README deberá describir de forma clara y concisa la finalidad de la aplicación, su funcionamiento y los pasos necesarios para su despliegue y ejecución. Además, incluirá información sobre las dependencias necesarias para configurar el entorno.
- #51) Conclusiones (8h):** como alumno del TFG, quiero redactar las conclusiones del proyecto.
- **Criterios de aceptación:** se recogerán las conclusiones de los objetivos del proyecto, la valoración global del desarrollo completo del sistema y se indicará si se han cumplido los requisitos establecidos. No se deberán introducir nuevas funcionalidades.

6.2. REQUISITOS NO FUNCIONALES

Los requisitos no funcionales definen las propiedades y restricciones que debe cumplir el sistema y establecen criterios de calidad que condicionan su funcionamiento. Estos requisitos abarcan aspectos como el rendimiento, la usabilidad, la seguridad, mantenibilidad y escalabilidad del sistema, con el objetivo de garantizar una buena experiencia de usuario y un funcionamiento eficiente de la aplicación.

RNF 1) Rendimiento

- RNF 1.1)** El tiempo de carga y renderizado de las interfaces no deberá superar los 2 segundos en condiciones normales de red.
- RNF 1.2)** El tiempo total desde que un usuario solicita una recomendación hasta que se le muestra la lista ordenada no debe superar los 6 segundos.
- RNF 1.3)** El backend y la base de datos deberán ser capaces de gestionar un mínimo de 50 peticiones concurrentes sin degradar los tiempos de respuesta del sistema.

RNF 2) Seguridad

- RNF 2.1)** El control de acceso y las sesiones de los usuarios se gestionarán mediante tokens con un tiempo de expiración de 24 horas.
 - RNF 1.2.1)** Los tokens se almacenarán de forma segura en el cliente mediante HTTPOnly Cookies que mitiguen los ataques.
- RNF 2.2)** Toda la comunicación entre la aplicación, el backend y las APIs se realizará mediante un protocolo criptográfico.
- RNF 2.3)** Las contraseñas de los usuarios nunca se almacenarán en texto plano en la base de datos. Se delegará la gestión en el servicio de Firebase.
- RNF 2.4)** En línea con el RGPD, la aplicación garantizará el derecho al olvido, permitiendo la eliminación completa del perfil de un usuario.

RNF 3) Disponibilidad

RNF 3.1) El sistema cloud deberá garantizar una disponibilidad del 99,5% anual, excluyendo las paradas de mantenimiento.

RNF 3.2) En caso de caída, timeout o agotamiento de la cuota de la API, el sistema no deberá bloquearse. El backend interceptará el error y activará un mecanismo para ofrecer resultados alternativos.

RNF 4) Mantenibilidad y escalabilidad

RNF 4.1) Tanto el backend como el frontend y la base de datos se estructurarán de forma aislada mediante contenedores. Esto garantizará la portabilidad absoluta del sistema y facilitará su escalabilidad.

RNF 4.2) El código crítico de la aplicación deberá estar respaldado por la suite de pruebas desarrollada.

RNF 5) Usabilidad

RNF 5.1) La aplicación se desarrollará utilizando tecnologías web adaptativas, lo que garantizará su correcta visualización, rendimiento y usabilidad en diferentes sistemas operativos y su adaptación a diferentes resoluciones de pantalla.

RNF 5.2) Cualquier acción principal de la aplicación no requerirá más de tres interacciones del usuario desde la pantalla principal.

6.2.1. Modelo de datos del dominio

Como se puede observar en la Ilustración 6-1, el modelo de datos del sistema define la estructura y la organización de la información utilizada por la aplicación, así como las relaciones entre las distintas entidades. El diseño de la base de datos se ha realizado siguiendo un modelo relacional, garantizando la integridad de la información y la eficiencia en las consultas y gestión de usuarios.

La arquitectura adoptada combina Firebase, un servicio externo para la gestión de usuarios y la autenticación, con una base de datos relacional en PostgreSQL para almacenar el resto de los datos relacionados con la lógica de la aplicación. De esta forma se simplifica la autenticación del sistema, delegando la gestión de la información personal en un tercero, simplificando procesos, como el registro o el inicio de sesión.

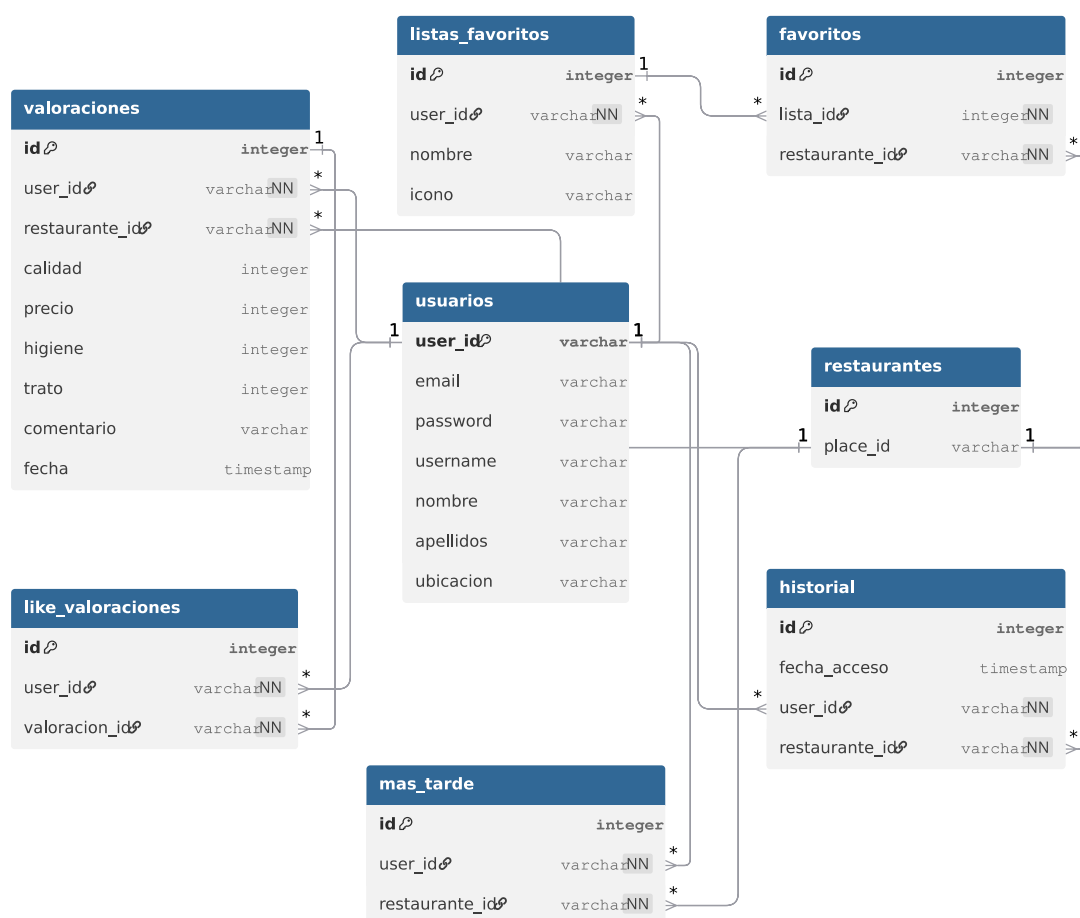


Ilustración 6-1 Modelo de datos

A continuación, se detallan las tablas que conforman el modelo de datos:

- **usuarios**: entidad central del modelo que almacena toda la información necesaria de los usuarios, tanto para la autenticación como para las recomendaciones.
- **restaurantes**: actúa como un registro local de los establecimientos con los que los usuarios interactúan. Su existencia centraliza las relaciones, evita la duplicidad de largas cadenas de texto y optimiza el rendimiento de las consultas.

- **historial:** almacena un registro temporal de los establecimientos seleccionados por el usuario. Además, se almacena la fecha de acceso para ordenar los restaurantes.
- **listas_favoritos:** almacena las listas de favoritos de los usuarios. No contiene los establecimientos en sí, sino que sirve como contenedor lógico para organizarlos.
- **favoritos:** vincula un lugar específico a una lista determinada, permitiendo que esta contenga distintos establecimientos.
- **mas_tarde:** almacena los restaurantes que el usuario desea visitar próximamente y funciona como una lista personalizada adicional.
- **valoraciones:** recoge las calificaciones del usuario de los establecimientos que ha visitado. Contiene un sistema multidimensional y un comentario textual para aportar información adicional.
- **like_valoraciones:** registra la interacción de los usuarios con las valoraciones de un restaurante. Almacena si un usuario ha indicado que le ha resultado útil el comentario de otro, permitiendo que cada usuario realice dicha acción una sola vez.

6.2.2. Interfaz de usuario

En este apartado se describe el diseño de las interfaces de la aplicación, concebida principalmente para su uso en dispositivos móviles.

El diseño está orientado a ofrecer una experiencia de usuario sencilla, intuitiva y eficiente que permite acceder rápidamente a las distintas funcionalidades disponibles. Para ello, se han priorizado aspectos como la claridad visual, la organización de contenidos, buscando mostrar la mayor cantidad de información en el menor espacio posible, y la adaptación a pantallas de reducido tamaño.

A lo largo de esta sección se presentarán las diferentes vistas que componen la aplicación, así como la estructura de navegación y los principales elementos de la interfaz.

Inicio de sesión

The image shows a hand-drawn prototype of a mobile application login screen. At the top, the app name 'DevorApp' is written. Below it, the title 'Iniciar Sesión' and the subtitle 'Bienvenido de nuevo' are centered. There are two input fields: the first is labeled 'Email o usuario' and contains the text 'tu@email.com'; the second is labeled 'Contraseña' and contains a series of dots, with a small eye icon to its right. Above the password field, the text '¿Olvidaste tu contraseña?' is written. Below the input fields is a large button labeled 'Entrar'. Underneath the button is a horizontal line with a small circle in the center. Below this line is another button labeled with the Google logo and the text 'Continúa con Google'. At the bottom, the text '¿No tienes cuenta? Regístrate' is written.

Prototipo 1 Inicio de sesión

El Prototipo 1 presenta, en primer lugar, un mensaje de bienvenida que refuerza la cercanía con el usuario. A continuación, se presentan los dos campos necesarios para la autenticación. El primero solicita el nombre de usuario o el correo electrónico, mientras que el segundo solicita la contraseña. Este último, además, presenta un botón para alternar la visibilidad del contenido introducido, facilitando su comprobación. Además, se incorpora una opción de recuperación de contraseña para los usuarios que no recuerden sus credenciales.

El botón de acceso se sitúa en una posición central para favorecer su identificación y uso.

Como alternativa al método de autenticación tradicional, hay un botón para usar una cuenta de Google. Finalmente, se incluye un enlace para que los nuevos usuarios puedan crear una cuenta en la aplicación.

Crear cuenta

DevorApp

Crear cuenta

Paso 1 de 2 - Datos de acceso

Email

Nombre de usuario

Contraseña

Nombre

Apellidos

Continuar


Regístrate con Google

¿Ya tienes cuenta? Inicia sesión

DevorApp

Crear cuenta

Paso 2 de 2 - Ubicación


Tu ubicación preferida
"Explicación"

Buscar ubicación

Gijón, Asturias

Ubicación detectada automáticamente

Crear cuenta

← Volver

Prototipo 2 Crear cuenta

El proceso de registro de nuevos usuarios se ha diseñado como un flujo dividido en dos pasos con el objetivo de simplificar la introducción de datos y para asegurar una buena experiencia en dispositivos móviles como se puede ver en el Prototipo 2.

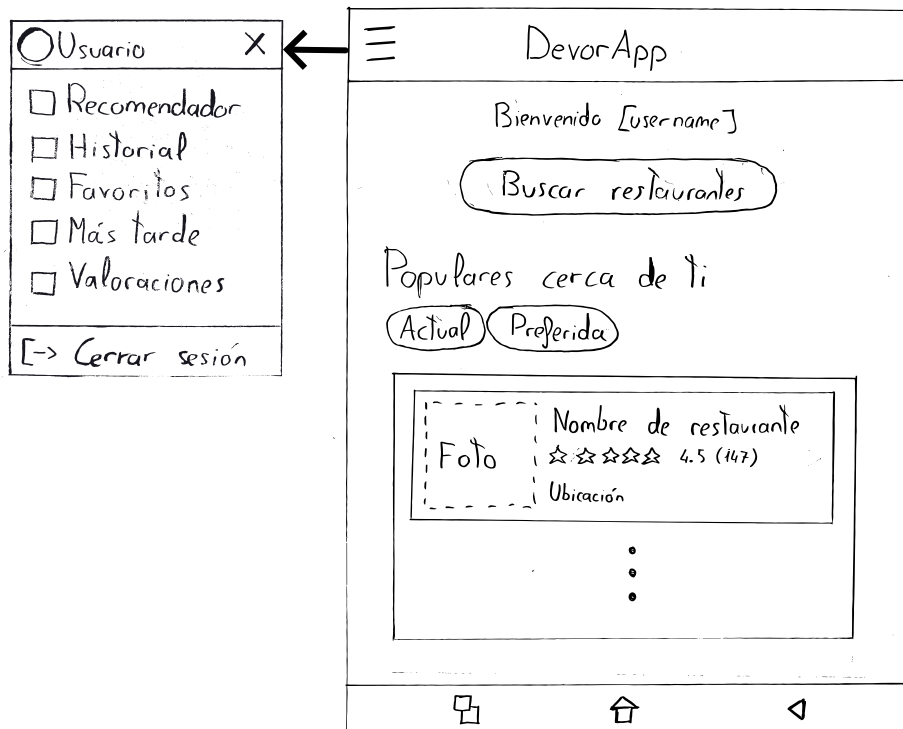
En el primer paso se solicita al usuario la información básica necesaria para la creación de la cuenta. Esta incluye el correo electrónico, nombre de usuario y la contraseña, así como datos personales como el nombre y los apellidos. El campo de contraseña permite visualizar la información introducida para evitar posibles errores en el registro. En caso de que el usuario ya tenga una cuenta creada, puede regresar al inicio de sesión mediante el enlace en la parte inferior de la interfaz.

Al igual que en el inicio de sesión, también se permite utilizar una cuenta de Google para realizar el registro en la aplicación.

El segundo paso del registro se centra en la configuración de la ubicación del usuario. En esta pantalla, el usuario puede introducir manualmente su ubicación preferida o permitir la detección automática de su localización actual. Además, dispone de una explicación para que los usuarios conozcan la utilidad de almacenar esta ubicación.

Finalmente, el usuario puede completar el registro mediante el botón de “Crear cuenta”, o por el contrario volver al paso anterior para revisar la información introducida o modificar algún dato si es necesario.

Página principal



Prototipo 3 Pantalla principal

Esta interfaz constituye la pantalla principal de la aplicación. En la parte superior se sitúa una barra de navegación fija que estará presente en las distintas vistas de la aplicación. En el centro se sitúa el logo de la aplicación, el cual funcionará como botón para volver a esta interfaz desde el resto de las vistas.

Como se ve en el Prototipo 3, al pulsar el botón de la esquina superior izquierda se despliega un menú. En su cabecera se muestra el nombre de usuario autenticado en la aplicación, mientras que en la parte inferior se encuentra el botón para salir de la sesión actual. El panel central del menú está compuesto por una lista vertical de las secciones disponibles en la aplicación. Cada una de estas opciones va precedida de un icono representativo, lo que reduce la carga cognitiva y agiliza el reconocimiento visual de los apartados.

Continuando con la página principal, justo debajo de la cabecera, se muestra un mensaje de bienvenida personalizado. A continuación, se incorpora un botón que redirige directamente a la interfaz de recomendación de restaurantes del Prototipo 4. Dado que constituye la funcionalidad principal de la aplicación, su diseño y ubicación central responden a la necesidad de ofrecer a los usuarios un acceso lo más simple y directo posible, guiando la interacción desde el primer momento.

Finalmente, se encuentra un apartado de sugerencias titulado “Populares cerca de ti”. Esto permite a los usuarios ver los locales con mayor afluencia de clientes en la zona. Para aportar flexibilidad, se incluyen dos botones de selección rápida que permiten al usuario alternar entre usar su geolocalización en tiempo real y la dirección configurada durante el registro. Los resultados se presentan en forma de lista. Cada uno expone la información más relevante de su establecimiento para facilitar la toma de decisiones.

Recomendación de restaurantes

The prototype consists of two hand-drawn screens for an app named 'DevorApp'.

Screen 1: Recomendador

- Header: DevorApp
- Section: Recomendador
- Categorías: A list with 'Pizza x' selected.
- Field: Añadir categoría (text input)
- Rango de precios: A row of four buttons: €, €, €, €. Below them, the words 'Económico' and 'Exclusivo' are written.
- Filters:
 - ☒ Incluir sitios sin precio confirmado
 - ☐ Solo lugares abiertos ahora
- Location:
 - ☐ Usar ubicación preferida "Ubicación"
 - ☐ Escoger otra ubicación
- Button: Buscar restaurantes

Screen 2: Sugerencias para ti

- Header: DevorApp
- Filters: Pizza x, €, €, Abierto ahora, and an Editar button.
- Section: Sugerencias para ti (Mejor valorados)
- Text: 10 resultados
- Results: A list of restaurant suggestions. Each entry is enclosed in a dashed box and contains:
 - Nombre de restaurante
 - Foto
 - Rating: ☆☆☆☆ 4.7 (867)
 - Ubicación
- Navigation: Anterior - 2 - Siguiente

Prototipo 4 Recomendación de restaurantes

El Prototipo 4 consta de dos interfaces. La primera funciona como el panel de configuración de búsqueda, en el que el usuario puede indicar sus preferencias para obtener recomendaciones precisas.

En primer lugar, se presenta la sección de categorías, que utiliza un sistema de etiquetas y un campo de texto para añadir preferencias gastronómicas. A continuación, se dispone el campo “Rango de precios”, en el que el usuario puede realizar una selección múltiple en una escala de cuatro niveles de asequibilidad. Esta sección se apoya en los textos “Económico” y “Exclusivo” para aportar claridad a la escala.

Para refinar aún más los resultados, se incluyen dos selectores adicionales: uno para incluir los restaurantes sin precio confirmado y otro para filtrar únicamente los establecimientos con apertura activa en el momento de la búsqueda. Como última opción de búsqueda se encuentra un selector de ubicación, que permite al usuario alternar entre su ubicación almacenada o escoger una dirección específica. El proceso finaliza con el botón “Buscar restaurantes” que redirige al usuario a la siguiente interfaz.

En esta segunda interfaz se muestran los establecimientos que cumplen los requisitos establecidos por el usuario. En la parte superior, se ha incorporado una barra de filtros rápidos. Se presentan en forma de píldoras interactivas que reflejan los parámetros previamente seleccionados. Además, en la parte derecha cuenta con un botón para editar los filtros y volver a la vista anterior.

Bajo esta barra se muestran los resultados obtenidos, precedidos por una etiqueta que indica la cantidad y por un selector de ordenación dinámica. Este último permite al usuario reordenar la lista según distintos criterios.

Los resultados finales aparecen siguiendo un patrón de tarjetas, en el que se muestra información básica de cada establecimiento, incluyendo una foto, su nombre, su valoración y su ubicación.

Finalmente, en la parte inferior de la lista se ha implementado un sistema de paginación para facilitar la navegación a través de grandes volúmenes de datos. Este componente está formado por los controles para avanzar y retroceder de página, junto con el número indicativo de la página actual.

Historial

Prototipo 5 Historial

El Prototipo 5 está diseñado para que el usuario pueda consultar un registro de los establecimientos que ha visitado en la aplicación.

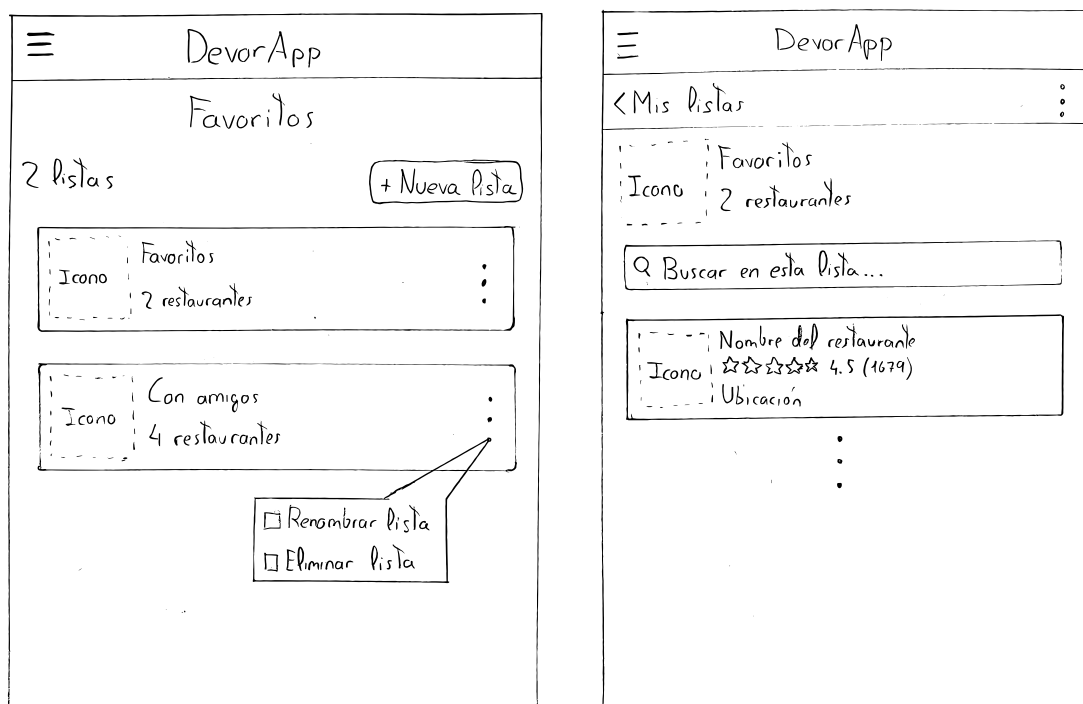
Bajo el título principal se ha incorporado una barra de búsqueda con un botón para filtrar los restaurantes por diferentes características. La inclusión de estas herramientas permite

al usuario localizar rápidamente un restaurante en concreto sin necesidad de realizar un desplazamiento manual extenso. Justo debajo de estos controles, la interfaz ofrece un resumen rápido del perfil, indicando el número de restaurantes visitados y cuántos de ellos están pendientes de valoración.

Para optimizar la interfaz y evitar la sobrecarga de información, el listado de restaurantes se ha estructurado en una agrupación cronológica mensual. Estas agrupaciones funcionan como paneles desplegable. Además, se indica el número de restaurantes visitados en ese periodo.

Al expandir uno de los meses, se muestra una lista similar a la de la pantalla principal, con la información clave de cada restaurante y el día en que fue visitado. Además, se incorpora una etiqueta visual en la parte superior derecha de algunos elementos con el texto “Sin reseña”. Esto funciona como un recordatorio visual y llamada a la acción para conseguir que el usuario valore sus visitas y así poder refinar el algoritmo de recomendación.

Favoritos



Prototipo 6 Favoritos

En la primera interfaz del Prototipo 6 se presentan al usuario sus listas de restaurantes favoritos guardadas. En la parte superior, junto al recuento total de listas, se ha ubicado un botón de acción para añadir nuevas colecciones.

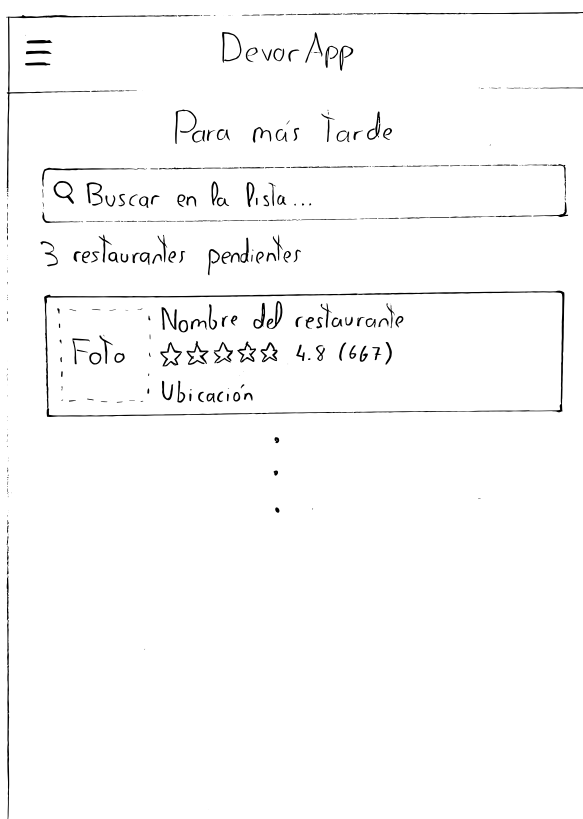
Las listas existentes se exponen mediante tarjetas rectangulares que actúan con contenedores. Cada una presenta un icono representativo y el nombre de la misma, junto con el número de restaurantes almacenados en ella. Se ha incorporado un menú en la parte derecha de cada tarjeta representado por el icono de tres puntos verticales. Al interactuar con él, se despliega un pequeño panel con dos opciones para gestionar las listas, “Renombrar lista” y “Eliminar lista”.

Al seleccionar una de las colecciones almacenadas, se muestra la segunda interfaz. Para garantizar que el usuario se mantenga orientado durante la navegación, se ha incorporado un botón de retroceso a la pantalla anterior en la esquina superior izquierda, que reemplaza el menú desplegable.

Al inicio de la interfaz, se repite la información de la lista seleccionada para confirmar al usuario su ubicación en la aplicación. A continuación, se ha integrado una barra de búsqueda local, al igual que en el historial, para filtrar los resultados de forma rápida y eficiente.

Finalmente, el contenido de la lista se muestra siguiendo el mismo diseño que en el resto de las interfaces, con tarjetas de los restaurantes mostrando información básica de los mismos.

Guardar para más tarde



Prototipo 7 Guardar para más tarde

El Prototipo 7 expone una sección de restaurantes guardados para más adelante. Su propósito es ofrecer al usuario un espacio en el que almacenar temporalmente aquellos restaurantes que han captado su atención durante la navegación, pero que aún no ha visitado.

Debajo del título de la sección, se ha integrado una barra de búsqueda local para garantizar la eficiencia de la interfaz. A medida que la lista de locales guardados crece, esta barra permite al usuario localizar opciones concretas rápidamente.

A continuación, se muestra un texto que actúa como contador de establecimientos. Este dato ofrece retroalimentación inmediata al usuario sobre la magnitud de su lista. Finalmente, los establecimientos guardados se muestran utilizando el mismo patrón que en el resto de las interfaces.

Valoraciones

Prototipo 8 Poner valoración

Prototipo 9 Reseñas de la comunidad

La interfaz Prototipo 8 muestra el formulario mediante el cual el usuario puede generar una nueva valoración para un restaurante visitado. La cabecera mantiene la consistencia de la navegación incorporando un botón de retroceso para volver al historial (Prototipo 5). Además, debajo del título de la sección, se encuentra el nombre del establecimiento, lo cual sirve para confirmar al usuario sobre qué local está opinando, previniendo posibles errores.

El núcleo de la interfaz reside en su sistema de evaluación multidimensional. En lugar de solicitar una única calificación general, el diseño desglosa la experiencia en cuatro parámetros. De esta forma, el usuario puede estructurar su opinión y, por otro lado, el algoritmo de recomendación puede refinarse con mayor precisión.

A continuación, se incluye un área de texto para añadir un comentario, de tal manera que pueda expresar su opinión más allá de los parámetros anteriormente comentados.

Finalmente, el usuario puede finalizar la acción con el botón situado en la parte inferior de la pantalla.

Por otro lado, la interfaz Prototipo 9 representa las reseñas que la comunidad deja a los diferentes restaurantes. Se integra dentro de los detalles de un restaurante como una sección desplegable para no sobrecargar de información al usuario.

En la parte superior de la sección se presenta un panel de resumen. En lugar de mostrar solo una nota media de cada uno de los aspectos anteriormente comentados, se utiliza una barra de progreso. Esto permite al usuario realizar un diagnóstico rápido del restaurante.

A continuación, se muestran las reseñas de todos los usuarios a modo de lista horizontal. Al inicio de cada una se observa el nombre de usuario y la antigüedad de la reseña, seguido de la puntuación del usuario en cada una de las cuatro categorías. Finalmente, aparece el comentario textual en caso de que exista. Además, en la parte inferior derecha, se encuentra un botón de “Me gusta”, que permite expresar a los usuarios que leen las reseñas si les ha resultado útil.



Prototipo 10 Mis valoraciones

El Prototipo 10 corresponde a la sección de valoraciones del usuario. Bajo el título de la sección, se incluye un contador informativo que ofrece una visión rápida de la cantidad de reseñas en la plataforma. Los diferentes restaurantes reseñados se disponen a modo de lista, al igual que en las interfaces anteriores.

Al desplegar una valoración, la interfaz revela el detalle completo de la reseña emitida por el usuario. Se muestra un desglose de las cuatro categorías, así como el comentario redactado. Al final de la sección desplegada se encuentran dos botones para realizar

diferentes acciones en la reseña seleccionada, uno para editarla y cambiar los valores establecidos (Prototipo 8), y otro para eliminar la reseña del local.

Detalles de restaurantes

< Historial Devor App

Foto

Nombre del restaurante

☆☆☆☆☆ 4.7 (248)

Ubicación

Visitado el 9 de abril de 2026

• Abierto ahora - Cierre a las 12:30 AM

Horario

Lunes	12:30 - 16:30	19:00 - 00:30
Martes	12:30 - 16:30	19:00 - 00:30
	•	
	•	

Enlaces de interés

☐ Sitio web ☐ Google Maps

Volver a seleccionar restaurante

Añadir a favoritos

Escribir reseña

Eliminar del historial

Prototipo 11 Detalles de restaurante

El Prototipo 11 representa la vista a la que accede el usuario tras seleccionar un restaurante en cualquiera de los listados de la aplicación.

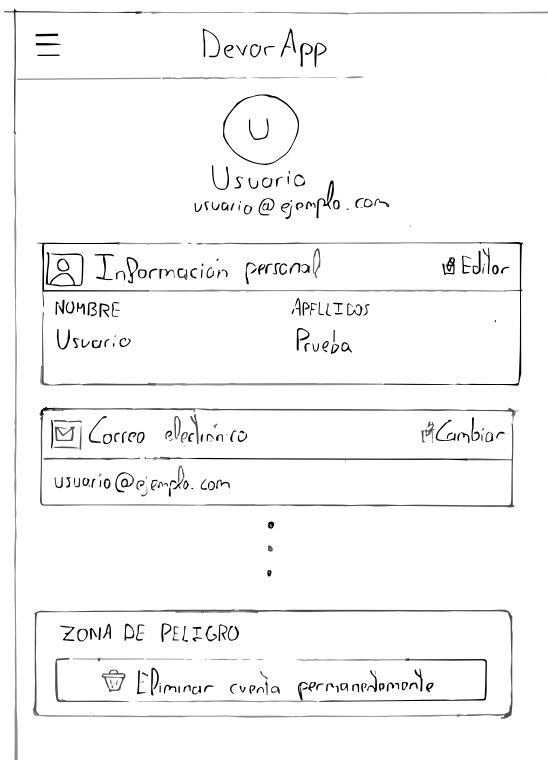
En la parte superior, el menú desplegable se sustituye por un botón de retroceso. Este detalle de usabilidad informa al usuario de dónde proviene y permite deshacer su acción de forma segura y predecible. En el ejemplo de la imagen, dicho botón llevaría al historial (Prototipo 5).

El bloque principal hereda y amplía las tarjetas de información de los restaurantes de otras interfaces. Muestra la fotografía, el nombre del local, la calificación, el número de valoraciones y la ubicación. Además, incorpora información específica según la vista de procedencia. En el caso del boceto, al provenir del historial muestra la fecha en la que se visitó el establecimiento. Inmediatamente a continuación se muestra el estado operativo del restaurante en tiempo real.

La sección central muestra la información operativa del local. Por un lado, presenta el horario detallado por días y turnos, estructurado en columnas para facilitar la lectura. Por otro lado, incorpora un bloque de “Enlaces de interés”, desde el cual se puede acceder al sitio web del restaurante o a su página en Google Maps. Esta integración evita que el usuario tenga que abandonar la aplicación para buscar información adicional.

Finalmente, la parte inferior de la pantalla está dedicada a la botonera de acciones. En ella, los botones disponibles cambian en función de la ruta de navegación que haya seguido el usuario. Este enfoque optimiza la interfaz móvil, evitando saturar al usuario con opciones innecesarias.

Perfil de usuario



Prototipo 12 Perfil de usuario

El Prototipo 12 constituye el centro de gestión de la identidad del usuario de la aplicación. Su objetivo principal es la visualización y edición de los datos de la cuenta.

En la parte superior se puede observar un avatar circular con las iniciales del usuario, acompañado del nombre y correo electrónico. De esta forma se confirma la sesión activa y se establece una jerarquía visual clara.

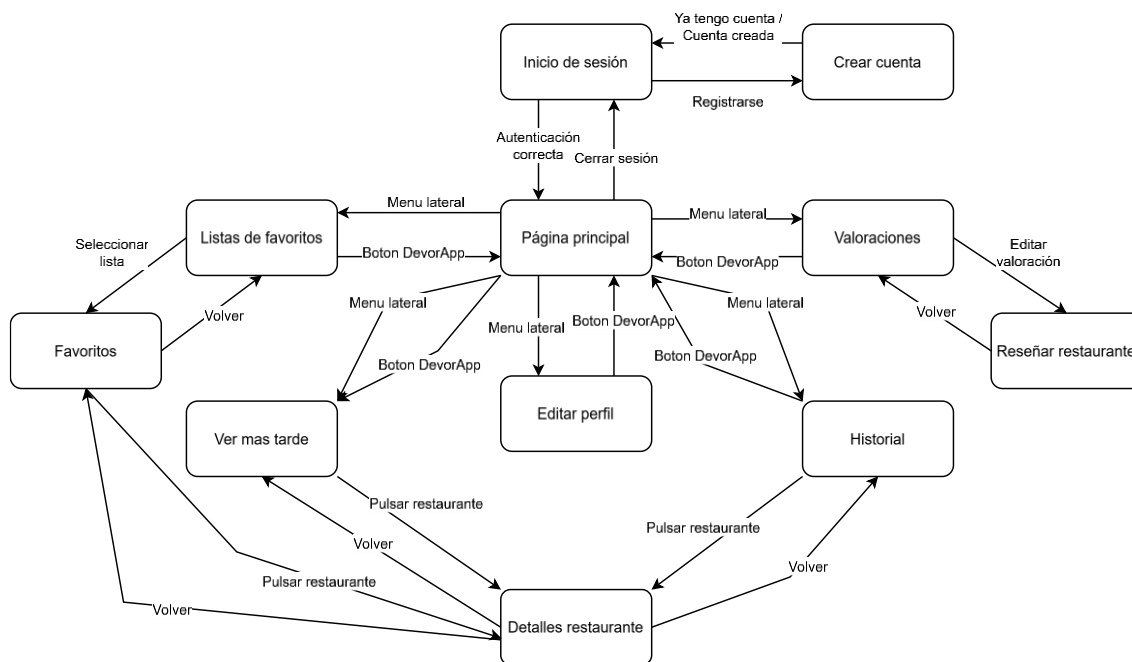
Justo debajo se encuentra el cuerpo principal de la interfaz. Está organizada en bloques de información independientes, estructurando los diferentes datos de forma modular. El primer bloque se centra en la información personal del usuario, permitiendo la consulta y edición tanto del nombre como de los apellidos. A continuación, se encuentra el bloque correspondiente al correo electrónico, al cual sucederán un bloque para la ubicación preferida y un último bloque que permita el cambio de contraseña.

Finalmente, en la parte inferior de la pantalla se encuentra la “Zona de peligro”. Este bloque se encuentra apartado del resto para resaltar su importancia. Cuenta con un botón que permite eliminar permanentemente la cuenta actual.

6.2.2.1. Diagrama de transiciones

A continuación, en la Figura 6-3 se presenta el diagrama de transiciones del sistema, el cual muestra el flujo y las rutas de navegación del usuario a través de las diferentes interfaces de la aplicación. Mediante esta representación se definen los eventos o acciones que provocan el cambio entre vistas.

Figura 6-3 Diagrama de transiciones



Capítulo 7 DISEÑO

7.1. DISEÑO DE LA ARQUITECTURA

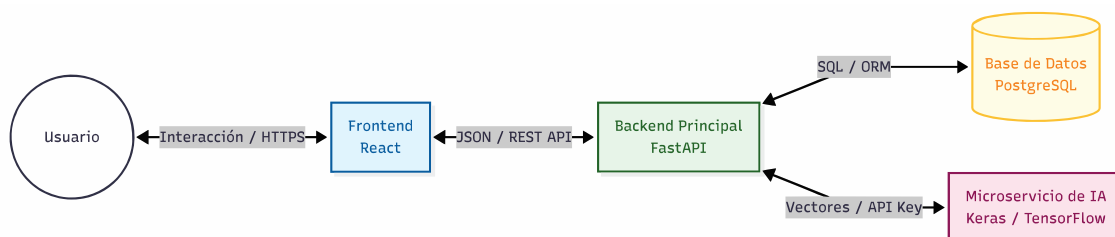
Para el desarrollo de este proyecto se ha decidido implementar una arquitectura distribuida cliente-servidor basada en tres capas principales. Esta separación de responsabilidades permite aislar la lógica de negocio de la interfaz de usuario y del acceso a los datos, facilitando futuras actualizaciones o integraciones sin afectar la integridad del sistema en su conjunto.

El sistema se divide en tres componentes independientes, cada uno con una arquitectura interna propia:

- **Frontend:** es la parte pública de la aplicación encargada de la interacción con el usuario final. Está implementada con la biblioteca React y su arquitectura interna se basa en una adaptación del patrón Modelo-Vista-Controlador orientado a componentes.
- **Backend:** actúa como el núcleo lógico y operativo del sistema. Ha sido desarrollado utilizando el framework FastAPI y su estructura interna sigue un patrón de arquitectura en N capas.
- **Microservicio de IA:** servicio encargado exclusivamente de cargar el modelo y ejecutarlo para el motor de recomendación. Se ha realizado utilizando Keras y TensorFlow como tecnologías principales.

Todos los subsistemas operan de manera desacoplada y se comunican entre sí de forma bidireccional a través de una API REST. El frontend consume los servicios del backend mediante peticiones HTTP, intercambiando información de forma estructurada, y el backend envía peticiones al modelo de recomendación, recibiendo los datos solicitados de vuelta.

Figura 7-1 Arquitectura general



Como se puede observar en el diagrama general Figura 7-1, el sistema se divide en los siguientes módulos principales:

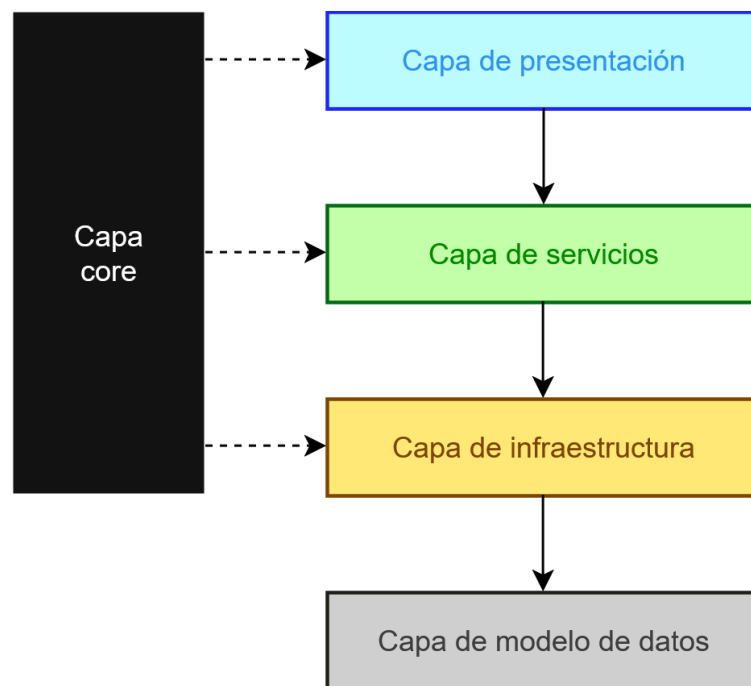
- **Cliente:** se encarga de la comunicación con el usuario final, siendo responsable de mostrar las diferentes interfaces, permitir la visualización de la información y capturar los diferentes eventos.
- **Servidor:** actúa como intermediario procesando las solicitudes de la capa de presentación, ejecutando el motor de recomendación y gestionando la persistencia en la base de datos.

- **Capa de persistencia:** se encarga de la persistencia de la información, garantizando un almacenamiento estructurado y seguro de los diferentes tipos de datos que se tratan en el sistema.
- **Motor de recomendación:** procesa los vectores de características de usuarios y restaurantes para devolver una puntuación de cada uno de los locales.

Arquitectura del backend

El backend del sistema se ha diseñado siguiendo una arquitectura de capas. Al igual que en la arquitectura general del proyecto, el objetivo principal es la separación de responsabilidades, además de garantizar la mantenibilidad y escalabilidad del backend.

Figura 7-2 Arquitectura Backend



A continuación, se describen las principales capas que conforman esta arquitectura mostrada en la Figura 7-2:

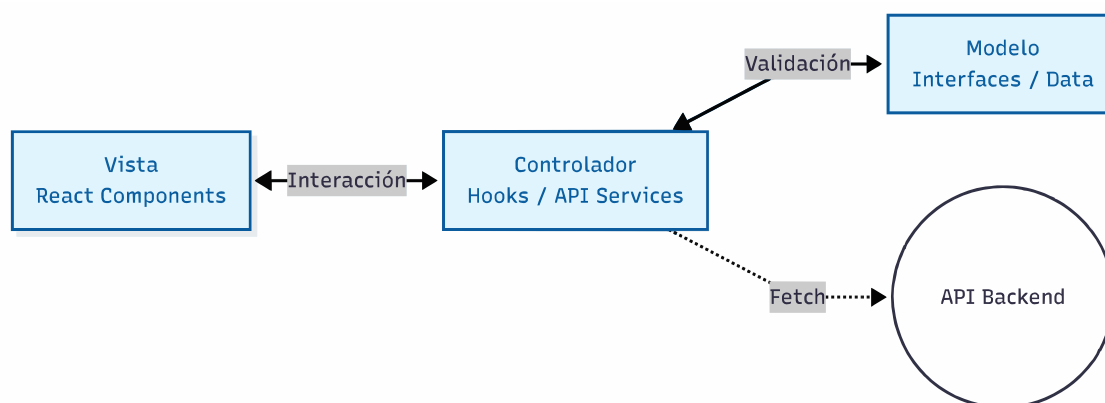
- **Capa de presentación:** es la encargada de gestionar las solicitudes del cliente. En ella se encuentra la lógica necesaria para recibir las peticiones entrantes y retornar las respuestas pertinentes.
- **Capa de servicios:** contiene la lógica de negocio del sistema. Procesa las solicitudes recibidas desde la capa de presentación y aplica las reglas de negocio y validaciones del dominio. Además, coordina el flujo de recomendación transformando los datos de la base de datos para enviarlos al modelo de recomendación y gestiona la respuesta recibida.
- **Capa de modelo de datos:** define las estructuras de información y entidades que rigen el sistema.
- **Capa de infraestructura:** se encarga de la comunicación con agentes externos y almacenamiento de datos. Gracias a esta capa se permite que el resto de la aplicación no dependa de la tecnología de la base de datos.

- **Capa core:** contiene las funcionalidades transversales que dan soporte a toda la arquitectura. Centraliza la configuración global y los mecanismos de protección del sistema.

Arquitectura del frontend

El frontend del sistema se ha desarrollado siguiendo el patrón Modelo-Vista-Controlador. Esta estructura permite separar las responsabilidades de la aplicación en distintos componentes, facilitando la organización del código, su mantenibilidad y la reutilización de elementos.

Figura 7-3 Arquitectura Frontend



A continuación, se describen los componentes principales de la arquitectura de la Figura 7-3:

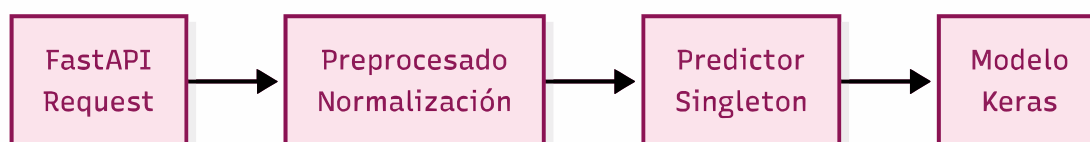
- **Modelo:** representa la estructura de los datos utilizados en la aplicación. Los modelos permiten definir la estructura de datos que maneja la aplicación, facilitando su reutilización y garantizando coherencia en el tratamiento de la información recibida del backend.
- **Vista:** es la encargada de componer las pantallas que el usuario visualiza. No gestiona la lógica completa, sino que la delega a la capa inferior.
- **Controlador:** actúa como intermediario entre los modelos y las vistas. Se encarga de realizar peticiones a la API, procesar las respuestas y actualizar los componentes correspondientes de la aplicación.

Además, la aplicación cuenta con un punto de entrada principal, donde se inicializa la estructura general de la interfaz.

Microservicio de IA

Dado el coste computacional y las dependencias específicas necesarias para el motor de recomendación, se ha implementado un microservicio independiente siguiendo un patrón de Servicio de Inferencia Ligero.

Figura 7-4 Arquitectura Microservicio



Los principales componentes de la Figura 7-4 son:

- **Interfaz de entrada:** expone un único punto de entrada que recibe los vectores de características y valida la identidad del emisor.
- **Gestor de modelo:** implementa un patrón Singleton para asegurar que el modelo neuronal se cargue en memoria una sola vez al iniciar el servicio, optimizando los tiempos de respuesta.
- **Capa de Preprocesamiento:** realiza las últimas transformaciones sobre los datos entrantes para asegurar que las entradas coincidan exactamente con la arquitectura de la red neuronal entrenada.

7.2. DISEÑO DE DETALLE

Tras definir la estructura general del sistema, en este apartado se describe la organización interna de los componentes, detallando las responsabilidades de cada módulo y el flujo de información que permite la ejecución del sistema de recomendación híbrido.

Detalle del backend

- **Capa de presentación:** gestiona el ciclo de vida de las peticiones HTTP. Su función principal es exponer los endpoints de la API REST, realizar la validación inicial de los datos de entrada y devolver las respuestas en formato JSON. Esta capa se estructura mediante un sistema de enrutadores segregados por dominio de negocio. En concreto, se exponen módulos de gestión de identidad y acceso, el punto de entrada del sistema de recomendación, y los controladores encargados de las interacciones del usuario.
- **Capa de servicios:** actúa como el núcleo del sistema y encapsula la lógica de negocio de la aplicación. Para garantizar la cohesión, esta capa se modulariza en servicios específicos de la misma manera que en la capa de presentación, tanto de gestión del comportamiento y de las preferencias del usuario como de validación y seguridad. Además, alberga el sistema de recomendación del proyecto. Este módulo actúa como un motor inteligente, encargándose de incorporar el contexto del usuario, preprocesar la información y ejecutar el algoritmo predictivo para devolver las sugerencias personalizadas.
- **Capa de modelo de datos:** garantiza que la información esté estructurada de forma coherente mediante dos subcomponentes principales:
 - **Entities:** contienen las representaciones de los objetos principales del sistema que corresponden al esquema de la base de datos. No solo modela a los actores iniciales, sino que también mapea de forma exhaustiva las métricas de comportamiento necesarias para el motor de recomendación.
 - **DTOs:** objetos optimizados para el intercambio de información entre capas. Su función es empaquetar, filtrar y transformar la información antes de enviarla a la capa de presentación, asegurando que el frontend solo reciba los datos necesarios y ocultando atributos sensibles de la base de datos. El proyecto implementa DTOs para cada módulo del sistema.
- **Capa de infraestructura:** actúa como puente de comunicación con todos los agentes y servicios externos al sistema. Al igual que en la capa de modelo de datos, se apoya en tres subcomponentes para aislar las dependencias externas del resto de la aplicación:
 - **Repositories:** clases encargadas de gestionar la persistencia y manipulación de datos en el sistema gestor de bases de datos. Implementan funciones de acceso específicas mapeadas a las necesidades de negocio. Entre ellas, hay clases sobre la gestión de usuarios, de restaurantes y de los registros de interacción de los usuarios.
 - **Servicios externos:** módulos dedicados exclusivamente a la comunicación mediante peticiones HTTP con plataformas de terceros.
 - **Firebase:** módulo dedicado a la comunicación con el servicio externo de identidad, que gestiona la validación de tokens y la seguridad de acceso,

delegando la infraestructura de autenticación de forma segura y transparente.

Detalle del frontend

- **Modelo:** define los tipos de datos y la lógica de comunicación con la API REST del backend. Esta capa se compone de un conjunto de servicios modulares encargados de centralizar y abstraer todas las peticiones del sistema. De este modo, la capa integra directamente los distintos servicios de la aplicación. Además, los modelos garantizan que la respuesta JSON se convierta en estructuras de datos coherentes para el cliente.
- **Vista:** capa responsable de definir la disposición de los elementos en pantalla que permiten mostrar la información y capturar las acciones del usuario. Consta de un conjunto de pantallas independientes que abarca todo el flujo del sistema, desde el acceso y el registro hasta las recomendaciones personalizadas.
- **Controlador:** gestiona el estado de la aplicación, reacciona a las acciones del usuario y coordina la lógica de interacción. El bloque integra componentes especializados para orquestar el ciclo de vida de la sesión del usuario y controlar los distintos procesos de la plataforma. Los controladores interceptan los eventos de las vistas, invocan los servicios del modelo y actualizan el estado global de forma reactiva.

Además, se ha implementado una estructura basada en componentes reutilizables. Este enfoque sigue el principio *Don't Repeat Yourself* [39], lo que permite que algunos elementos, como las tarjetas de información de restaurantes, sean independientes del contexto de la página. Esto no solo facilita la consistencia visual, sino que simplifica el mantenimiento y escalabilidad de la interfaz.

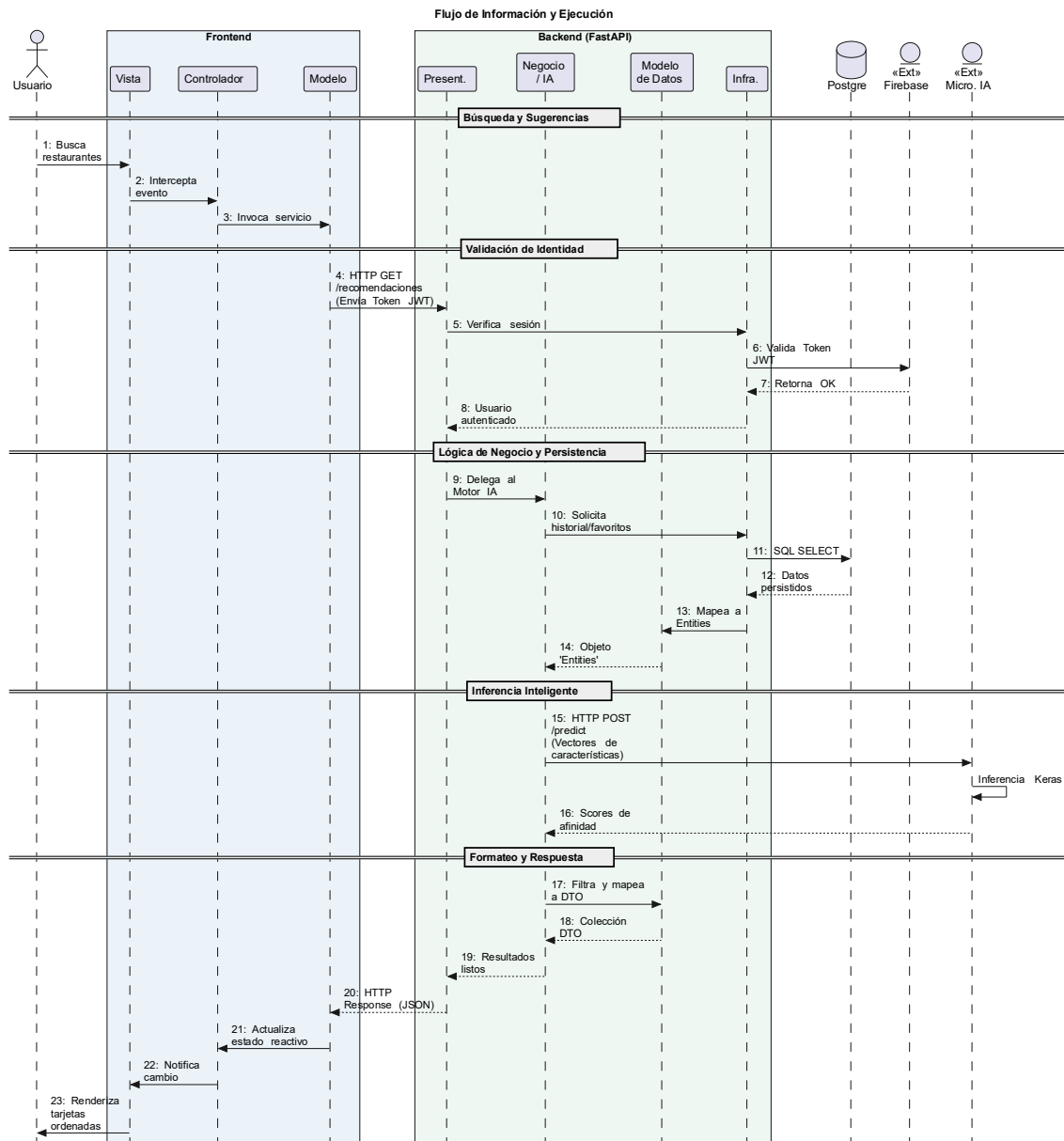
Flujo de información y ejecución

Para comprender la interacción y el acoplamiento entre los diferentes componentes descritos, el flujo de información del sistema puede ilustrarse mediante el escenario de generación de sugerencias, al mostrar los restaurantes de una búsqueda. El proceso descrito en la Figura 7-5 comienza en el frontend, donde la vista de los resultados de la búsqueda requiere los datos. Para ello, el controlador intercepta la petición del usuario e invoca al componente correspondiente del modelo. Este servicio realiza una llamada HTTP a la capa de presentación del backend, que valida la estructura y verifica la identidad del usuario, apoyándose en el módulo de Firebase de la capa de infraestructura.

Tras verificar la sesión, la petición se delega en la capa de servicios, donde el motor de recomendación orquesta la lógica de negocio invocando a los Repositories de la capa de infraestructura para extraer del usuario los datos almacenados en la base de datos, transformando dicha persistencia en un componente de Entities. Seguidamente, el servicio de recomendación se comunica con el microservicio que aloja el modelo (Keras). Finalmente, el resultado del modelo se moldea en la capa de datos mediante los DTOs y vuelve a viajar por la red hasta el frontend, donde el modelo del cliente actualiza el estado para que la vista muestre las tarjetas de los restaurantes.

A continuación, se muestra una representación del flujo mencionado en la Figura 7-5

Figura 7-5 Flujo de información



7.3. DISEÑO DE PRUEBAS

El diseño de pruebas constituye la fase sistemática en la que se analizan exhaustivamente los requisitos funcionales de la aplicación para obtener los escenarios de prueba. La norma específica, sigue las directrices de la norma ISO/IEC/IEEE 29119 [40] asegurando que cada componente responda a las expectativas de comportamiento.

Para garantizar una verificación exhaustiva y mitigar los riesgos en el software, el plan de pruebas de la aplicación se articula de forma incremental a través de diferentes niveles:

- Por un lado, se adopta un enfoque de verificación en paralelo para las pruebas unitarias y de integración, que se diseñan e implementan de inmediato tras la codificación de cada funcionalidad. Estas pruebas se encargan tanto de aislar y validar de forma temprana el correcto funcionamiento de las funciones, los

componentes y la lógica de negocio, como de verificar que los componentes validados interactúen entre sí correctamente. El propósito principal de este nivel es asegurar la calidad del código, garantizando que cada bloque lógico funcione exactamente como se diseñó y devuelva los resultados esperados tanto en flujos normales como ante casos límite o de error. Estas pruebas se implementarán junto con el desarrollo de cada una de las funcionalidades.

- Por otro lado, la evaluación global de la aplicación se realiza mediante un conjunto de pruebas de sistema centradas en verificar los flujos de trabajo de extremo a extremo. De esta forma, se valida la correspondencia exacta entre las entradas proporcionadas por el usuario en la interfaz y las salidas o respuestas esperadas del servidor. El diseño analítico y la especificación detallada de los escenarios de estas pruebas se abordan con mayor detalle a continuación.
- Finalmente, se incluye una fase de pruebas estáticas aplicada sobre el código fuente. A diferencia de los niveles anteriores, este proceso no requiere la ejecución del software, sino que se centra en el análisis automatizado del repositorio para asegurar el cumplimiento de las buenas prácticas de programación, verificar los estándares de estilo del stack tecnológico y detectar de forma proactiva vulnerabilidades, redundancias o código muerto.

El objetivo principal de este diseño es establecer una trazabilidad estricta que valide el correcto funcionamiento de las funcionalidades implementadas en el sistema. Debido a su complejidad, se decide descomponer en varios subcomponentes según las funcionalidades:

- Inicio de sesión (Tabla 7-1).
- Registro (Tabla 7-2).
- Búsqueda de restaurantes (Tabla 7-3).
- Valoración de un restaurante (Tabla 7-4).
- Consultar el historial (Tabla 7-5).
- Consultar listas de favoritos (Tabla 7-6)
- Interacción con el menú lateral (Tabla 7-7)
- Editar datos personales en el perfil de usuario (Tabla 7-8)

Inicio de sesión

Para diseñar las pruebas de inicio de sesión, se realiza un análisis de partición equivalente de las entradas del formulario. De este proceso se derivan las clases de equivalencia y elementos de cobertura de prueba que condicionan el comportamiento del sistema. Para los campos identificados, se establecen los siguientes elementos a cubrir:

- Google: decisión del usuario entre utilizar la autenticación de Google o completar el formulario de la aplicación. Tiene dos posibles valores:
 - Si.
 - No.
- Identificación: campo que solicita al usuario un identificador único para iniciar sesión en la aplicación. Sus posibles valores son:
 - Usar correo electrónico:
 - Correo existente.
 - Correo inexistente.
 - Usar el nombre de usuario:

- Nombre de usuario existente.
- Nombre de usuario inexistente.
- No introducir nada
- Contraseña: campo vinculado al identificador introducido. El usuario puede realizar las siguientes interacciones:
 - Contraseña correcta.
 - Contraseña incorrecta.
 - No introducir nada.

Debido a la cantidad de entradas, realizar una combinación total derivaría en un número de situaciones no abarcables, lo que no permitiría probar el proyecto en un tiempo y con una cantidad de recursos admisibles. Se procede a aplicar la técnica combinatoria de pruebas Base-Choice [41] para reducir el número de situaciones de prueba.

Tabla 7-1 Pruebas Inicio de sesión

	Google		Identificación					Contraseña		
	Si	No	Correo		Usuario		Nada	Si		No
			Existe	No existe	Existe	No existe		Bien	Mal	
BASE		X	X							X
S2	X		X							X
S3		X		X						X
S4		X			X					X
S5		X				X				X
S6		X					X			X
S7		X	X					X		
S8		X	X						X	

Como se puede observar en la Tabla 7-1, se ha seleccionado una base sin contraseña. A pesar de que no es la situación más probable, se ha tomado esta decisión para que la estrategia combinatoria no derivase en varias situaciones no realizables. Además, permite probar flujos importantes como la autenticación con Google (S2, Tabla 7-1) o el inicio de sesión con todos los campos correctos (S7, Tabla 7-1).

Registro

Para las pruebas de registro en la aplicación, se analizan las restricciones del formulario aplicando las mismas técnicas que en el inicio de sesión. A partir de los criterios de validación, se definen los elementos de cobertura obligatorios para la certificación del componente:

- Correo: campo de texto para introducir el correo electrónico del usuario. Puede tomar cuatro valores diferentes:
 - Correo válido.
 - Correo no válido.
 - No cumple con el patrón de REGEX de un correo electrónico.
 - Correo electrónico ya en uso.
 - No se introduce nada.

- Nombre de usuario: campo para introducir una cadena única que identificará al usuario en la aplicación. Existen tres posibles situaciones:
 - Nombre de usuario válido.
 - Nombre de usuario en uso.
 - No se introduce nada.
- Nombre: campo de texto en el que el usuario deberá introducir su nombre. Puede tomar dos valores:
 - Se introduce un nombre.
 - No se introduce nada.
- Apellidos: campo de texto análogo al anterior, pero para los apellidos del usuario. Puede tomar dos valores:
 - Se introducen apellidos.
 - No se introduce nada.
- Contraseña: campo de texto para crear una contraseña que se use para identificarse en la aplicación. Este cuenta con una restricción de 8 caracteres como mínimo y la necesidad de introducir al menos un número y una letra. Por tanto, es necesario tener en cuenta los valores límite y analizar cada una de las opciones posibles. Por tanto, los diferentes valores posibles son:
 - Longitud de la contraseña:
 - 0 (un valor menor que 8)
 - 7 (valor límite)
 - 8 (valor límite)
 - 16 (un valor superior a 8)
 - Presencia de números:
 - Si.
 - No.
 - Presencia de letras:
 - Sí.
 - No.
- Ubicación: campo en el que el usuario debe introducir su ubicación preferida para futuras recomendaciones. Tiene tres valores posibles:
 - Ubicación válida.
 - Ubicación no válida.
 - No se introduce nada.

Debido a la gran cantidad de combinaciones posibles entre los distintos campos, al igual que en el caso anterior, se decide aplicar Base-Choice para reducir el número de situaciones de prueba.

Tabla 7-2 Pruebas Registro

	Correo				Nombre de usuario			Nombre		Apellidos		Contraseña								Ubicación		
	Válido	No válido			Si	No	En uso	Si	No	Si	No	Longitud				Números		Letra		Si		No
		No REGEX	En uso	Nada								0	7	8	16	Si	No	Si	No	Valida	No válida	
BASE	X				X			X		X				X		X		X		X		
S2		X			X			X		X				X		X		X		X		
S3			X		X			X		X				X		X		X		X		
S4				X	X			X		X				X		X		X		X		
S5	X					X		X		X				X		X		X		X		
S6	X						X	X		X				X		X		X		X		
S7	X				X				X	X				X		X		X		X		
S8	X				X			X			X			X		X		X		X		
S9	X				X			X		X		X				X		X		X		
S10	X				X			X		X			X			X		X		X		
S11	X				X			X		X					X	X		X		X		
S12	X				X			X		X				X			X	X		X		
S13	X				X			X		X						X			X	X		
S14	X				X			X		X						X		X			X	
S15	X				X			X		X						X		X				X

A diferencia del caso anterior, como se observa en la Tabla 7-2, se ha seleccionado como caso base un estado en el que todos los campos cumplen con los requisitos de formato y validación. Con estas combinaciones se puede comprobar si la aplicación detecta correctamente los errores de formato en cada campo del formulario.

Buscar restaurante

Para el motor de búsqueda y recomendación, las clases de equivalencia se extraen de los parámetros lógicos que el usuario puede parametrizar. El conjunto de elementos de cobertura diseñado para evaluar la flexibilidad del algoritmo comprende:

- Etiquetas: permite al usuario seleccionar una serie de etiquetas indicando el tipo de restaurante que quiere buscar. Pueden darse las siguientes situaciones:
 - Cero etiquetas.
 - Una etiqueta.
 - Varias etiquetas.
- Precio: el usuario puede seleccionar un rango de precio entre cuatro opciones disponibles. Se pueden dar las mismas situaciones que en el caso anterior:
 - Cero precios.
 - Un precio.
 - Varios precios.
- Sin precio: casilla que el usuario puede marcar para incluir sitios sin precio confirmado. Puede tomar dos valores:
 - Marcar la casilla.
 - No marcar la casilla.
- Abierto: casilla que el usuario puede marcar para incluir solo sitios abiertos en el momento de la búsqueda. Puede tomar dos valores:
 - Marcar la casilla.
 - No marcar la casilla.
- Ubicación: campo para seleccionar la ubicación en la que realizar la búsqueda. Las posibles entradas son:
 - Usar ubicación preferida.
 - Introducir ubicación:
 - Ubicación válida.
 - Ubicación no válida.
- Resultado: lista de restaurantes recomendados al usuario tras la búsqueda realizada. Se pueden dar las siguientes situaciones:
 - 0 restaurantes.
 - 1 restaurante.
 - Varios restaurantes.

Tabla 7-3 Pruebas Buscar restaurante

	Etiquetas			Precio			Sin precio		Abierto		Ubicación			Resultado		
	0	1	Varias	0	1	Varios	Si	No	Si	No	Preferida	Otra		0	1	Varios
												Válida	No Válida			
BASE			X			X	X		X		X					X
S2	X					X	X		X		X					X
S3		X				X	X		X		X					X
S4			X	X			X		X		X					X
S5			X		X		X		X		X					X
S6			X			X		X	X		X					X
S7			X			X	X			X	X					X
S8			X			X	X		X			X				X
S9			X			X	X		X				X			X
S10			X			X	X		X		X			X		
S11			X			X	X		X		X				X	

Para la búsqueda de restaurantes, como se puede ver en la Tabla 7-3, se ha seleccionado como caso base la combinación de múltiples criterios simultáneos, lo que garantiza que el usuario obtenga resultados favorables a pesar de la complejidad de la consulta. Además, se prueban diferentes combinaciones de búsquedas para comprobar el correcto funcionamiento de la aplicación a pesar de la omisión de parámetros. También cabe destacar la S9, Tabla 7-3, en la que se comprueba la respuesta del sistema ante una ubicación inválida. Por último, las situaciones finales comprueban el correcto funcionamiento de la aplicación ante diferentes cantidades de resultados.

Valoraciones

Esta interfaz se compone de varios campos para que el usuario dé su opinión acerca de un establecimiento:

La interfaz de reseñas emplea escalas de tipo Linkert (1 a 5 estrellas), permitiendo aplicar un análisis de valores límite en los extremos del intervalo. Los elementos de cobertura seleccionados para verificar la integridad de las métricas son:

- Calidad: campo para indicar la calidad del restaurante valorado. Pueden darse varias situaciones:
 - Cero estrellas.
 - Una estrella.
 - Tres estrellas.
 - Cinco estrellas.
- Precio: campo para indicar la conformidad con el precio del restaurante valorado. Pueden darse varias situaciones:
 - Cero estrellas.
 - Una estrella.
 - Tres estrellas.

- Cinco estrellas.
- Higiene: campo para indicar la higiene del restaurante valorado. Pueden darse varias situaciones:
 - Cero estrellas.
 - Una estrella.
 - Tres estrellas.
 - Cinco estrellas.
- Trato: campo para indicar el trato recibido en el restaurante valorado. Pueden darse varias situaciones:
 - Cero estrellas.
 - Una estrella.
 - Tres estrellas.
 - Cinco estrellas.
- Comentario: campo de texto que permite al usuario introducir un comentario adicional sobre el restaurante. Pueden darse dos situaciones:
 - Campo rellenado.
 - Campo vacío.

Tabla 7-4 Pruebas Valoraciones

	Calidad				Precio				Higiene				Trato				Comentario	
	0	1	3	5	0	1	3	5	0	1	3	5	0	1	3	5	Si	No
BASE				X				X				X				X	X	
S2	X							X				X				X	X	
S3		X						X				X				X	X	
S4			X					X				X				X	X	
S5				X	X							X				X	X	
S6				X		X						X				X	X	
S7				X			X					X				X	X	
S8				X				X	X							X	X	
S9				X				X		X						X	X	
S10				X				X			X					X	X	
S11				X				X				X	X				X	
S12				X				X				X		X			X	
S13				X				X				X			X		X	
S14				X				X				X				X		X

Como se detalla en la Tabla 7-4, se ha tomado como base la situación en la que todos los campos han sido completados. El resto de los escenarios comprueba que la aplicación responde correctamente a los distintos valores de cada una de las métricas consideradas en la valoración.

Historial

En la vista del historial, el usuario puede ver los restaurantes que ha visitado anteriormente. En ella se pueden observar diferentes apartados:

- Meses: agrupamiento de restaurantes por meses. Puede haber diferentes cantidades:
 - 0 grupos de meses.
 - 1 grupo de meses.
 - Varios grupos de meses.
- Restaurantes: tarjetas de restaurantes dentro de cada agrupación mensual. Puede haber diferentes cantidades:
 - 0 restaurantes.
 - 1 restaurante
 - Varios restaurantes.
- Búsqueda: campo de texto para filtrar los restaurantes que el usuario puede usar. Se pueden dar dos situaciones:
 - Se aplica el filtro de búsqueda.
 - No se aplica el filtro de búsqueda.

Tabla 7-5 Pruebas Historial

	Meses			Restaurantes			Búsqueda	
	0	1	Varios	0	1	Varios	Si	No
BASE			X			X		X
S2*	X					X		X
S3		X				X		X
S4*			X	X				X
S5			X		X			X
S6			X			X	X	

En este caso, como se puede observar en la Tabla 7-5, se ha seleccionado como caso base la situación más probable entre los usuarios, en la cual han visitado varios restaurantes a lo largo de varios meses. A partir de esta base, el resto de las situaciones de prueba permiten verificar la adaptabilidad de la interfaz ante diferentes volúmenes de información. Además, en el Caso 6 se evalúa el correcto funcionamiento de la barra de búsqueda para filtrar los establecimientos deseados.

Cabe destacar que la Situación 2 es una situación irrealizable, ya que, si no existen grupos mensuales, no puede haber restaurantes en el historial. Ocurre lo mismo en la Situación 4, ya que no se crea una agrupación mensual si no hay restaurantes almacenados en ella.

Favoritos

En esta interfaz, el usuario puede acceder a los restaurantes que tiene guardados como favoritos. Consta de los siguientes apartados:

- Listas: los usuarios almacenan sus restaurantes favoritos en listas. Se pueden dar las siguientes situaciones:
 - 0 listas.
 - 1 lista.
 - Varias listas.
- Restaurantes: tarjetas de restaurantes dentro de cada lista de favoritos. Puede haber diferentes cantidades:

- 0 restaurantes.
- 1 restaurante.
- Varios restaurantes.
- Búsqueda: campo de texto para filtrar los restaurantes que el usuario puede usar. Se pueden dar dos situaciones:
 - Se aplica el filtro de búsqueda.
 - No se aplica el filtro de búsqueda.

Tabla 7-6 Pruebas Favoritos

	Listas			Restaurantes			Búsqueda	
	0	1	Varias	0	1	Varios	Si	No
BASE			X	X			X	
S2	X			X			X	
S3		X		X			X	
S4			X		X		X	
S5			X			X	X	
S6			X	X				X

Al contrario que en la interfaz anterior, en la Tabla 7-6 se selecciona como caso base una situación en la que no existen restaurantes almacenados. De esta manera, se evitan situaciones imposibles en las que hay restaurantes guardados, pero no existen listas. Por otro lado, al no haber restaurantes en el caso base, es necesario incluir el filtrado por búsqueda en el mismo, ya que si no el Caso 6 sería el único que probase dicho filtro y no tendría ningún efecto debido a la falta de establecimientos.

Menú lateral

El menú lateral presente en la mayoría de vistas tiene un apartado de configuración con una serie de opciones:

- Tema: permite al usuario elegir el tema de la aplicación. Tiene dos posibles valores:
 - Claro.
 - Oscuro.
- Letra: permite al usuario elegir el tamaño de la letra en la aplicación. Tiene tres posibles valores:
 - Pequeña (S).
 - Mediana (M).
 - Grande (L).

Tabla 7-7 Pruebas Menú lateral

	Tema		Letra		
	Claro	Oscuro	S	M	L
BASE	X			X	
S2		X		X	
S3	X		X		
S4	X				X

En el caso del menú lateral, como se puede observar en la Tabla 7-7, se prueban diferentes combinaciones tanto del tema como del tamaño de letra para garantizar una buena experiencia de usuario en la personalización de la aplicación, seleccionando como caso base un tema claro con una letra de tamaño medio.

Perfil de usuario

Desde la vista del perfil, el usuario dispone de una serie de campos para actualizar la información introducida al registrarse en la aplicación. Cada uno de ellos representa una entrada diferente que puede condicionar el resultado de las pruebas:

- Nombre: permite al usuario cambiar su nombre en la aplicación. Se pueden dar dos situaciones:
 - Si cambia el nombre
 - No cambia el nombre
- Apellidos: permite al usuario cambiar sus apellidos en la aplicación. Se pueden dar dos situaciones:
 - Si cambia los apellidos
 - No cambia los apellidos
- Ubicación: permite al usuario cambiar su ubicación preferida en la aplicación. Se pueden dar dos situaciones:
 - Si cambia la ubicación
 - No cambia la ubicación
- Correo electrónico: permite al usuario cambiar el correo electrónico vinculado a su cuenta. Se pueden dar las siguientes situaciones:
 - Se introduce un correo electrónico (mismas comprobaciones que en el registro)
 - Se introduce una contraseña
 - Contraseña correcta
 - Contraseña incorrecta
 - No se introduce contraseña
 - No se introduce un correo electrónico
 - Se introduce una contraseña
 - Contraseña correcta
 - Contraseña incorrecta
 - No se introduce contraseña
- Eliminar cuenta: permite al usuario eliminar su cuenta de forma permanente del sistema. Se pueden dar dos situaciones:
 - El usuario elimina su cuenta
 - El usuario no elimina su cuenta

Tabla 7-8 Pruebas Perfil de usuario

	Nombre		Apellidos		Ubicación			Correo Electrónico							Contraseña										Eliminar cuenta	
	Si	No	Si	No	Si		No	Si				Contraseña			Contraseña antigua			Si							Si	No
					Bien	Mal		Válido	No válido			Si		Vacía	Si			Contraseña nueva								
									No REGEX	En uso	Nada	Bien	Mal		No	Longitud				Números		Letra				
																0	7	8	16	Si	No	Si	No			
BASE		X		X		X		X				X			X					X		X				X
S2	X			X		X		X				X			X					X		X				X
S3		X	X			X		X				X			X					X		X				X
S4		X		X	X		X					X			X					X		X				X
S5		X		X		X			X			X			X					X		X				X
S6		X		X		X				X		X			X					X		X				X
S7		X		X		X				X		X			X					X		X				X
S8		X		X		X		X					X		X					X		X				X
S9		X		X		X		X						X	X					X		X				X
S10		X		X		X		X				X				X				X		X				X
S11		X		X		X		X				X					X			X		X				X
S12		X		X		X		X				X			X			X				X				X
S13		X		X		X		X				X			X			X				X				X
S14		X		X		X		X				X			X				X			X				X
S15		X		X		X		X				X			X					X			X			X
S16		X		X		X		X				X			X					X		X			X	X
S17		X		X		X		X				X			X					X		X		X		X

Como se puede observar en la Tabla 7-8 se ha escogido como situación base un caso en el que el usuario quiera cambiar el correo y la contraseña y deba hacer uso de su contraseña actual para efectuar el cambio. A partir de ella, se varía la interacción del usuario con su perfil, comprobando que todos los campos pueden ser actualizados sin problema y que detecten errores en el formato, evitando datos incorrectos.

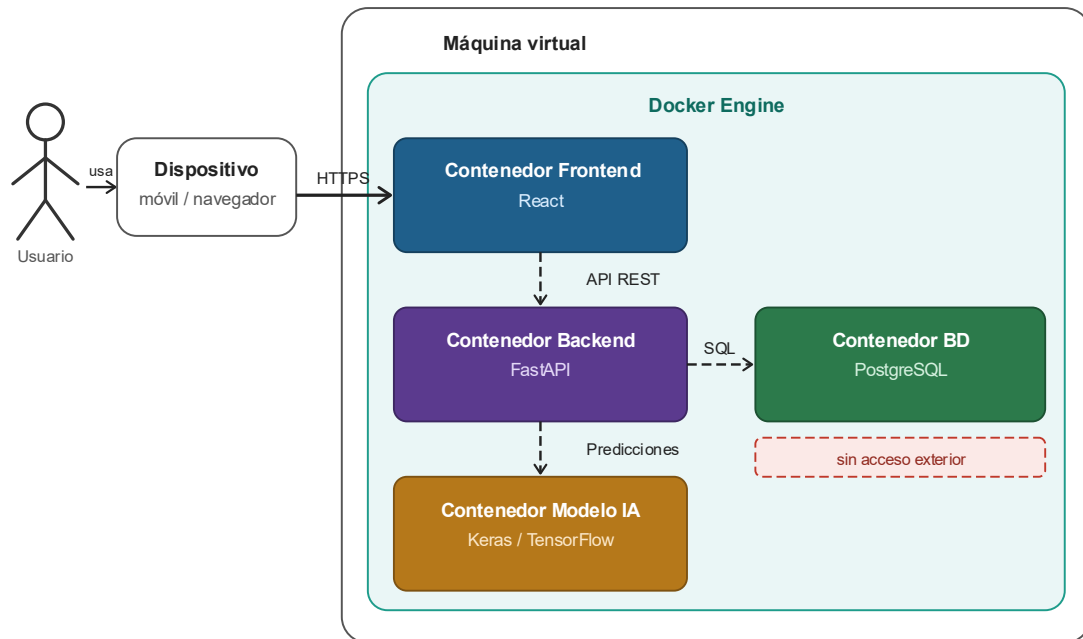
7.4. DESPLIEGUE

Para la distribución del sistema, se ha optado por un modelo basado en contenedores independientes. Esta decisión permite empaquetar cada componente con sus propias dependencias, aislando la ejecución y garantizando que el sistema funcione exactamente igual tanto en el entorno de desarrollo local como en el de producción.

- **Contenedor Frontend:** alberga la interfaz gráfica de usuario. Empaqueta el servidor web ligero, encargado de distribuir los archivos a los navegadores o dispositivos de los usuarios, y aísla la capa visual del resto del sistema.
- **Contenedor Backend:** se ejecuta únicamente el servidor principal. Este contenedor expone un puerto público hacia el exterior para recibir las solicitudes del contenedor frontend y, a su vez, se conecta con los contenedores de datos y del modelo de recomendación para procesar la lógica de negocio.
- **Contenedor del Modelo de recomendación:** encapsula el entorno de ejecución del modelo y sus dependencias. Este servicio expone una API REST interna para recibir los vectores de características desde el backend y asegura un espacio de memoria dedicado a mantener la red neuronal, evitando que la carga computacional de las predicciones interfiera en el rendimiento del servidor o de la persistencia de datos.
- **Contenedor de la Base de datos:** aloja el motor y los volúmenes de almacenamiento en disco necesarios para asegurar que los datos de los restaurantes y usuarios no se pierdan. A diferencia del módulo anterior, este contenedor no opera con la arquitectura REST, sino que se comunica mediante el protocolo nativo de PostgreSQL. Este contenedor está completamente cerrado al tráfico externo y solo escucha las peticiones de lectura y escritura que le llegan desde el contenedor backend, lo que garantiza un entorno seguro y de alta velocidad.

La infraestructura de contenedores comentada anteriormente y sus respectivas conexiones se encuentran representadas gráficamente en el diagrama de la Figura 7-6.

Figura 7-6 Diagrama despliegue



Cuando un usuario accede a la aplicación, la petición llega al contenedor frontend. El usuario navega por la aplicación y solicita distintos datos. Esta solicitud se envía como petición al contenedor backend. El servidor intercepta la petición y la verifica. En caso de ser válida, el backend se conecta, dentro de la red interna, al servidor de base de datos mediante consultas SQL para obtener la información solicitada en la petición. De forma simultánea, en caso de requerir una recomendación personalizada, el backend envía los datos del usuario al contenedor del modelo de recomendación, que ejecuta la red neuronal para generar sugerencias.

Una vez que el contenedor backend recibe tanto los datos de la base de datos como las predicciones del modelo, combina ambas fuentes para generar una respuesta completa. Esta viaja de vuelta al frontend, que finalmente se la presenta al usuario a través de las interfaces de la aplicación.

Capítulo 8 IMPLEMENTACIÓN

8.1. ESTRUCTURA DE LA APLICACIÓN

Una vez definido el diseño conceptual del sistema, en este apartado se detalla cómo dicha arquitectura se traslada a la implementación.

Para garantizar la claridad de la memoria y evitar incluir bloques de código o detalles excesivos, este apartado se centrará en las decisiones arquitectónicas y flujos principales. El detalle de la implementación está disponible en el portal de documentación técnica alojado en GitHub Pages ¹.

A continuación, se detalla la organización interna de cada uno de los componentes.

```
/
├── backend/
│   ├── alembic/
│   │   └── versions/
│   ├── app/
│   │   ├── core/
│   │   ├── data/
│   │   ├── infrastructure/
│   │   │   ├── firebase/
│   │   │   ├── repositories/
│   │   │   ├── database.py
│   │   │   ├── google_places_client.py
│   │   │   └── keras_api_client.py
│   │   ├── models/
│   │   │   ├── dtos/
│   │   │   └── entities/
│   │   ├── presentation/
│   │   │   └── routers/
│   │   ├── services/
│   │   └── main.py
│   └── tests/
├── keras-api/
│   ├── models/
│   ├── training/
│   └── prediction/
├── frontend/
│   ├── src/
│   ├── android/
│   ├── e2e/
│   └── fastlane/
```

8.1.1. Implementación del Backend

Se organiza en una estructura por carpetas que refleja su arquitectura de capas:

- **/presentation (Capa de presentación):** alberga los routers encargados de exponer los endpoints públicos de la API REST. Su responsabilidad se centra en interceptar las peticiones HTTP entrantes, aplicar los mecanismos de validación sintáctica de datos en los puntos de entrada y delegar el flujo de ejecución al núcleo del servidor.

¹<https://github.com/TorreAlejandro/EPI-DevorApp>

- **/services (Capa de servicios):** constituye el núcleo del backend. Centraliza la implementación de toda la lógica de negocio y las reglas del dominio de la aplicación. En este directorio se lleva a cabo el procesamiento de la información de los establecimientos, la gestión de las interacciones y la coordinación con los agentes externos, actuando como el nexo que conecta la presentación con la infraestructura.
- **/models (Capa de modelo de datos):** contiene las definiciones estructurales del sistema. Este módulo se encarga de modelar tanto las entidades de persistencia mapeadas para la base de datos relacional usando SQLAlchemy, garantizando integridad de los registros mediante tablas y relaciones, como los DTOs que declaran el formato del intercambio de información.
- **/infraestructura (Capa de infraestructura):** implementa el código encargado de interactuar con agentes externos y herramientas de terceros que escapan al control directo de la lógica de negocio. Alberga el cliente de conexión y sesión de la base de datos, el cliente para el uso de la API de Google Places, el módulo de integración del SDK de Firebase y el cliente especializado con el microservicio de recomendación.
- **/core:** actúa como bloque de soporte del servidor. Almacena la configuración global de la aplicación, la carga segura de variables de entorno, la inicialización de servicios y los middlewares de seguridad generales que afectan a todos los niveles de la arquitectura.

8.1.2. Implementación del Frontend

La aplicación cliente, construida en React, sigue la estructura descrita a continuación:

- **/pages (Vista):** contiene los componentes que representan las vistas o pantallas completas de la aplicación. Actúan como los contenedores principales encargados de definir la disposición de la interfaz.
- **/components:** contiene elementos independientes de la interfaz. Estos componentes se aíslan del contexto de una página concreta, permitiendo su reutilización en múltiples vistas, garantizando así la consistencia estética.
- **/hooks (Controlador):** implementa la lógica de estado mediante Hooks. Estos componentes interceptan las acciones del usuario en las vistas, encapsulan el comportamiento de la interfaz, gestionan el estado reactivo y coordinan las llamadas asíncronas al modelo.
- **/services (Modelo):** actúa como la capa de abstracción encargada de centralizar la lógica de comunicación con el exterior. Contiene los módulos que realizan las peticiones HTTP directas hacia la API REST del backend, gestionando las promesas de red, el ciclo de vida de las solicitudes y los mecanismos de optimización mediante almacenamiento en la caché del cliente.

8.1.3. Implementación del Microservicio de IA

El motor de recomendación se ha implementado como un servicio independiente y ligero con la siguiente estructura:

- **main (Punto de entrada):** archivo encargado de inicializar el servicio, cargar la configuración del entorno y exponer los endpoints específicos que utiliza el backend para solicitar predicciones.
- **predictor (Capa de preprocesamiento y Gestor del modelo):** implementa la lógica de carga del modelo. Para optimizar el rendimiento y uso de recursos, este módulo implementa un patrón Singleton, asegurando que la red neuronal se cargue en memoria una única vez durante el arranque, evitando una reducción de rendimiento por volver a instanciar el modelo en cada petición.
- **models:** almacena los archivos resultantes de la fase de entrenamiento previa, listos para ser utilizados.

Funcionamiento del modelo

El modelo de recomendación se carga en memoria y permanece disponible durante toda la ejecución del servicio para atender las peticiones realizadas por el backend. Cuando un usuario solicita recomendaciones, el backend recopila la información necesaria del usuario y de los restaurantes candidatos, enviándola al microservicio mediante una petición HTTP.

El modelo recibe diferentes características relacionadas con el usuario, el restaurante y el contexto temporal. A partir de esta información, la red neuronal genera una estimación de la valoración que dicho usuario otorgaría a cada restaurante.

La arquitectura implementada sigue el enfoque Two-Tower, en el que la información del usuario y del restaurante se procesa de forma independiente mediante dos ramas de la red neuronal. Este enfoque permite representar ambos elementos mediante vectores de características, facilitando la identificación de relaciones complejas entre las preferencias del usuario y las propiedades de cada establecimiento. Posteriormente, ambas representaciones se fusionan para obtener una predicción final, representada mediante un valor normalizado entre 0 y 1. Finalmente, dicho valor se transforma a la escala de puntuaciones utilizada por la aplicación, comprendida entre 1 y 5 estrellas.

Aunque la predicción generada por la red neuronal es uno de los principales indicadores del sistema, esta no se utiliza de forma aislada. Se combinan otros factores como la valoración global del restaurante, la similitud con usuarios de gustos afines, las preferencias gastronómicas del usuario en función de sus restaurantes favoritos y la distancia al establecimiento. De este modo, se obtiene una puntuación final que permite ofrecer recomendaciones más precisas y adaptadas al contexto de cada usuario.

8.1.4. Diagrama de clases

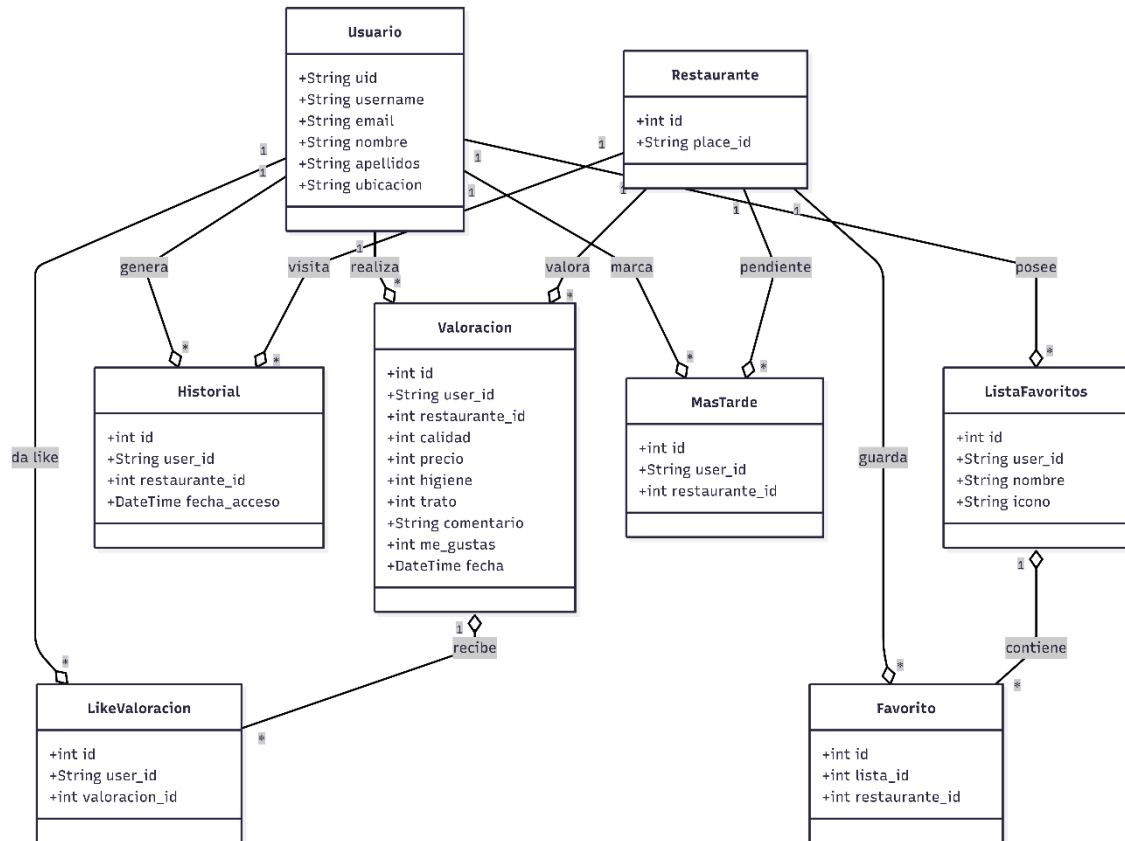
A continuación, se presenta el diagrama de clases que detalla la estructura, atributos y dependencias de los módulos del sistema, dividido en los diferentes componentes mencionados anteriormente. El detalle de la documentación se puede consultar en GitHub Pages²

² <https://github.com/TorreAlejandro/EPI-DevorApp>

Backend

En la Figura 8-1 se pueden observar tres aspectos fundamentales de la implementación del backend:

Figura 8-1 Diagrama de clases backend

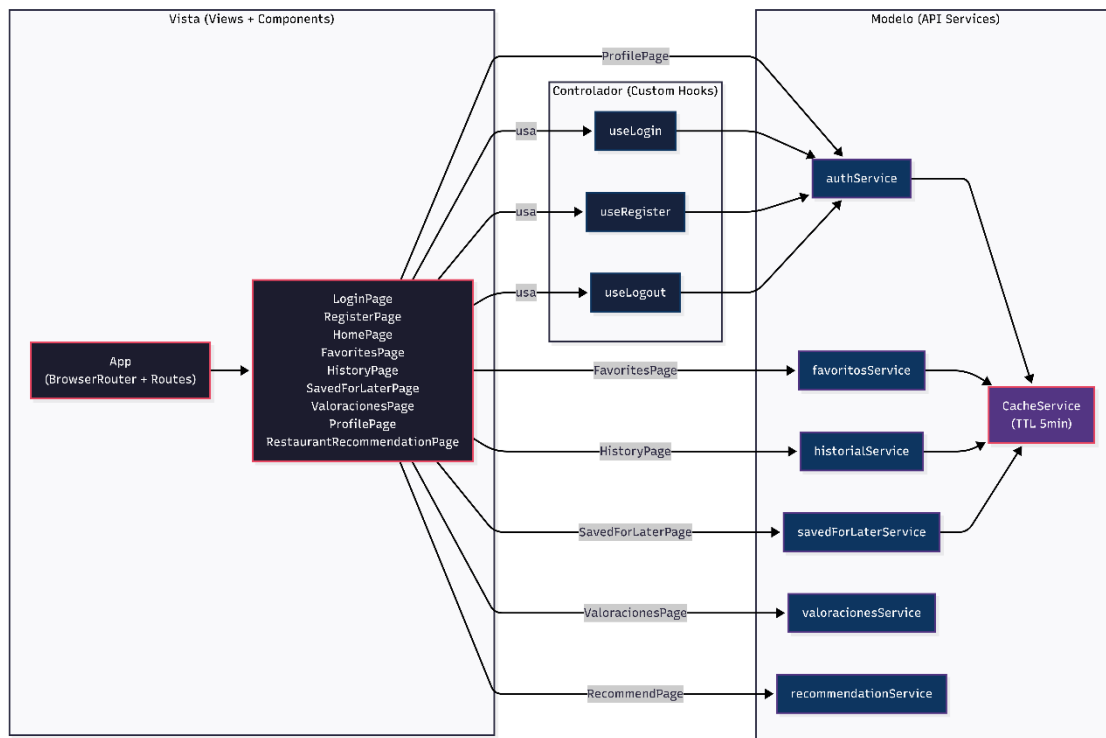


- **Mapeo Objeto-Relacional:** las clases situadas en el bloque izquierdo del diagrama se implementan usando SLQAlchemy. Cada atributo especificado se traduce en una columna con restricciones en la base de datos.
- **Gestión de identidad sincronizada:** la clase *Usuario* no almacena contraseñas. En su lugar, implementa el atributo *uid*, que se sincroniza con el identificador de Firebase y permite asociar cualquier petición autenticada con los registros locales de PostgreSQL.
- **Clase restaurante:** la clase *Restaurante* está diseñada para actuar como puente entre la aplicación y los servicios externos. Cuenta con dos atributos, un *id* como clave primaria local y un *place_id* que almacena el identificador único de Google Places API. De esta manera, se agilizan todas las interacciones del sistema usando el *id* local, y reservando el uso de *place_id* cuando sea necesario consultar datos externos.

Frontend

En la Figura 8-2 se pueden observar varios aspectos fundamentales de la implementación del frontend:

Figura 8-2 Diagrama de clases frontend

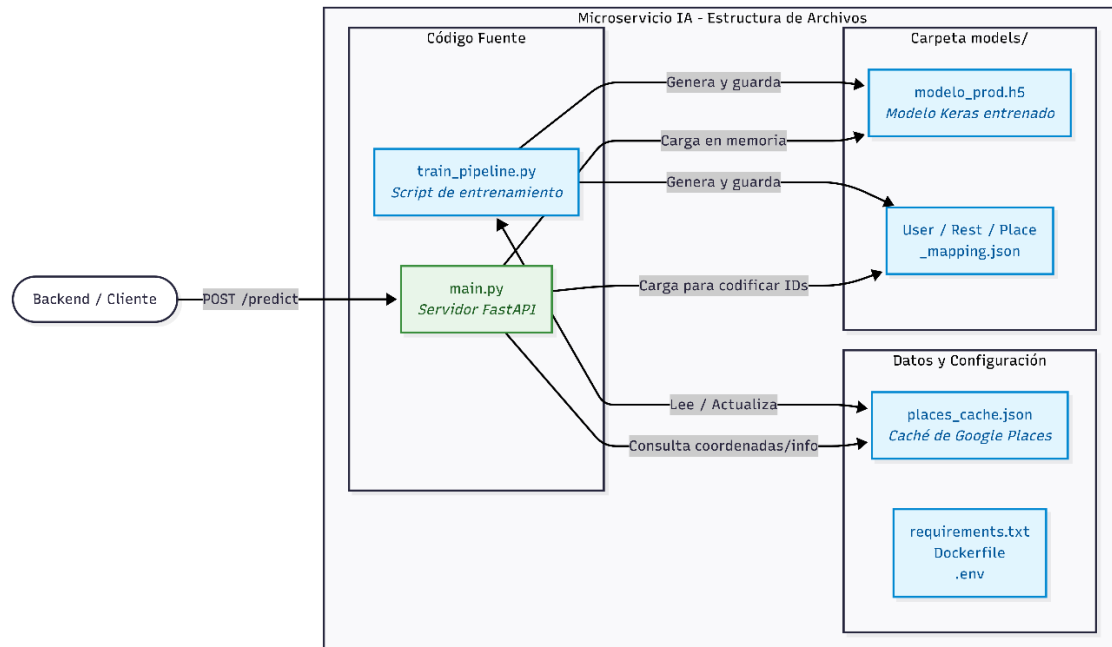


- **Enrutamiento centralizado:** la aplicación cuenta con un punto de entrada principal denominado *App*, el cual encapsula los componentes de navegación *BrowserRouter* y *Routes*. Se encarga del enrutamiento del lado del cliente, interceptando las URLs de la aplicación y renderizando de manera dinámica el conjunto de interfaces.
- **Uso de hooks:** la lógica de control de las vistas no se acopla directamente a los servicios de red, sino que el sistema implementa una capa intermedia de controladores. Las pantallas de acceso consumen de manera obligatoria los *hooks* especializados. Estos controladores aíslan los ciclos de vida de los formularios del frontend, capturan los eventos de carga o error y coordinan las llamadas lógicas.
- **Uso de almacenamiento local:** para mitigar el número de peticiones duplicadas hacia el servidor, reducir el consumo de red y mejorar la fluidez de la aplicación, se incorpora un *CacheService*. Los servicios que manejan datos de consulta frecuente desvían sus lecturas a través de este gestor de caché antes de lanzar una petición HTTP. El *CacheService* almacena las respuestas en memoria local aplicando una política de tiempo de vida de los datos. Si el usuario vuelve a acceder alguna de las funcionalidades como el historial o favoritos, la interfaz se renderiza instantáneamente desde la caché local.

Microservicio de IA

En la Figura 8-3 se pueden observar dos aspectos fundamentales de la implementación del microservicio de IA:

Figura 8-3 Diagrama de clases microservicio de IA



- **Punto de entrada:** el núcleo operativo del microservicio está controlado por `main.py`. Este script expone un endpoint mediante una petición `POST /predict`, que actúa como interfaz de comunicación con el backend. Su función principal es recibir los datos de la solicitud y retornar las predicciones.
- **Ciclo de vida del modelo:** el ciclo de entrenamiento y la lógica de Deep Learning se encapsulan en el script `train_pipeline.py`. Este componente tiene dos responsabilidades en el ciclo de desarrollo. Por un lado, tiene una fase de entrenamiento en la que procesa el conjunto de datos inicial, construye la red neuronal, entrena el modelo y genera los artefactos binarios de salida. Por otro lado, al iniciar el servidor, tanto `main.py` como la lógica de inferencia se apoyan en él para realizar la carga en memoria del modelo entrenado.

8.2. IMPLEMENTACIÓN DE LAS PRUEBAS

Una vez finalizada la implementación de la aplicación, es necesario realizar un proceso de pruebas para verificar su correcto funcionamiento. En este apartado se detallan las pruebas realizadas sobre la aplicación, junto con la metodología empleada y los resultados obtenidos.

8.2.1. Pruebas unitarias

Todas las suites de pruebas unitarias coexisten en el mismo repositorio del proyecto y se organizan en directorios según el ecosistema tecnológico de cada componente.

Estas pruebas se implementaron y ejecutaron en paralelo al desarrollo de la aplicación. A medida que se implementaban los diferentes módulos, controladores y componentes, se diseñaban e implementaban de inmediato sus respectivas suites de pruebas.

Frontend

Las pruebas se encuentran en el directorio *frontend/src/tests*. La estructura separa las pruebas de los elementos visuales de la lógica de negocio y consultas de red.

- **Tecnologías utilizadas:** se utiliza *Vitest* para ejecutar las pruebas y un entorno de emulación optimizado para *Vite*. La renderización de componentes y la simulación de eventos de usuario se gestionan mediante *React Testing Library*.
- **Nomenclatura y casos de prueba:** los archivos han sido nombrados en el formato *<NombreComponente>.test.tsx* o *<NombreServicio>.test.tsx*. Sigue un diseño estructurado en bloques *describe* e *it*. Los casos se describen en lenguaje natural especificando la acción que realizan con el prefijo “debe”.

Backend

Las pruebas se sitúan en el directorio independiente *backend/tests*, estructurando de manera simétrica al código fuente del backend. Cuenta con paquetes específicos para la lógica de negocio y los controladores de endpoints.

- **Tecnologías utilizadas:** el motor de ejecución principal es *Pytest*, complementado con la extensión *Pytest-asyncio* para ofrecer soporte nativo para funciones asíncronas. Las peticiones simuladas a la API se inyectan mediante el cliente *HTTPX* asíncrono. Finalmente, el aislamiento de dependencias externas se realiza mediante la librería *unittest.mock*.
- **Nomenclatura y casos de prueba:** los archivos se indexan en el formato *test_<modulo>_router.py* o *test_<modulo>_service.py*. Los casos de prueba se implementan como funciones globales en inglés, con guiones bajos, y detallan el escenario y el resultado esperado.

Microservicio de IA

Debido a la naturaleza compacta y acotada del microservicio, la suite se reduce al archivo *test_main.py*, ubicado en la carpeta *keras-api/*, junto al script de producción.

- **Tecnologías utilizadas:** *pytest* como ejecutor del entorno de pruebas y *FastAPI TestClient* como cliente de solicitudes HTTP síncronas. En este módulo es crítico el

uso de *unittest.mock*, empleado para simular el comportamiento de las funciones de Deep Learning.

- **Nomenclatura y casos de prueba:** están estructurados en funciones con el prefijo `test_` seguido del escenario técnico evaluado.

8.2.2. Pruebas de sistema

Estas pruebas se han desarrollado en un repositorio completamente independiente y desacoplado del código de producción del proyecto ³. El propósito de esta separación es evaluar el sistema bajo prueba de forma externa, garantizando que la suite de automatización actúe como un cliente real, sin privilegios.

Se pueden diferenciar dos enfoques de pruebas complementarios:

- Por un lado, las pruebas de API integradas que se fueron desarrollando de manera iterativa a lo largo del desarrollo de la aplicación. A medida que se estabilizaban los controladores y la lógica del servidor, se construían y se enviaban peticiones HTTP a los endpoints para verificar la correcta integración entre los componentes del sistema.
- Por otro lado, las pruebas de interfaz de extremo a extremo (de ahora en adelante E2E), diseñadas y ejecutadas en instancias de un navegador web real. Estas pruebas simulan las interacciones funcionales de un usuario. Su implementación se llevó a cabo siguiendo la técnica Base-choice, detallada en los apartados anteriores de esta memoria.

8.2.2.1. Diseño del Entorno de pruebas

Ubicación y estructura

La automatización de las pruebas está gestionada mediante Maven, y el repositorio se estructura de la siguiente manera:

- **common/:** alberga las clases abstractas. Estos componentes encapsulan el ciclo de vida del navegador web, la orquestación del entorno y las llamadas a la API del backend.
- **pages/:** contiene clases que implementan el patrón de diseño Page Object. Estas clases encapsulan los selectores de los elementos HTML y las interacciones lógicas del DOM. De este modo, las clases de test se abstraen por completo de la estructura y cambios visuales de las plantillas web.
- **tests/:** está dividida en dos directorios:
 - **e2e/:** suites destinadas a la emulación visual y funcional directa sobre instancias del navegador Google Chrome.
 - **api/:** suites orientadas a la validación a nivel de sistema de los *endpoints* de la aplicación mediante la inyección directa de peticiones HTTP.
- **utils/:** contiene clases de soporte logístico, especializadas en la gestión de esperas y sincronización de cargas asíncronas de la interfaz.

³ <https://github.com/augustocristian/retorch-st-devorapp>

Tecnologías utilizadas

La automatización de las pruebas se apoya sobre una pila tecnológica estandarizada y robusta:

- **Java 8 y JUnit:** constituyen el lenguaje base y el framework de pruebas encargado de estructurar el ciclo de vida de los escenarios, coordinar la ejecución y validar los resultados.
- **Selenium WebDriver:** se emplea para manipular, controlar y automatizar las acciones del usuario en el navegador Google Chrome.
- **Selema:** framework auxiliar integrado para simplificar la gestión del ciclo de vida del controlador de Selenium en JUnit. Proporciona capacidades avanzadas como la ejecución en modo oculto, la captura automática de evidencias visuales ante fallos y la integración nativa con Selenoid para la monitorización en tiempo real y grabación de video de los test.
- **Apache HTTPClient y Gson:** librerías utilizadas para interactuar con la API del servidor y Firebase. Permiten registrar, autenticar y limpiar los registros de los usuarios de pruebas de forma rápida, garantizando un entorno aislado.

8.2.2.2. Cobertura de situaciones de prueba

Este apartado introduce la matriz de trazabilidad entre el diseño teórico de las pruebas y su implementación. Esta matriz actúa como un mecanismo de auditoría bidireccional. Por un lado, asegura que cada una de las situaciones de prueba identificadas mediante la técnica Base-Choice en la fase de diseño está cubierta por, al menos, un caso de prueba en la suite. Por otro lado, asegura que no se ha invertido esfuerzo en el desarrollo de código ajeno a los requisitos del sistema.

Inicio de sesión

Tabla 8-1 Casos de prueba inicio de sesión

Caso de prueba	Situaciones cubiertas
testSuccessfullLogin	S2, S7
testLoginValidationErrors	BASE, S3, S4, S5, S6, S8

Registro

Tabla 8-2 Casos de prueba registro

Caso de prueba	Situaciones cubiertas
testRegistroExitosoBase	BASE
testValidacionesPaso1	S2, S3, S4, S5, S6, S7, S8, S9, S10
testValidacionesPaso2PasswordLarga	S11, S12, S13, S14, S15

Búsqueda de restaurantes

Tabla 8-3 Casos de prueba búsqueda de restaurantes

Caso de prueba	Situaciones cubiertas
testBase_BusquedaFiltrosBaseYUbicacionAlternativa	BASE, S8
testFiltrosOpcionales	S2, S3, S4, S5, S6, S7
testS9_OtraUbicacionVacía	S9, S10, S11

Valoraciones

Tabla 8-4 Casos de prueba valoraciones

Caso de prueba	Situaciones cubiertas
testGuardarValoracionCompletaBase	BASE
testValidarCeroEstrellasYComentarioVacio	S2, S5, S8, S11, S14
testPuntuacionesVariables	S3, S4, S6, S7, S9, S10, S12, S13

Favoritos

Tabla 8-5 Casos de prueba favoritos

Caso de prueba	Situaciones cubiertas
testBase_BusquedaListas	BASE
testCasosVacio	S2, S3, S6
testCasosConRestaurantes	S4, S5

Menu lateral

Tabla 8-6 Casos de prueba menú lateral

Caso de prueba	Situaciones cubiertas
testSeleccionTema	BASE, S2
testSeleccionTamanoLetra	S3, S4

Perfil de usuario

Tabla 8-7 Casos de prueba perfil de usuario

Caso de prueba	Situaciones cubiertas
testCargarYGestionarInformacionPersonalYUbicacion	BASE, S2, S3, S4
testGestionarCorreoYContraseña	S5 – S16
testEliminarCuenta	S17

8.2.2.3. Informe de ejecución

Este apartado reúne el catálogo completo de escenarios de prueba diseñados e implementados en la suite automatizada.

Pruebas E2E

Cada caso de prueba consiste en una combinación de las situaciones creadas al aplicar la técnica de diseño *Base-Choice* en apartados anteriores de esta memoria.

Tabla 8-8 Pruebas E2E

Archivo	Número de casos	Casos exitosos	Casos fallidos
TestFavoritesView	3	3	0
TestHistoryView	3	2	1
TestLogin	2	2	0
TestProfileView	3	3	0
TestRatingView	3	3	0
TestRecommendView	3	2	1
TestRegisterView	3	2	1
TestSideMenu	2	2	0

Gracias a la ejecución de estas pruebas, como se puede ver en la Tabla 8-8, se han detectado errores en el desarrollo:

- **Fallo en TestRegisterView:** el caso de prueba detectó que el formulario de registro permitía introducir cualquier cadena de texto aleatoria en el campo de dirección en lugar de seleccionar una localización válida del autocompletado. Esto se debía a la falta de una validación que verificara que el texto correspondiese a una dirección con coordenadas válidas. Para solucionarlo, se bloqueó el avance en el formulario hasta seleccionar una dirección válida.
- **Fallo en TestRecommendView:** al intentar ejecutar una interacción de usuario en la vista de las recomendaciones, el test falló. El caso de prueba mostró un error en las hojas de estilo de CSS, las cuales establecían una altura fija. Ante nombres largos, el texto se desbordaba de los límites y se superponía al botón de “Añadir a favoritos”, bloqueando la interacción. Al cambiar el CSS, el error se solucionó.
- **Fallo en TestHistoryView:** un test falló al validar el número de tarjetas de locales renderizadas, detectando más elementos de los existentes. El caso de prueba mostró que, al cargar la vista, se lanzaban dos peticiones paralelas y, debido a la falta de un controlador de limpieza, se concatenaba la misma respuesta dos veces. Se solucionó al añadir una función de limpieza para abortar peticiones duplicadas.

Pruebas API

El conjunto de casos de prueba implementado a lo largo del desarrollo está formado por:

Tabla 8-9 Pruebas API

Archivo	Número de casos	Casos exitosos	Casos fallidos
TestApiFavoritos	6	6	0
TestApiHistorial	5	5	0
TestApiMasTarde	4	4	0
TestApiProfile	7	6	1
TestApiRecommend	7	7	0
TestApiRegister	6	6	0
TestApiValoraciones	7	6	1

Al igual que en el caso de las pruebas E2E, como se puede ver en la Tabla 8-9, esta suite ha permitido localizar errores cometidos durante el desarrollo:

- **Fallo en TestApiProfile:** el test de la API encargado de la limpieza del entorno fallaba al intentar eliminar la cuenta de un usuario con restaurantes almacenados en favoritos o historial. Esto se debía a la ausencia de *ON DELETE CASCADE* en las claves foráneas de la base de datos. Al añadir dicha cláusula, se solucionó el problema.
- **Fallo en TestApiValoraciones:** el test encargado de validar las puntuaciones falló al comprobar la nota media de un establecimiento tras registrar una valoración con texto y otra sin el comentario escrito. Durante el desarrollo, se añadió una cláusula en la consulta de las valoraciones que excluía aquellas que no contaban con un comentario textual. Este error se corrigió al eliminar dicha cláusula.

Tras aplicar las correcciones descritas, la ejecución completa de ambas suites finaliza sin fallos.

8.2.3. Pruebas estáticas

Con el objetivo de maximizar la calidad y robustez del software desarrollado, se ha integrado una fase de pruebas estáticas mediante SonarQube. El propósito principal es auditar el código fuente sin necesidad de ejecutarlo, permitiendo detectar de forma automatizada vulnerabilidades de seguridad, errores y malas prácticas.

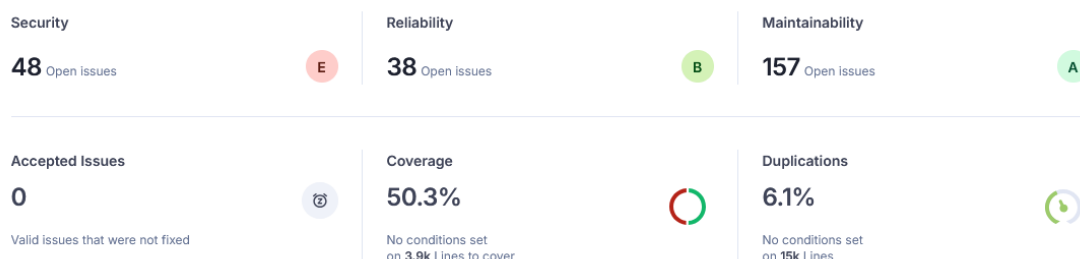


Ilustración 8-1 Pruebas estáticas iniciales

A partir del informe detallado en la Ilustración 8-1, se procede a realizar una fase de refactorización sobre el repositorio. En vista de los resultados y alertas mostradas por el programa, se aplicaron todas las correcciones necesarias solucionar los problemas/smells mostrados. En la Ilustración 8-2 se muestra el estado a día 12 de julio de 2026:

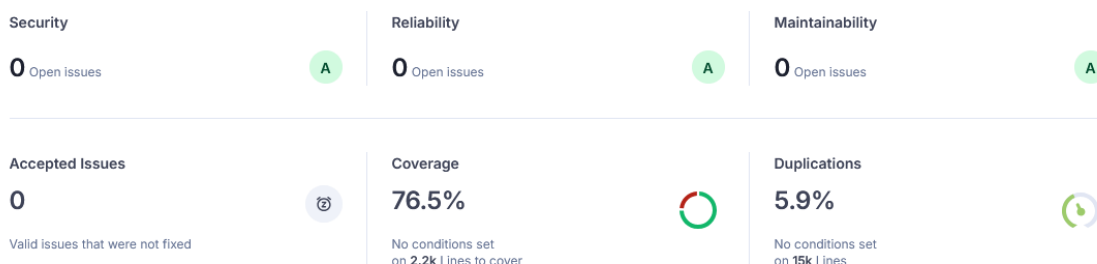


Ilustración 8-2 Pruebas estáticas finales

8.3. DESPLIEGUE

En esta sección se detalla el despliegue de la infraestructura de la aplicación, así como el flujo automatizado diseñado para su integración y entrega. Con el propósito de reducir el riesgo asociado a los desajustes de configuración entre máquinas, se ha adoptado una estrategia basada en contenedores y en la automatización mediante Pipelines CI/CD.

8.3.1. Arquitectura de contenedores

La arquitectura descrita en la fase de diseño se implementa mediante un ecosistema de cuatro contenedores independientes, orquestados con Docker Compose. Esto permite aislar las dependencias y entornos de ejecución de cada componente, garantizando una arquitectura desacoplada.

El ecosistema se organiza a través de una red interna privada. Esta topología aísla los componentes del exterior por motivos de seguridad, tanto la base de datos como la red

neuronal son inaccesibles desde el tráfico externo de internet, permitiendo únicamente comunicaciones internas controladas. Los únicos componentes que exponen sus puertos son el servidor de API (puerto 8000) y la interfaz de usuario (puerto 80).

La configuración y responsabilidades de cada uno de los contenedores se definen de la siguiente manera:

- **Contenedor de base de datos (db):** utiliza la imagen oficial *postgres:16-alpine* debido a su ligereza y eficiencia de almacenamiento. Para contrarrestar la naturaleza volátil de los contenedores, se configura un volumen persistente de Docker vinculado al almacenamiento físico del servidor. A diferencia de las configuraciones convencionales, no se utiliza el directorio *docker-entrypoint-initdb.d/* para la carga inicial. En su lugar, el arranque se sincroniza con el backend mediante una directiva de *healthcheck*, que bloquea los servicios dependientes del puerto 5432 hasta que el motor de base de datos esté totalmente operativo.
- **Contenedor backend (backend):** construido a partir de la imagen *python:3.12-slim*, este servicio aloja la lógica de negocio, estructurada en el framework FastAPI. Su entorno se define mediante un archivo *Dockerfile* que realiza las siguientes operaciones:
 - Instala las herramientas esenciales del sistema para la compilación y la conectividad con PostgreSQL.
 - Incorpora el gestor *Poetry* para garantizar un aislamiento estricto y reproducir las dependencias de Python en producción.
 - Copia el código fuente, los archivos de configuración de identidades y expone el puerto 8000.

El ciclo de vida de este contenedor está determinado por el script *entrypoint.sh*. Al arrancar, se ejecuta el módulo *initialize_db.py*, que inspecciona el estado de la base de datos y actúa según si está vacía o ya existen tablas.

- **Contenedor del motor de recomendación (ia-service):** consiste en un contenedor independiente, basado en la imagen *python:3.12-slim*, que encapsula el microservicio desarrollado. Con el fin de optimizar recursos y reducir el peso en disco, el entorno instala la librería *tensorflow-cpu>=2.15.0* en lugar de utilizar imágenes oficiales. Esto permite que el modelo de recomendación opere aprovechando de forma exclusiva y eficiente la CPU. Este servicio permanece aislado dentro de la red privada e interactúa con el backend mediante endpoints HTTP.
- **Contenedor frontend (frontend):** este servicio se estructura mediante una estrategia de construcción en múltiples etapas dentro de su *Dockerfile*:
 - Fase de construcción: utiliza una imagen *node:22-alpine* para realizar una instalación limpia y segura de las dependencias del cliente y compilar la interfaz de usuario mediante el comando *npm run build*.
 - Fase de producción: instancia una imagen de *nginx:alpine*. El contenedor descarta por completo todo el entorno previo y copia exclusivamente los activos compilados a la ruta nativa de distribución web. Finalmente, inyecta un archivo de configuración de *nginx.conf* para gestionar de forma eficiente el enrutamiento del lado del cliente, la entrega de recursos y el uso del puerto HTTP estándar.

8.3.2. Orquestación de contenedores

Para la gestión, el despliegue y la sincronización de los diferentes subsistemas del proyecto, se ha utilizado la herramienta de orquestación Docker Compose. Con el objetivo de aislar las configuraciones según el ciclo de vida del software, se han definido dos entornos independientes:

- **Entorno de desarrollo (*docker-compose.dev.yml*):** diseñado para acelerar los ciclos de desarrollo. Este archivo sobrescribe los parámetros del archivo principal en tiempo de ejecución, utilizando técnicas de montaje de volúmenes para enlazar el código fuente del disco local con el sistema de archivo interno de los contenedores. Gracias a esto se habilitan los mecanismos de recarga en caliente, reflejando cualquier cambio en el código de forma inmediata.
- **Entorno de producción (*docker-compose.yml*):** configurado para maximizar el rendimiento y la seguridad. A diferencia del entorno de desarrollo, compila las imágenes definitivas y sirve el cliente web a través del servidor web de producción.

Con el objetivo de unificar los flujos de trabajo de los desarrolladores y simular localmente las verificaciones de calidad que posteriormente exigirá el servidor de integración continua, se han diseñado scripts de automatización multiplataforma (*deploy.ps1* para Windows y *deploy.sh* para Unix) destinados a desplegar el SUT en el entorno local. Estos ejecutan una secuencia de tres fases: verificación de prerequisites, validación de la calidad del código y compilación y puesta en marcha.

8.3.3. Integración y despliegue continuo

El ciclo de vida del desarrollo y la gestión de la infraestructura del software han experimentado una evolución a lo largo del proyecto. En las fases iniciales, el desarrollo se encontraba centralizado en un repositorio privado de GitLab proporcionado por la empresa, cuya infraestructura de integración y despliegue continuos fue implementada para cumplir con los requisitos iniciales impuestos.

No obstante, tras recibir autorización para liberar el código con el objetivo de fomentar la transferencia tecnológica y permitir que la comunidad reutilice y contribuya a la plataforma, se procedió a abrir un repositorio público en GitHub.

Con el fin de maximizar la portabilidad y la compatibilidad del sistema en diferentes entornos, se optó por una arquitectura de integración híbrida:

- **Repositorio de desarrollo (GitLab):** tras cada actualización del código del repositorio, se ejecuta un pipeline automatizado en GitLab CI, estructurado secuencialmente en cuatro etapas.
- **Repositorio público (GitHub):** una vez validado el código en GitLab, se sincroniza con el repositorio público de GitHub. En este entorno, se ha replicado y adaptado el mismo flujo de trabajo utilizando GitHub Actions, implementando la compilación y despliegue automático tras pasar las pruebas y garantizando así que cualquier desarrollador externo pueda compilar, probar y desplegar la aplicación de forma nativa e independiente.

Capítulo 9 MANUAL DE USUARIO

En este apartado se construye la guía visual y funcional de DevorApp, diseñada para orientar al usuario final en el uso de las diferentes funcionalidades que componen el ecosistema de la aplicación.

Para el manual del sistema, que contiene tanto de la información de cómo realizar el despliegue como del proceso de descarga de la aplicación, se puede consultar la documentación del repositorio⁴.

Acceso al Sistema

Lo primero que se ve al entrar en la aplicación es la Pantalla 1:

- Por un lado, se puede iniciar sesión escribiendo un correo electrónico o nombre de usuario junto con una contraseña. Tras introducirlos, se debe pulsar el botón de iniciar sesión para entrar en la aplicación.
- Por otro lado, se puede usar el botón “Continuar con Google” para iniciar sesión con una cuenta de Google y evitar el proceso anterior.

Si es la primera vez que se accede a la aplicación, es necesario crearse una nueva cuenta:

- Al pulsar sobre “Regístrate”, se redirige a la Pantalla 2. Aquí se puede crear una cuenta en la aplicación. Para ello, se deben rellenar todos los campos del formulario. Una vez hecho, se debe pulsar sobre el botón “Continuar” para dirigirse a la Pantalla 3. En ella se continúa el registro, o bien introduciendo una ubicación, o usando la función “Usar mi ubicación” para guardar la ubicación actual. Al terminar se debe pulsar el botón “Registrar”, lo que llevara de nuevo a la Pantalla 1 para iniciar sesión con su nueva cuenta.
- Si se pulsa el botón “Continuar con Google” y no tiene una cuenta creada en el sistema, al seleccionar su cuenta le aparecerá un cuadro de texto con dos campos, el nombre de usuario y la ubicación preferida, que deberá rellenar de la misma forma que en el proceso de registro.

Después de este proceso, se redirigirá a la pantalla principal de la aplicación.

⁴ <https://github.com/TorreAlejandro/EPI-DevorApp>

DevorApp

Iniciar sesión
Bienvenido de nuevo

Email o usuario
tu@email.com

Contraseña ¿Olvidaste tu contraseña?
.....

Entrar

Continuar con Google

¿No tienes cuenta? [Regístrate](#)

Pantalla 1 Inicio de sesión

DevorApp

Crear cuenta
Paso 1 de 2 · Datos de acceso

Email
tu@email.com

Nombre de usuario
@usuario

Contraseña
Minimo 8 caracteres

Nombre
Apellidos

Continuar

¿Ya tienes cuenta? [Inicia sesión](#)

Pantalla 2 Registro 1

DevorApp

Crear cuenta
Paso 2 de 2 · Tu ubicación

Tu ubicación preferida
Usaremos esta zona para mostrarte restaurantes cercanos. Podrás cambiarla después desde tu perfil.

Busca tu ubicación
Q Ciudad, barrio o d Usar mi ubicación

Crear cuenta

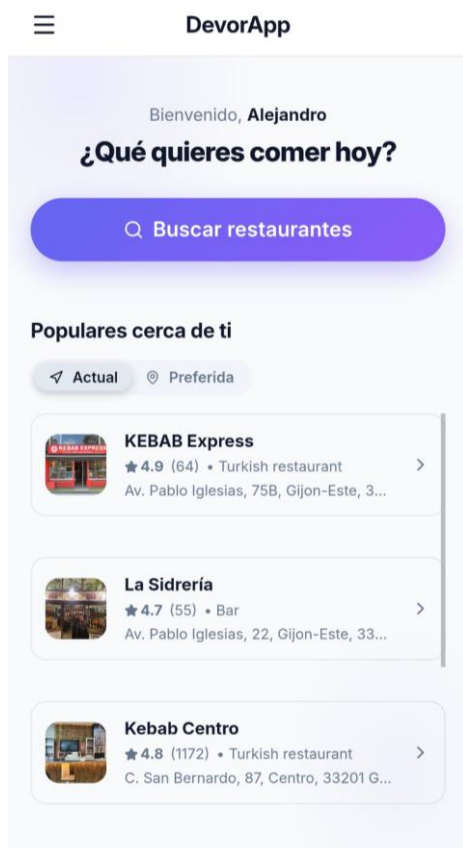
← Volver

Pantalla 3 Registro 2

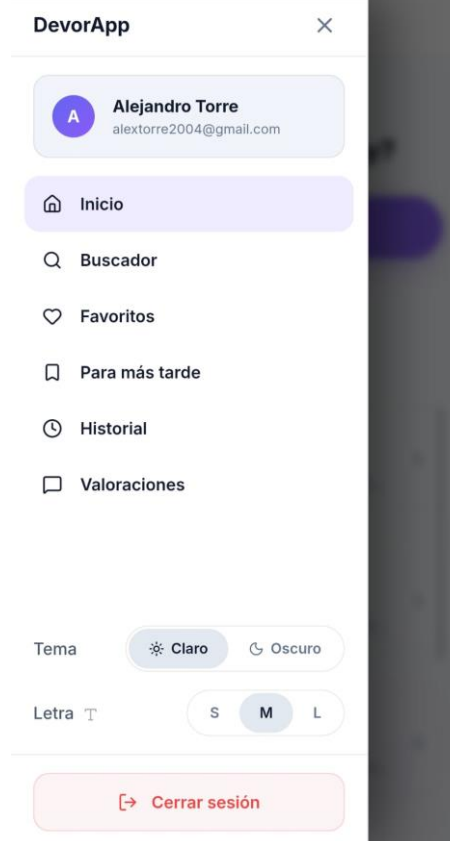
Pantalla principal

Al iniciar sesión, se redirige a la Pantalla 4, donde se pueden encontrar diferentes funcionalidades:

- **Menú lateral:** al pulsar el icono situado en la esquina superior izquierda, se despliega el menú de la Pantalla 5. Desde este menú se pueden acceder a todas las funcionalidades de la aplicación. Además, en la parte inferior se puede cambiar el tema y el tamaño de letra. Finalmente hay un botón “Cerrar sesión” para salir de la aplicación y volver a la Pantalla 1.
- **Acceso directo al motor de recomendación:** pulsar el botón “Buscar Restaurantes” lleva a la pantalla para buscar restaurantes.
- **Sugerencias cerca de ti:** aquí se muestran los restaurantes más visitados en una ubicación. Se puede seleccionar tanto la ubicación actual como la ubicación preferida guardada.



Pantalla 4 Página principal



Pantalla 5 Menú lateral

Buscar un restaurante

En la Pantalla 6 se pueden introducir las preferencias para encontrar restaurantes:

- El campo 1) permite seleccionar etiquetas para especificar el tipo de restaurante.
- A continuación, el campo 2) permite seleccionar el rango de precios que se prefiera, pudiendo seleccionar varios rangos.
- En la sección 3) hay dos casillas. La primera permite incluir restaurantes que no tengan un rango de precios determinado (por lo que puede no coincidir con el rango especificado anteriormente), y la segunda permite incluir solamente locales abiertos en ese momento.
- Finalmente, en el campo 4) se debe especificar la ubicación en la que se realizará la búsqueda. Permite tanto usar la ubicación preferida guardada como escoger una diferente en el momento.

Para obtener los resultados, se debe pulsar el botón “Buscar recomendaciones”, que lleva a la Pantalla 7. Al inicio, 1) se muestran las opciones seleccionadas para confirmar que no hay ningún error, junto con un botón que permite editarlas (volviendo a la Pantalla 6). A continuación, se presenta la lista de 10 restaurantes, ordenados por mejor coincidencia. Si se desea, se puede usar el selector 2) para ordenarlos con otro criterio. Al final de la lista se muestra el número de página actual junto con dos botones para avanzar o retroceder entre los resultados de la búsqueda.

Al pulsar en la tarjeta de un restaurante, se abre la Pantalla 8. Aquí se muestran los detalles del local, comenzando por los datos generales. A continuación, se muestra el horario de

apertura, seguido de una serie de enlaces de interés para obtener más información del restaurante seleccionado. Debajo de ellos se encuentran dos botones: “Seleccionar este restaurante” para escoger el local e indicar a la aplicación que será visitado, y “Guardar para más tarde” para el caso en que el restaurante sea una opción interesante, pero no se desee ir en ese momento. Finalmente, aparecen las reseñas de otros usuarios en la aplicación.

Al seleccionar un restaurante, se vuelve a la Pantalla 4, y en caso de que no se quiera visitar el restaurante, se puede pulsar en “Resultados” arriba a la izquierda para volver a la Pantalla 7.



Pantalla 6 Buscador de restaurantes



Pantalla 7 Resultados de búsqueda

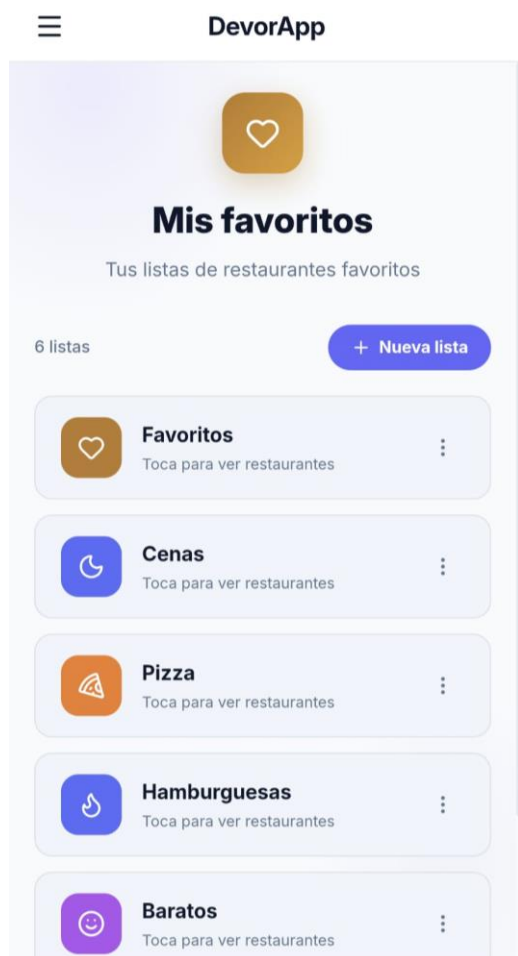


Pantalla 8 Detalles de restaurante

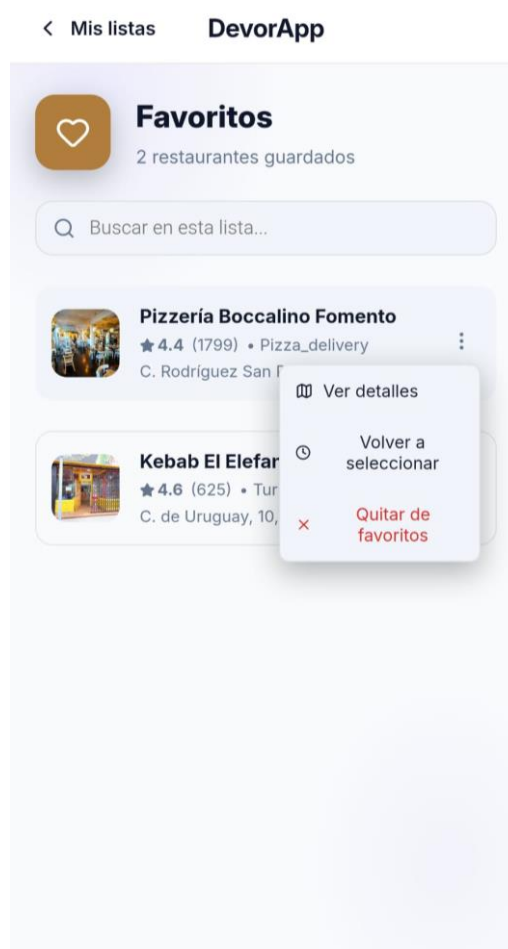
Consultar favoritos

Desde el menú de navegación, mediante la opción “Mis favoritos”, se puede acceder a la Pantalla 9. En ella se pueden ver las listas de restaurantes favoritos guardadas, o añadir una nueva lista con el botón “Nueva lista”.

En una lista (Pantalla 10) se muestran los restaurantes guardados precedidos de una barra de búsqueda para localizar establecimientos específicos. Además de acceder a los detalles del restaurante pulsando sobre él (Pantalla 8), se pueden realizar dos acciones principales con los tres puntos a la derecha de cada tarjeta: seleccionarlo de nuevo y eliminarlo de favoritos. También se puede volver a la pantalla con todas las listas con el botón “Mis listas” en la parte superior izquierda.



Pantalla 9 Listas de Favoritos



Pantalla 10 Restaurantes favoritos

Consultar historial y guardados para más tarde

Al seleccionar “Historial” desde el menú desplegable, se accede a la Pantalla 11, en la que se muestran todos los restaurantes que se han visitado. En la parte superior de la lista se encuentra, al igual que en la Pantalla 10, una barra de búsqueda junto a un botón “Filtrar” que permite alternar entre los restaurantes reseñados y los que aún no tienen valoración. En la lista, los restaurantes se agrupan en desplegables según el mes en que fueron visitados. En este caso, además de ver los detalles y volver a seleccionar un restaurante, se puede añadir a favoritos, dejar o editar una reseña y eliminarlo del historial.

Por otro lado, desde el menú lateral también se accede a la pantalla Pantalla 12 seleccionando “Para más tarde”. Como en otras pantallas, hay una barra de búsqueda seguida de una lista con los restaurantes, a los cuales se puede acceder pulsando sobre ellos. Al final se encuentra un botón “Buscar más restaurantes” para facilitar al usuario la búsqueda en caso de no decidirse entre ninguno de los restaurantes que ha almacenado.



Pantalla 11 Historial



Pantalla 12 Guardar para más tarde

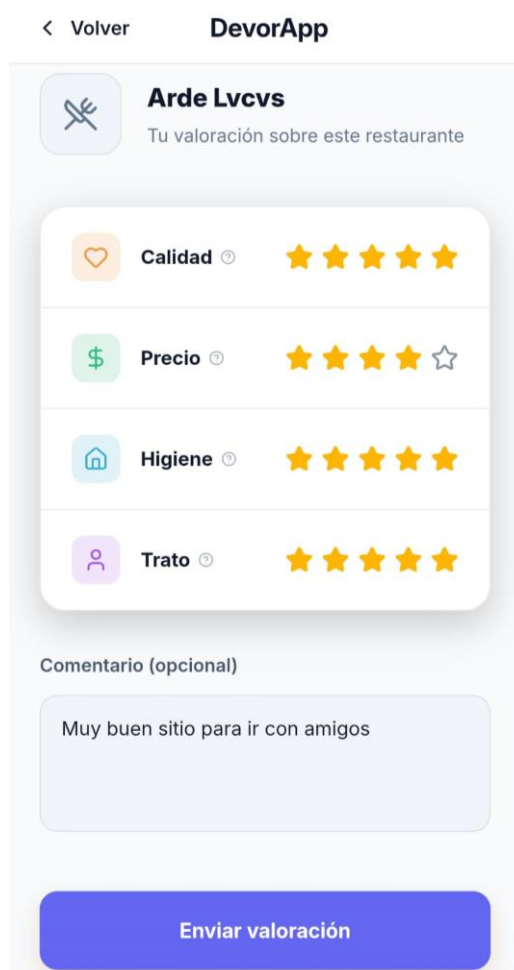
Valoraciones

Desde el menú lateral se puede acceder a la Pantalla 13 al seleccionar “Valoraciones”. En ella se pueden ver todas las reseñas de los locales visitados. Al pulsar en un restaurante, se despliegan los detalles de la reseña, mostrando la valoración desglosada. A continuación, se encuentran dos botones, uno para editar la reseña y otro para eliminarla.

Al pulsar el botón “Editar reseña” se muestra la Pantalla 14. En ella se debe dar una nota del 1 al 5 a cuatro aspectos del restaurante. Cada uno cuenta con un símbolo “?” con la descripción del mismo. De forma adicional, se puede añadir un comentario textual para expresar mejor la experiencia a otros usuarios. La reseña se guarda al pulsar “Enviar valoración”



Pantalla 13 Lista de valoraciones



Pantalla 14 Editar reseña

Perfil de usuario

Desde el menú lateral se puede acceder al perfil de usuario (Pantalla 15). Aquí se pueden cambiar los datos de la cuenta pulsando el botón habilitado para ello (indicado con el icono de un lápiz).

En la parte inferior, el usuario puede eliminar su cuenta de forma permanente. Al pulsar el botón “Eliminar cuenta permanentemente”, se habilita un campo de texto en el que se debe escribir CONFIRMAR y, a continuación, pulsar el botón “Eliminar permanentemente”. En caso de querer volver atrás, junto al botón mencionado anteriormente existe un botón “Cancelar”.



Pantalla 15 Perfil de usuario



Pantalla 16 Zona de peligro

Capítulo 10 CONCLUSIONES Y TRABAJO FUTURO

10.1. CONCLUSIONES

Con la finalización de este Trabajo de Fin de Grado se han alcanzado con éxito los objetivos técnicos y funcionales planteados al inicio del proyecto. Se ha diseñado e implementado una aplicación capaz de mitigar la sobrecarga de información gastronómica mediante un sistema de recomendación personalizado. A pesar de no haber implementado el modelo de lenguaje planteado al comienzo del desarrollo, se adoptó una solución basada en una red neuronal local capaz de generar recomendaciones eficientes y adaptadas a las preferencias del usuario.

Uno de aspectos más destacados de este proyecto ha sido la infraestructura de control de calidad desarrollada. Por un lado, se han integrado pruebas unitarias y de integración en paralelo al desarrollo del proyecto. Por otro lado, se ha implementado una suite de pruebas de API y E2E alojada en un repositorio independiente, que constituye el pilar central de esta infraestructura. Esta separación del código ha permitido aislar el entorno de verificación y facilitar un mantenimiento desacoplado. Además, se han realizado pruebas estáticas automatizadas para garantizar la limpieza del código y el cumplimiento de los estándares.

A nivel personal y académico, este trabajo ha supuesto una inmersión real en la industria, teniendo que enfrentar una incertidumbre inicial debido a la falta de experiencia. No obstante, los conocimientos adquiridos en el grado, tanto en el ámbito de la programación y la implementación de modelos locales, como en los fundamentos para trabajar con Scrum, proporcionaron la base necesaria para superar dicha barrera. Por otro lado, el desarrollo del proyecto también requirió del aprendizaje autónomo de nuevos conocimientos, entre los que destacan el framework React, el diseño de arquitecturas de software y la redacción y composición de memorias técnicas.

En conclusión, este proyecto no es solo una aplicación funcional, sino una solución software sólida, que establece la base adecuada para futuras mejoras y ampliaciones.

10.2. TRABAJO FUTURO

Aunque los objetivos planteados para este proyecto han sido alcanzados satisfactoriamente, existen diversas líneas de mejora y ampliación que podrían incrementar las funcionalidades y el valor de la solución desarrollada. Las limitaciones de tiempo y recursos han condicionado el alcance final de la aplicación, dejando abiertas nuevas posibilidades de evolución.

En este apartado se presentan algunas propuestas de trabajo futuro que permitirían mejorar el sistema de recomendación, optimizar la experiencia de usuario e incorporar nuevas funcionalidades que contribuyan a ofrecer recomendaciones más precisas y personalizadas.

Desafío 1. *Procesamiento de lenguaje natural del buscador.* La implementación actual restringe la exploración de establecimientos a filtros estáticos y categorías predefinidas. Una línea de trabajo futuro consiste en introducir una búsqueda semántica libre mediante procesamiento de lenguaje natural. Esto permitiría al comensal introducir expresiones textuales.

Desafío 2. *Transición a modelos de lenguaje de gran escala.* Aunque la red neuronal implementada ofrece un rendimiento ágil y eficiente, sus predicciones se limitan a patrones de entrada numéricos. Como evolución tecnológica, se propone migrar el microservicio hacia un LLM entrenado para tareas de recomendación. De esta manera, el sistema podrá analizar no solo variables numéricas, sino también el contexto del usuario, logrando un nivel de personalización superior.

Desafío 3. *Desarrollo para iOS.* Se plantea el desarrollo y adaptación de la interfaz de usuario para dispositivos del ecosistema iOS. Esta expansión permitiría llevar la experiencia de la aplicación a usuarios de Apple de manera nativa, aprovechando las capacidades del sistema operativo para integrar nuevas funciones.

Desafío 4. *Página para administradores.* Se plantea el desarrollo de un perfil de administrador para garantizar la moderación de contenido. Esto requerirá ampliar el modelo de datos actual para gestionar un control de acceso basado en roles. El administrador podrá gestionar al resto de usuarios, así como eliminar reseñas fraudulentas o que vulneren las políticas de la comunidad.

Desafío 5. *Permitir reservas desde la aplicación.* Permitir gestionar reservas desde la aplicación evitará que el usuario deba abandonar la plataforma para consolidar su elección. Esto requerirá el diseño de un sistema para el control de aforos y mesas disponibles sincronizado con la base de datos, además de la integración de notificaciones asíncronas para confirmar el estado de la reserva tanto al cliente como al restaurante.

Capítulo 11 BIBLIOGRAFÍA Y APÉNDICES

11.1. REFERENCIAS

- [1] M. A. F. Gámez, S. M. F. Miguélez, R. B. Vicario, and A. R. Trujillo, “Efectos financieros de los comentarios online de los restaurantes”.
- [2] M. Luca, “Reviews, Reputation, and Revenue: The Case of Yelp.Com,” *SSRN Electron. J.*, 2011, doi: 10.2139/ssrn.1928601.
- [3] M. Anderson and J. Magruder, “Learning from the Crowd: Regression Discontinuity Estimates of the Effects of an Online Review Database,” *Econ. J.*, vol. 122, no. 563, pp. 957–989, Sep. 2012, doi: 10.1111/j.1468-0297.2012.02512.x.
- [4] J. A. Chevalier and D. Mayzlin, “The effect of word of mouth on sales”.
- [5] M. R. L. , S. S. Iyengar, “When Choice is Demotivating,” 2000, [Online]. Available: [https://faculty.washington.edu/jdb/345/345%20Articles/Iyengar%20%26%20Lepper%20\(2000\).pdf](https://faculty.washington.edu/jdb/345/345%20Articles/Iyengar%20%26%20Lepper%20(2000).pdf)
- [6] “Solving the Paradox of Choice in Zomato Enhancing User Decision-Making for Long-Term Engagement & Revenue Growth”, [Online]. Available: <https://assets.nextleap.app/submissions/ParadoxOfChoice-0e0ea1d4-6592-46ab-8c03-110494ce821f.pdf>
- [7] “Tipos de lugares (nuevo) | Places API,” Google for Developers. Accessed: May 22, 2026. [Online]. Available: <https://developers.google.com/maps/documentation/places/web-service/place-types?hl=es-419>
- [8] “JavaScript | MDN,” MDN Web Docs. Accessed: Jul. 08, 2026. [Online]. Available: <https://developer.mozilla.org/en-US/docs/Web/JavaScript>
- [9] “Dart documentation.” Accessed: Jul. 08, 2026. [Online]. Available: <https://dart.dev/docs>
- [10] “Introduction · An Introduction to Elm.” Accessed: Jul. 08, 2026. [Online]. Available: <https://guide.elm-lang.org/>
- [11] “The starting point for learning TypeScript.” Accessed: Jul. 08, 2026. [Online]. Available: <https://www.typescriptlang.org/docs/>
- [12] P. Kothiyal, “Backend 2025: Node.js vs Python vs Go vs Java,” Talent500 blog. Accessed: Jul. 08, 2026. [Online]. Available: <https://talent500.com/blog/backend-2025-nodejs-python-go-java-comparison/>
- [13] “Documentation - The Go Programming Language.” Accessed: Jul. 08, 2026. [Online]. Available: <https://go.dev/doc/>
- [14] “Java Documentation,” Oracle Help Center. Accessed: Jul. 08, 2026. [Online]. Available: <https://docs.oracle.com/en/java/>
- [15] “Python 3.14 documentation,” Python documentation. Accessed: Jul. 08, 2026. [Online]. Available: <https://docs.python.org/3/>
- [16] “Python vs Java: Differences and Similarities in 9 Key Areas.” Accessed: Jul. 08, 2026. [Online]. Available: <https://www.datacamp.com/blog/python-vs-java>
- [17] A. Team, “What is Angular? • Angular.” Accessed: Jul. 08, 2026. [Online]. Available: <https://angular.dev/>
- [18] “Vue.js.” Accessed: Jul. 08, 2026. [Online]. Available: <https://vuejs.org/>
- [19] “React.” Accessed: Jul. 08, 2026. [Online]. Available: <https://react.dev/>
- [20] “Django documentation | Django documentation,” Django Project. Accessed: Jul. 08, 2026. [Online]. Available: <https://docs.djangoproject.com/en/6.0/>

- [21] “Welcome to Flask — Flask Documentation (3.1.x).” Accessed: Jul. 08, 2026. [Online]. Available: <https://flask.palletsprojects.com/en/stable/>
- [22] “FastAPI - FastAPI.” Accessed: Jul. 08, 2026. [Online]. Available: <https://fastapi.tiangolo.com/>
- [23] “Getting Started with the Yelp Places API,” Yelp Developers. Accessed: Jul. 08, 2026. [Online]. Available: <https://docs.developer.yelp.com/docs/places-intro>
- [24] “TripAdvisor API — Free Public API | Public APIs Directory,” Public APIs. Accessed: Jul. 08, 2026. [Online]. Available: <https://publicapis.io/trip-advisor-api>
- [25] “Places Overview,” Data Products. Accessed: Jul. 08, 2026. [Online]. Available: <https://docs.foursquare.com/data-products/docs/places-overview>
- [26] “Descripción general | Places API | Google for Developers.” Accessed: Jul. 08, 2026. [Online]. Available: <https://developers.google.com/maps/documentation/places/web-service/overview?hl=es-419>
- [27] “SQL vs NoSQL: Choosing the right database for your project.” Accessed: Jul. 08, 2026. [Online]. Available: <https://appwrite.io/blog/post/sql-vs-nosql>
- [28] “Bases de datos relacionales frente a los no relacionales: diferencia entre los tipos de bases de datos - AWS,” Amazon Web Services, Inc. Accessed: Jul. 08, 2026. [Online]. Available: <https://aws.amazon.com/es/compare/the-difference-between-relational-and-non-relational-databases/>
- [29] “MySQL :: MySQL Documentation.” Accessed: Jul. 08, 2026. [Online]. Available: <https://dev.mysql.com/doc/>
- [30] rwestMSFT, “Documentación de Microsoft SQL - SQL Server.” Accessed: Jul. 08, 2026. [Online]. Available: <https://learn.microsoft.com/es-es/sql/?view=sql-server-ver17>
- [31] “PostgreSQL: Documentation.” Accessed: Jul. 08, 2026. [Online]. Available: <https://www.postgresql.org/docs/>
- [32] “Authentication - OWASP Cheat Sheet Series.” Accessed: Jul. 08, 2026. [Online]. Available: https://cheatsheetseries.owasp.org/cheatsheets/Authentication_Cheat_Sheet.html?utm_source=chatgpt.com
- [33] Supabase, “Auth | Supabase Docs.” Accessed: Jul. 08, 2026. [Online]. Available: <https://supabase.com/docs/guides/auth>
- [34] “Firebase Authentication,” Firebase. Accessed: Jul. 08, 2026. [Online]. Available: <https://firebase.google.com/docs/auth?hl=es-419>
- [35] “Content-Based vs Collaborative Filtering: Difference,” GeeksforGeeks. Accessed: Jul. 08, 2026. [Online]. Available: <https://www.geeksforgeeks.org/machine-learning/content-based-vs-collaborative-filtering-difference/>
- [36] “Keras: Deep Learning for humans.” Accessed: Jul. 08, 2026. [Online]. Available: <https://keras.io/>
- [37] C. R. C. Goodwin Michael, “IaaS, PaaS, SaaS: ¿cuál es la diferencia? | IBM.” Accessed: Jul. 08, 2026. [Online]. Available: <https://www.ibm.com/es-es/think/topics/iaas-paas-saas>
- [38] J. Carlsson, “IaaS vs PaaS soluciones cloud,” Opsio. Accessed: Jul. 08, 2026. [Online]. Available: <https://opsiocloud.com/es/blogs/iaas-vs-paas-que-solucion-en-la-nube-es-la-adecuada-para-tu-empresa-opsio/>
- [39] “Don’t Repeat Yourself en programación | Blog de Arsys,” Arsys. Accessed: May 18, 2026. [Online]. Available: <https://www.arsys.es/blog/dont-repeat-yourself-en-programacion>
- [40] “ISO/IEC/IEEE 29119-2:2021,” ISO. Accessed: Jun. 17, 2026. [Online]. Available: <https://www.iso.org/standard/79428.html>
- [41] S. Eklund, “On the use of Base-Choice strategy for testing industrial control software”.